

PayrollFiti — Complete Build Guide

Engineering Reference

August 2026

Table of Contents

Part 1 — Introduction & Architecture

- 1.1 What We're Building
- 1.2 Tech Stack
- 1.3 Monorepo Layout
- 1.4 Core Design Principles

Part 2 — Scaffolding & Database Schema

- 2.1 Monorepo Setup
- 2.2 Local Infrastructure — Docker Compose
- 2.3 Database Schema — Full Walkthrough

Part 3 — The Payroll Calculation Engine

- 3.1 Package Layout
- 3.2 Core Types
- 3.3 Money & Determinism Helpers
- 3.4 Proration
- 3.5 The Engine Entrypoint
- 3.6 The Registry — Versioned Rules by Country
- 3.7 Kenya (KE) — Two Versions, One Historical Transition
- 3.8 Nigeria (NG)
- 3.9 South Africa (ZA)
- 3.10 Testing the Engine

Part 4 — Backend Foundations

- 4.1 Environment Validation — Fail Fast, Not Silently Wrong
- 4.2 Bootstrap — `main.ts`
- 4.3 `AppModule` — Wiring Order Matters
- 4.4 Authentication — JWT Access + Refresh
- 4.5 Dual-Guard RBAC — Roles + Fine-Grained Permissions
- 4.6 Structural Multi-Tenancy
- 4.7 Field-Level Encryption
- 4.8 Audit Logging

Part 5 — Core Payroll API

- 5.1 Employees Module — The CRUD Template
- 5.2 Payroll Module — Execution, Idempotency, Concurrency
- 5.3 Payslips — Branded PDF Generation
- 5.4 Payroll Calculator & Bank Export

Part 6 — Compliance Reports

- 6.1 Design: Country-Gated, JSON-Driven
- 6.2 Kenya — P9, P10, NSSF & NHIF/SHIF Remittance
- 6.3 Nigeria — PAYE, Pension, NHF Remittance Schedules
- 6.4 South Africa — EMP201 & IRP5
- 6.5 Adding a Fourth Country's Compliance Reports

Part 7 — Billing & Payments

- 7.1 Schema Recap & Provider Abstraction
- 7.2 Paystack Provider
- 7.3 M-Pesa (Daraja) Provider — STK Push
- 7.4 Webhook Verification — Signed vs. Unsigned
- 7.5 The Payment Flow, End to End
- 7.6 Automated Monthly Billing Cycle

Part 8 — Notifications & Background Jobs

- 8.1 Two Independent Queues, One Reason: Keep the Request Thread Free
- 8.2 In-App Notifications + Multi-Channel Fan-Out
- 8.3 Mail Service
- 8.4 SMS — Africa's Talking
- 8.5 Payslip Email — Bulk Dispatch Off the Payroll-Run Thread
- 8.6 Queue Registration

Part 9 — HR Features

- 9.1 Leave Management
- 9.2 Loans & Advances
- 9.3 Public Holidays
- 9.4 Attendance
- 9.5 Employee Documents

Part 10 — Frontend Foundations & Marketing Site

- 10.1 Route Groups: Three Audiences, One App
- 10.2 The API Client — Token Refresh Is Transparent to Every Caller
- 10.3 AuthContext & BrandingContext
- 10.4 Data Fetching — React Query, No Hand-Rolled `useEffect`
- 10.5 The Public Payroll Calculator — Same Engine, Zero Auth
- 10.6 Reviews & Testimonials
- 10.7 Offline Support — Serwist

Part 11 — Frontend Application Pages

- 11.1 The Authenticated Shell
- 11.2 Page Inventory
- 11.3 Template: Payroll Run List → Detail
- 11.4 Template: Create → Approve Mutation Flow (Loans)
- 11.5 Analytics — Real Data, Not Mock Fixtures
- 11.6 Employee Self-Service Portal
- 11.7 Payslip Branding on the Frontend

Part 12 — Testing, DevOps & Environment Reference

- 12.1 Testing Strategy — Three Layers
- 12.2 CI Pipeline
- 12.3 Dockerfiles — Turborepo Prune + Multi-Stage Build
- 12.4 Environment Variable Reference
- 12.5 Local Development Quickstart

Part 13 — Employee Onboarding & Authentication

- 13.1 Who Logs In Where
- 13.2 Employee Invites — From HR Action to Portal Access
- 13.3 Forgot & Reset Password — One Flow for Every Role

Part 14 — Progressive Web App: Install, Manifest & Push

- 14.1 Manifest & Icons — Auditing What's There
- 14.2 The Install Prompt
- 14.3 Web Push — Backend
- 14.4 Web Push — Frontend & the Service Worker

Part 15 — Developer Self-Service: API Keys, Webhooks & the Audit Log

- 15.1 API Keys — Machine Credentials for the Public API
- 15.2 Webhooks — Outbound Event Delivery
- 15.3 The Audit Log — and a Real Leak It Caught

Part 16 — Two-Factor Authentication & Session Management

- 16.1 TOTP Setup — Three Calls, Not One

- 16.2 The Login Flow's Second Step
- 16.3 Sessions — Replacing a Single Shared Token Slot
- 16.4 The Bug This Refactor Found

Part 17 — Payslip Delivery at Scale

- 17.1 One Job Per Run Was the Wrong Grain
- 17.2 Splitting the Job, Splitting the Guarantee

Part 18 — Accounting Sync & the In-App Notification Center

- 18.1 The Notification Bell — Reading What Part 8 Writes
- 18.2 Accounting Integrations — the OAuth Dance
- 18.3 Three Platforms, One Interface
- 18.4 Wired Into Billing and Payroll

Part 19 — The Blog: Sanity CMS, a Custom Dashboard, and SEO

- 19.1 Why a Headless Store, Not a Studio
- 19.2 The Trust Boundary — Two Clients, One Project
- 19.3 The Admin Dashboard
- 19.4 The Public Blog — SSG, ISR, and Structured Data
- 19.5 One-Time Setup — Creating the Sanity Project
- 19.6 One More Gate: Restricting Blog Management to One Tenant

A step-by-step engineering reference for building a multi-country, multi-tenant payroll & HR SaaS platform from scratch.

This guide documents the full architecture of PayrollFiti exactly as implemented: a Turborepo monorepo with a Next.js 15 frontend, a NestJS backend, a PostgreSQL database via Prisma, a pure-function multi-country payroll calculation engine, Paystack and M-Pesa payment integrations, BullMQ background jobs, and a complete set of HR features (leave, loans, attendance, documents, compliance reporting).

It is written to be followed in order — each part builds on the ones before it, the same way the real system was built: schema first, then the pure calculation core, then the API layer, then the integrations, then the UI.

Covers: monorepo scaffolding · database schema · the payroll calculation engine (Kenya, Nigeria, South Africa) · authentication & multi-tenancy · RBAC · core payroll API · compliance reports · billing (Paystack & M-Pesa) · notifications & background jobs · leave, loans, attendance, documents · frontend architecture · testing · DevOps & deployment · employee onboarding, login & password reset · the installable PWA and web push notifications.

Part 1 — Introduction & Architecture

1.1 What We're Building

PayrollFiti is a multi-tenant SaaS platform that runs payroll and manages HR data for companies across Kenya, Nigeria, and South Africa. Each tenant (a customer organization) has one or more companies, employees, salary structures, and its own subscription and billing history — fully isolated from every other tenant in the same database.

The three things that make this a non-trivial system to build correctly are:

1. **Statutory correctness per country.** Kenya, Nigeria, and South Africa each have their own income tax bands, social security schemes, and levies, and those rules *change over time* (Kenya replaced NHIF with SHIF on 1 Oct 2024, for example). The system has to produce the right numbers today and still be able to reproduce a payslip generated under last year's rules.
2. **Tenant isolation.** One Postgres database serves every customer. A bug that lets tenant A see tenant B's employees is the single worst thing this kind of product can do, so isolation has to be structural, not just “remember to add a `WHERE tenantId = ...`” on every query.
3. **Money has to be exactly right, every time, deterministically.** The same inputs must always produce the same payslip, forever, even after code changes — because a payslip is a legal record.

Everything else in the architecture — the monorepo layout, the module boundaries, the queueing, the testing strategy — exists in service of those three constraints.

1.2 Tech Stack

Layer	Choice	Why
Monorepo tooling	Turborepo + pnpm workspaces	Shared packages (<code>payroll-rules</code> , <code>pricing</code> , <code>api</code> types) consumed by both frontend and backend with no publish step
Backend framework	NestJS (Node/TypeScript)	Modules + DI map cleanly onto domain boundaries (employees, payroll, billing, ...); guards/interceptors give first-class hooks for RBAC and tenant scoping
Database	PostgreSQL + Prisma ORM	Prisma's client-extension API lets us enforce tenant scoping <i>structurally</i> (Part 4), not just by convention
Frontend framework	Next.js 15 (App Router)	Route groups cleanly separate the public marketing site from the authenticated app; server components for static marketing pages, client components + React Query for the data-driven app
Data fetching	TanStack React Query	Cache invalidation, optimistic updates, and loading/error state for every API-backed page — no hand-rolled <code>useEffect</code> fetching anywhere
UI	shadcn/ui + Tailwind CSS	Owned component code (not a black-box npm dependency) styled with CSS variables, which is what makes tenant branding and dark mode both trivial
Background jobs	BullMQ (Redis-backed)	Payslip email bulk-dispatch and other async notification work shouldn't block the HTTP request that triggered them
Payments	Paystack (cards/bank, primary) + M-Pesa Daraja (STK push)	Both African-market-first providers, integrated behind one <code>PaymentProvider</code> interface
PDF generation	<code>@react-pdf/renderer</code>	Payslips and compliance reports rendered as React components, so branding (logo, color) is just props
Offline support	Serwist (service worker)	The employee self-service portal must show a cached payslip with no connection
Error tracking	Sentry (DSN-gated, off by default)	Backend exception filter + frontend integration, opt-in via env var so local dev never phones home

1.3 Monorepo Layout

```

payrollpro/
├─ apps/
│   ├─ web/                # Next.js 15 App Router frontend
│   │   └─ app/
│   │       └─ (marketing)/ # public site: /, /pricing, /reviews, /contact, ...
│   │       └─ (app)/       # authenticated app: /dashboard, /employees, /payroll, ...
│   │   └─ components/
│   │   └─ lib/             # one *-api.ts file per backend module, React Query hooks
│   │   └─ contexts/        # AuthContext, BrandingContext
│   └─ api/                 # NestJS backend
│       └─ prisma/
│           └─ schema.prisma
│           └─ migrations/
│       └─ src/
│           └─ auth/ employees/ payroll/ payslips/ billing/ leave/ loans/ ...
│           └─ common/      # guards, decorators, tenant context, encryption
│           └─ prisma/      # PrismaService + tenant-scoping extension
├─ packages/
│   └─ payroll-rules/       # pure calculation engine – zero NestJS/DB dependency
│   └─ pricing/            # per-country plan catalog shared by API and web
│   └─ api/                # shared request/response TypeScript types
├─ docker-compose.yml
└─ turbo.json

```

The critical architectural decision here is that `packages/payroll-rules` depends on **nothing** — no Prisma, no NestJS, no HTTP. It is pure TypeScript functions operating on plain objects. That is what makes it possible to unit-test statutory tax logic exhaustively without spinning up a database, and to reuse the exact same calculation code in the public, unauthenticated payroll calculator on the marketing site and inside the real backend payroll run.

1.4 Core Design Principles

These four principles recur throughout the guide — understanding them up front makes the rest of the implementation read as inevitable rather than arbitrary.

Pure calculation core. `runPayrollCalculation(input, ruleSet)` (Part 3) has no side effects: no clock reads, no I/O, no randomness. Given the same input and ruleset it always returns the same result. This is what lets the system hash `(input, ruleVersion)` into a deterministic `inputHash` and use that as an idempotency key — re-submitting the same payroll run request twice never double-processes it.

Versioned country rules, resolved by effective date. Every country's rules are a *list* of `CountryRuleSet` objects, each with an `effectiveFrom` date. Resolving “which rules apply” always takes an explicit date (the payroll period, not `Date.now()`), so a payroll run from March 2024 recomputed in 2026 still uses the March 2024 rules. New legislation is added as a new version, never by mutating the old one.

Structural multi-tenancy. Tenant scoping is enforced by a Prisma client extension that reads the current tenant out of `AsyncLocalStorage` -backed request context and automatically injects `tenantId` into every query — so a developer who forgets to filter by tenant gets safe behavior by default rather than a silent data leak. Manual `assertCompanyBelongsToTenant` -style checks fill the gap for relations that don't carry `tenantId` directly (Part 4).

Dual-guard RBAC. Two independent guards run on every request: `RolesGuard` (coarse: ADMIN/HR/EMPLOYEE) and `PermissionsGuard` (fine-grained: `payroll:run`, `employees:write`, ...). They're both registered globally and both must pass — they are not alternatives to each other (Part 4).

The rest of this guide builds the system in the order it was actually built: schema → pure engine → backend foundations → API modules → integrations → frontend → tests → deployment.

Part 2 — Scaffolding & Database Schema

2.1 Monorepo Setup

Initialize the workspace with pnpm and Turborepo:

```
mkdir payrollfiti && cd payrollfiti
pnpm init
pnpm add -Dw turbo typescript prettier eslint

mkdir -p apps/web apps/api packages/payroll-rules packages/pricing packages/api
```

`pnpm-workspace.yaml`:

```
packages:
  - "apps/*"
  - "packages/*"
```

Root `package.json` scripts delegate everything to Turborepo, which fans work out to whichever workspaces define the matching script and caches results by content hash:

```
{
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "db:generate": "turbo run db:generate",
    "db:push": "turbo run db:push"
  }
}
```

`turbo.json` declares every environment variable the tasks depend on (so Turborepo's cache correctly invalidates when they change) and the output globs to cache per task:

```

{
  "$schema": "https://turborepo.com/schema.json",
  "globalDependencies": ["**/.env.*local"],
  "globalEnv": [
    "APP_NAME", "NODE_ENV", "PORT", "COUNTRY_DEFAULT", "CORS_ORIGIN",
    "THROTTLE_TTL", "THROTTLE_LIMIT",
    "JWT_ACCESS_SECRET", "JWT_ACCESS_EXPIRES_IN",
    "JWT_REFRESH_SECRET", "JWT_REFRESH_EXPIRES_IN",
    "PAYSTACK_SECRET_KEY",
    "MPESA_CONSUMER_KEY", "MPESA_CONSUMER_SECRET", "MPESA_SHORTCODE",
    "MPESA_PASSKEY", "MPESA_ENV", "MPESA_CALLBACK_URL", "MPESA_CALLBACK_TOKEN",
    "REDIS_URL", "PAYSLIP_STORAGE_DIR", "DOCUMENT_STORAGE_DIR",
    "DATABASE_URL", "ENCRYPTION_KEY",
    "SMTP_HOST", "SMTP_PORT", "SMTP_USER", "SMTP_PASS", "SMTP_FROM",
    "AFRICAS_TALKING_API_KEY", "AFRICAS_TALKING_USERNAME", "AFRICAS_TALKING_SENDER_ID",
    "SENTRY_DSN"
  ],
  "tasks": {
    "dev": { "cache": false, "persistent": true },
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "!next/cache/**"],
      "env": ["NEXT_PUBLIC_API_URL", "NEXT_PUBLIC_APP_NAME"]
    },
    "lint": {},
    "test": {},
    "test:e2e": {},
    "db:generate": { "cache": false },
    "db:push": { "cache": false },
    "db:seed": { "cache": false }
  }
}

```

Scaffold the two apps:

```

cd apps/web && pnpm create next-app@latest . --typescript --tailwind --app --turbopack
cd ../api && pnpm dlx @nestjs/cli new . --skip-git --package-manager pnpm

```

2.2 Local Infrastructure — Docker Compose

Postgres and Redis run in containers even in local dev; the API and web images are built the same way for a `docker compose up` production-like smoke test. The key detail worth calling out is the healthchecks: neither the `node:20-alpine` API image nor the web image ships `curl` or `wget`, so healthchecks shell out to Node's built-in `http` module instead of pulling in a package just for this:


```

name: payrollfiti

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-payrollfiti}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-payrollfiti}
      POSTGRES_DB: ${POSTGRES_DB:-payrollfiti}
    ports: ["5432:5432"]
    volumes: ["db-data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-payrollfiti}"]
      interval: 5s
      timeout: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    build: { context: ., dockerfile: apps/api/Dockerfile }
    environment:
      # ...full env list, see Part 12 Appendix
      DATABASE_URL: postgresql://${POSTGRES_USER:-payrollfiti}:${POSTGRES_PASSWORD:-payrollfiti}@db:5432/${POSTGRES_DB:-payrollfiti}
      REDIS_URL: redis://redis:6379
    ports: ["3000:3000"]
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD-SHELL", "node -e \"require('http').get('http://localhost:3000/health', res => process.exit(res.statusCode === 200 ? 0 : 1)).on('error', () => process.exit(1))\""]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 15s

  web:
    build:
      context: .
      dockerfile: apps/web/Dockerfile
    args:
      NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL:-http://localhost:3000}
    ports: ["3001:3001"]
    depends_on:
      api: { condition: service_healthy }
    healthcheck:
      test: ["CMD-SHELL", "node -e \"require('http').get('http://localhost:3001/', res => process.exit(res.statusCode < 500 ? 0 : 1)).on('error', () => process.exit(1))\""]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 15s

volumes:
  db-data:

```

Note `web`’s `depends_on` uses `condition: service_healthy` on `api`, not just “started” — the frontend container won’t be marked ready by Compose until the API has actually passed its own healthcheck (Postgres + Redis reachable), which matters for any orchestration that waits on service health before routing traffic.

2.3 Database Schema — Full Walkthrough

The schema is organized around six clusters. Read them in this order — each cluster depends conceptually on the one before it.

Cluster 1: Tenancy & Identity

```
enum Role {
  ADMIN
  HR
  EMPLOYEE
}

model Tenant {
  id          String      @id @default(uuid())
  name        String
  countryCode  String
  defaultCurrency String   @default("KES")
  settings    Json?
  branding    BrandingConfig?
  companies   Company[]
  users       User[]
  subscription Subscription?
  // ...invoices, usageRecords, auditLogs, notifications, apiKeys,
  //   webhooksEndpoints, salaryComponents, leaveTypes, loans
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
}
```

Tenant is the SaaS customer boundary — every row that needs isolation carries a **tenantId** back to it, directly or through a parent. **Scope decision:** a tenant runs payroll in exactly one currency, derived from its **countryCode** via the shared pricing catalog. There is no FX conversion logic anywhere in the codebase; **currency** fields on **Company**, **Employee**, **SalaryStructure**, **PayrollRun**, etc. are accounting labels, not convertible amounts. **PayrollService** actively rejects running a payroll for an employee whose salary-structure currency doesn't match the run's country currency, rather than silently mixing currencies into one run's totals.

```

model BrandingConfig {
  id          String @id @default(uuid())
  tenantId    String @unique
  tenant      Tenant  @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  appName     String?
  logoUrl     String?
  primaryColor String?
  secondaryColor String?
}

model Company {
  id          String      @id @default(uuid())
  tenantId    String
  tenant      Tenant      @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  name        String
  currency    String      @default("KES")
  employees   Employee[]
  payrollRuns PayrollRun[]

  @@index([tenantId])
}

model User {
  id          String      @id @default(uuid())
  tenantId    String
  tenant      Tenant      @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  email       String      @unique
  passwordHash String
  role        Role        @default(EMPLOYEE)
  isActive    Boolean     @default(true)
  refreshTokenHash String?
  phone       String?     // E.164; SMS delivery target only
  employeeId  String?     @unique
  employee    Employee?   @relation(fields: [employeeId], references: [id])
  // ...auditLogs, notifications, apiKeysCreated, approved/recorded relations

  @@index([tenantId])
}

```

BrandingConfig is a per-tenant white-label override — payslip PDFs and the app shell fall back to the **APP_NAME** env var when it’s absent.

User.employeeId is the load-bearing detail for the employee self-service portal: when a user’s **role** is **EMPLOYEE**, this optional 1:1 link lets “my payslips” / “my leave requests” endpoints resolve the caller’s own employee record without trusting a client-supplied ID.

Cluster 2: Employee Records

```

enum EmploymentType { PERMANENT CONTRACT CASUAL INTERN }

model Employee {
  id          String          @id @default(uuid())
  companyId   String
  company     Company         @relation(fields: [companyId], references: [id], onDelete: Cascade)
  employeeNumber String?
  firstName   String
  lastName    String
  email       String          @unique
  kraPin      String?
  nssfNumber  String?
  nhifNumber  String?
  taxIdNumber String?         // Nigeria TIN / SA SARS reference – non-KE countries
  pensionNumber String?       // Nigeria RSA PIN – non-KE countries
  jobRole     String?
  department  String?
  employmentType EmploymentType @default(PERMANENT)
  currency    String          @default("KES")
  status      String          @default("ACTIVE")
  terminatedAt DateTime?
  terminationReason String?
  bankName    String?
  bankAccountNumber String?
  bankCode    String?
  bankBranchCode String?
  user        User?
  contracts   Contract[]
  salaryStructures SalaryStructure[]
  documents   Document[]
  payrollEntries PayrollEntry[]
  leaveBalances LeaveBalance[]
  leaveRequests LeaveRequest[]
  attendanceRecords AttendanceRecord[]
  loans       Loan[]
  onboardingTasks OnboardingTask[]

  @@index([companyId])
}

```

Statutory ID fields are deliberately split: `kraPin` / `nssfNumber` / `nhifNumber` are Kenya-specific (named after their actual Kenyan institutions), while `taxIdNumber` / `pensionNumber` are the generic equivalents used by Nigeria and South Africa. This keeps the schema honest about which fields are truly country-agnostic versus which encode Kenya-specific naming, without needing a fully generic key-value bag.

```

model OnboardingTask {
  id          String          @id @default(uuid())
  employeeId   String
  employee     Employee       @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  title        String
  isRequired   Boolean        @default(true)
  completed    Boolean        @default(false)
  completedAt  DateTime?
  order        Int            @default(0)

  @@index([employeeId])
}

model Contract {
  id          String          @id @default(uuid())
  employeeId   String
  employee     Employee       @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  type         EmploymentType
  startDate    DateTime
  endDate      DateTime?
  terms        Json?

  @@index([employeeId])
}

```

A default onboarding checklist (universal items + country-specific ones, e.g. “collect KRA PIN” for a KE employee) is seeded whenever an employee is created. An employee stays in an **ONBOARDING** status — excluded from payroll runs and from active-employee billing counts — until every **isRequired** task is complete.

Cluster 3: Compensation

```
enum SalaryComponentType { EARNING DEDUCTION }
enum SalaryComponentCalcType { FIXED PERCENTAGE_OF_BASIC }

model SalaryStructure {
  id          String      @id @default(uuid())
  employeeId  String
  employee    Employee    @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  basicSalary Float
  allowances  Json?       // LEGACY — kept for backward compat; see SalaryComponent fallback
  currency    String
  effectiveFrom DateTime
  effectiveTo  DateTime?
  components  SalaryStructureComponent[]

  @@index([employeeId, effectiveFrom])
}

model SalaryComponent {
  id          String      @id @default(uuid())
  tenantId    String
  tenant      Tenant      @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  name        String
  code        String      // stable key, e.g. "TRANSPORT"
  type        SalaryComponentType
  calcType    SalaryComponentCalcType @default(FIXED)
  isTaxable   Boolean      @default(true)
  isActive    Boolean      @default(true)
  defaultAmount Float?     // used when calcType = FIXED
  defaultRate  Float?      // percentage points, when calcType = PERCENTAGE_OF_BASIC
  structureLines SalaryStructureComponent[]

  @@unique([tenantId, code])
  @@index([tenantId])
}

model SalaryStructureComponent {
  id          String      @id @default(uuid())
  salaryStructureId String
  salaryStructure SalaryStructure @relation(fields: [salaryStructureId], references: [id], onDelete: Cascade)
  salaryComponentId String
  salaryComponent SalaryComponent @relation(fields: [salaryComponentId], references: [id])
  amount       Float?     // overrides defaultAmount
  rate         Float?     // overrides defaultRate

  @@unique([salaryStructureId, salaryComponentId])
  @@index([salaryStructureId])
}
```

SalaryStructure is versioned by **effectiveFrom** / **effectiveTo** rather than mutated in place — a payslip generated last month must keep reflecting the salary that was actually in effect then, even after a raise takes effect this month. **SalaryComponent** is a tenant-defined catalog of named earnings/deductions (transport allowance, union dues, etc.) that a company can attach per employee via **SalaryStructureComponent**, each optionally overriding the component’s tenant-wide default amount or rate. The legacy **allowances** JSON field on **SalaryStructure** predates this component system and is kept only as a fallback read path for structures created before it existed.

Cluster 4: Payroll Execution

```

enum PayrollRunStatus { DRAFT PROCESSING COMPLETED FAILED }

model PayrollRun {
  id          String          @id @default(uuid())
  companyId   String
  company     Company          @relation(fields: [companyId], references: [id], onDelete: Cascade)
  period      String
  periodStart DateTime
  periodEnd   DateTime
  countryCode String
  currency    String
  ruleVersion String
  status      PayrollRunStatus @default(DRAFT)
  idempotencyKey String        @unique
  totals      Json?
  initiatedById String?
  isOffCycle  Boolean          @default(false)
  reason      String?
  entries     PayrollEntry[]

  @@index([companyId, period])
}

model PayrollEntry {
  id          String          @id @default(uuid())
  payrollRunId String
  payrollRun  PayrollRun @relation(fields: [payrollRunId], references: [id], onDelete: Cascade)
  employeeId  String
  employee    Employee       @relation(fields: [employeeId], references: [id])
  currency    String
  prorationFactor Float      @default(1)
  grossPay    Float
  totalTax    Float
  totalStatutoryDeductions Float
  totalVoluntaryDeductions Float
  totalDeductions Float
  netPay      Float
  earningsBreakdown Json
  statutoryDeductions Json
  taxBreakdown  Json
  inputHash     String
  payslip       Payslip?
  correctionsAsOriginal PayrollCorrection[] @relation("OriginalEntry")
  correctionAsCorrected PayrollCorrection? @relation("CorrectedEntry")
  loanRepayments LoanRepayment[]

  @@index([payrollRunId])
  @@index([employeeId])
}

```

`PayrollRun.idempotencyKey` is `@unique` — re-submitting the same run request returns the existing run instead of double-processing.

`PayrollEntry` is deliberately country-agnostic: it never encodes “NSSF” or “PAYE” as columns — that detail lives inside the `earningsBreakdown` / `statutoryDeductions` / `taxBreakdown` JSON blobs, whose *shape* is produced by whichever `CountryRuleSet` ran (Part 3). This is what lets one schema serve three (and eventually more) countries without a migration per country.

```

model PayrollCorrection {
  id          String          @id @default(uuid())
  originalEntryId String
  originalEntry PayrollEntry @relation("OriginalEntry", fields: [originalEntryId], references: [id], onDelete: Cascade)
  correctedEntryId String      @unique
  correctedEntry PayrollEntry @relation("CorrectedEntry", fields: [correctedEntryId], references: [id], onDelete: Cascade)
  reason        String
  createdById   String?

  @@index([originalEntryId])
}

model Payslip {
  id          String          @id @default(uuid())
  payrollEntryId String      @unique
  payrollEntry PayrollEntry @relation(fields: [payrollEntryId], references: [id], onDelete: Cascade)
  pdfPath     String?
  generatedAt  DateTime       @default(now())
}

```

When a mistake is discovered after a run completes, the fix is a *new* off-cycle **PayrollEntry** linked back via **PayrollCorrection** — the original entry is never mutated. This preserves an immutable audit trail of what was actually paid, and when, and why it was corrected.

Cluster 5: HR Operations

```

enum LeaveRequestStatus { PENDING APPROVED REJECTED }
enum AttendanceStatus { PRESENT ABSENT LEAVE }
enum LoanStatus { PENDING REJECTED ACTIVE PAID_OFF }
enum LoanRepaymentStatus { PENDING PAID SKIPPED }

model LeaveType {
  id          String @id @default(uuid())
  tenantId    String
  tenant      Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  name        String
  daysPerYear Float
  isPaid      Boolean @default(true)
  isActive    Boolean @default(true)
  balances    LeaveBalance[]
  requests    LeaveRequest[]

  @@unique([tenantId, name])
}

model LeaveBalance {
  id          String @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  leaveTypeId String
  leaveType   LeaveType @relation(fields: [leaveTypeId], references: [id], onDelete: Cascade)
  year        Int
  accruedDays Float @default(0)
  usedDays    Float @default(0)

  @@unique([employeeId, leaveTypeId, year])
}

model LeaveRequest {
  id          String @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  leaveTypeId String
  leaveType   LeaveType @relation(fields: [leaveTypeId], references: [id])
  startDate   DateTime
  endDate     DateTime
  daysRequested Float
  reason       String?
  status       LeaveRequestStatus @default(PENDING)
  approverId  String?
  approver    User? @relation(fields: [approverId], references: [id])
  decidedAt   DateTime?

  @@index([employeeId, status])
}

model Loan {
  id          String @id @default(uuid())
  tenantId    String
  tenant      Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  principal   Float
  currency    String
  installments Int
  installmentAmount Float?
  startPeriod String // "YYYY-MM" - first period the deduction applies from once approved
  reason       String?
  status       LoanStatus @default(PENDING)
  requestedById String?
  requestedBy  User? @relation("LoanRequestedBy", fields: [requestedById], references: [id])
  approvedById String?
  approvedBy  User? @relation("LoanApprovedBy", fields: [approvedById], references: [id])
  decidedAt   DateTime?
  closedAt    DateTime?
  repayments  LoanRepayment[]

  @@index([employeeId, status])
}

```



```

}

model LoanRepayment {
  id          String          @id @default(uuid())
  loanId      String
  loan        Loan            @relation(fields: [loanId], references: [id], onDelete: Cascade)
  installmentNo Int
  period      String
  amountDue   Float
  amountPaid  Float           @default(0)
  status      LoanRepaymentStatus @default(PENDING)
  payrollEntryId String?
  payrollEntry PayrollEntry?   @relation(fields: [payrollEntryId], references: [id])
  paidAt      DateTime?

  @@unique([loanId, installmentNo])
  @@index([period, status])
}

```

`installmentAmount` and the full `LoanRepayment` schedule are only generated once a loan transitions to `ACTIVE` (approved) — a `REJECTED` loan never ties up a future payroll period. `LoanRepayment.payrollEntryId` is set only once an installment is actually deducted through a real payroll run; an early payoff instead marks the remaining rows `SKIPPED` without ever populating it, so the schema can always answer “was this installment actually paid through payroll, or written off?”

```

model Document {
  id          String          @id @default(uuid())
  employeeId  String
  employee    Employee        @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  type        String
  fileName    String
  mimeType    String
  sizeBytes   Int
  url         String          // relative path under DOCUMENT_STORAGE_DIR
  uploadedById String?
  uploadedBy  User?           @relation(fields: [uploadedById], references: [id])
  uploadedAt  DateTime        @default(now())

  @@index([employeeId, type])
}

model AttendanceRecord {
  id          String          @id @default(uuid())
  employeeId  String
  employee    Employee        @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  date        DateTime
  status      AttendanceStatus
  notes       String?
  recordedById String?
  recordedBy  User?           @relation(fields: [recordedById], references: [id])

  @@unique([employeeId, date])
}

```

Cluster 6: Billing, Audit, Platform

```

enum SubscriptionStatus { TRIALING ACTIVE PAST_DUE CANCELED }
enum InvoiceStatus { DRAFT OPEN PAID VOID UNCOLLECTIBLE }
enum PaymentProviderType { PAYSTACK MPESA }

model Plan {
  id String @id @default(uuid())
  code String @unique
  name String
  pricePerEmployee Float
  currency String @default("USD")
  tier String?
  countryCode String? // null = legacy/global plan not tied to a country catalog
  isActive Boolean @default(true)
  subscriptions Subscription[]

  @@index([countryCode])
}

model Subscription {
  id String @id @default(uuid())
  tenantId String @unique
  tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  planId String
  plan Plan @relation(fields: [planId], references: [id])
  status SubscriptionStatus @default(TRIALING)
  provider PaymentProviderType @default(PAYSTACK)
  providerCustomerId String?
  providerSubscriptionId String?
  currentPeriodStart DateTime
  currentPeriodEnd DateTime
  invoices Invoice[]
}

model Invoice {
  id String @id @default(uuid())
  tenantId String
  tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  subscriptionId String
  subscription Subscription @relation(fields: [subscriptionId], references: [id])
  amount Float
  currency String
  status InvoiceStatus @default(DRAFT)
  dueDate DateTime
  paidAt DateTime?
  provider PaymentProviderType @default(PAYSTACK)
  providerInvoiceId String?
  transactions PaymentTransaction[]

  @@index([tenantId, status])
}

model UsageRecord {
  id String @id @default(uuid())
  tenantId String
  tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  period String
  employeeCount Int
  computedAt DateTime @default(now())

  @@unique([tenantId, period])
}

// Raw log of provider-side attempts against an invoice, kept for
// reconciliation independent of invoice.status.
model PaymentTransaction {
  id String @id @default(uuid())
  invoiceId String
  invoice Invoice @relation(fields: [invoiceId], references: [id], onDelete: Cascade)
  provider PaymentProviderType
  reference String
  amount Float
  currency String

```

```
status      String
rawResponse Json?

@@index([invoiceId])
}
```

```

// Append-only trail of who changed what. Interceptor-populated (Part 4),
// never updated or deleted by application code.
model AuditLog {
  id          String    @id @default(uuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  actorId     String?
  actor       User?     @relation(fields: [actorId], references: [id])
  action      String
  entityType  String
  entityId    String
  before      Json?
  after       Json?
  ipAddress   String?

  @@index([tenantId, entityType, entityId])
  @@index([tenantId, createdAt])
}

model Notification {
  id          String    @id @default(uuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  type        String
  message     String
  read        Boolean @default(false)
  metadata    Json?

  @@index([userId, read])
}

// Machine credential for the read-only public API. hashedKey is a one-way
// SHA-256 digest of a server-generated high-entropy token – the raw key is
// never stored (Part 9).
model ApiKey {
  id          String    @id @default(uuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  name        String
  hashedKey   String    @unique
  keyPrefix   String
  lastUsedAt  DateTime?
  revokedAt   DateTime?
  createdById String?
  createdBy   User?     @relation(fields: [createdById], references: [id])

  @@index([tenantId])
}

// Tenant-configured outbound webhook subscription. `secret` is stored
// retrievable – unlike ApiKey.hashedKey – because the server uses it to SIGN
// outgoing deliveries rather than verify an inbound presentation of it.
model WebhookEndpoint {
  id          String    @id @default(uuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  url         String
  secret      String
  events      String[]
  isActive    Boolean   @default(true)
  deliveries  WebhookDeliveryLog[]

  @@index([tenantId])
}

model WebhookDeliveryLog {
  id          String    @id @default(uuid())
  webhookEndpointId String
  webhookEndpoint WebhookEndpoint @relation(fields: [webhookEndpointId], references: [id], onDelete: Cascade)
  event       String

```

```
payload      Json
statusCode   Int?
success      Boolean
error        String?

@@index([webhookEndpointId, createdAt])
}
```

Migrations

Every schema change is a Prisma migration, generated and applied together — never edited by hand after generation:

```
pnpm --filter api exec prisma migrate dev --name add_employee_department
```

This writes a timestamped SQL file under `apps/api/prisma/migrations/` and applies it to the local database in one step. In CI/production, `prisma migrate deploy` applies any pending migrations without generating new ones or prompting — the deploy pipeline never has interactive access to decide on schema changes, only to apply already-reviewed ones.

Part 3 — The Payroll Calculation Engine

This is the architectural core of the product: `packages/payroll-rules`. It has zero dependency on NestJS, Prisma, or HTTP — every function here is pure, so it can be tested exhaustively and reused anywhere (the real payroll run, the public marketing-site calculator, a future CLI).

3.1 Package Layout

```
packages/payroll-rules/src/
├─ types.ts           # every shared interface
├─ money.ts           # round2/round4/sum/stableStringify
├─ proration.ts       # mid-period joiner/leaver math
├─ engine.ts          # runPayrollCalculation – the single endpoint
├─ registry.ts        # country + version resolution
├─ index.ts           # public exports
└─ countries/
    ├─ kenya/         # paye.ts nssf.ts nhif.ts shif.ts housingLevy.ts constants.ts index.ts
    ├─ nigeria/       # paye.ts pension.ts nhf.ts constants.ts index.ts
    └─ south-africa/  # paye.ts uif.ts sdl.ts constants.ts index.ts
```

3.2 Core Types

Every country implementation satisfies the same `CountryRuleSet` contract, which is what makes the engine and registry country-agnostic:

```
// types.ts
export interface EarningsInput {
  basicSalary: number;
  allowances?: Record<string, number>;
  overtimeAmount?: number;
  commissionAmount?: number;
  bonusAmount?: number;
}

export interface EmploymentPeriod {
  periodStart: string; // ISO date
  periodEnd: string;
  employmentStartDate?: string; // set for mid-period joiners
  employmentEndDate?: string; // set for mid-period leavers
}

export interface PayrollCalculationInput {
  employeeId: string;
  countryCode: string;
  currency: string;
  earnings: EarningsInput;
  deductions?: { voluntary?: Record<string, number> };
  period: EmploymentPeriod;
}

export interface StatutoryDeductionLine {
  code: string; // "NSSF", "PENSION", "UIF", ...
  label: string;
  employeeAmount: number;
  employerAmount: number;
}

export interface TaxResult {
  code: string; // "PAYE"
  taxableIncome: number;
  grossTax: number;
  relief: number;
  netTax: number;
}

/**
 * Strategy-pattern contract every country payroll module implements.
 * Implementations must be pure – no I/O, no clock reads, no randomness –
 * so a given (input, ruleset) pair always yields the same result.
 */
export interface CountryRuleSet {
  countryCode: string;
  version: string; // opaque tag recorded on every payroll run
  effectiveFrom: string;
  currency: string;
  calculateEarnings(input: EarningsInput): EarningsResult;
  calculateStatutoryDeductions(ctx: { grossPay: number; earnings: EarningsResult }): StatutoryDeductionLine[];
  calculateTax(ctx: { grossPay: number; statutoryDeductions: StatutoryDeductionLine[] }): TaxResult;
  validate(input: PayrollCalculationInput): ValidationIssue[];
}
```

3.3 Money & Determinism Helpers

Floating point math on money needs disciplined rounding, and idempotency needs a hash that doesn't depend on object key order:

```
// money.ts
export function round2(value: number): number {
    return Math.round((value + Number.EPSILON) * 100) / 100;
}

export function sum(values: number[]): number {
    return values.reduce((total, value) => total + value, 0);
}

/** Deterministic JSON serialization – sorts object keys recursively so key order never affects the output */
export function stableStringify(value: unknown): string {
    if (value === null || typeof value !== 'object') return JSON.stringify(value);
    if (Array.isArray(value)) return `[$${value.map(stableStringify).join(',')}]`;
    const keys = Object.keys(value as Record<string, unknown>).sort();
    const entries = keys.map(
        (key) => `${JSON.stringify(key)}:${stableStringify((value as any)[key])}`,
    );
    return `{$${entries.join(',')}}`;
}
```

Every intermediate monetary value is rounded to 2 decimal places with `round2` immediately after computing it — never accumulated unrounded and rounded once at the end — so a payslip’s line items always sum exactly to its displayed totals.

3.4 Proration

Mid-period joiners and leavers get their fixed pay elements scaled by the fraction of the period actually worked:

```
// proration.ts
function diffDaysInclusive(start: Date, end: Date): number {
    const msPerDay = 24 * 60 * 60 * 1000;
    return Math.round((end.getTime() - start.getTime()) / msPerDay) + 1;
}

export function calculateProrationFactor(period: EmploymentPeriod): number {
    const periodStart = new Date(period.periodStart);
    const periodEnd = new Date(period.periodEnd);
    const totalDays = diffDaysInclusive(periodStart, periodEnd);
    if (totalDays <= 0) return 0;

    const employmentStart = period.employmentStartDate ? new Date(period.employmentStartDate) : undefined;
    const employmentEnd = period.employmentEndDate ? new Date(period.employmentEndDate) : undefined;

    const effectiveStart = employmentStart && employmentStart > periodStart ? employmentStart : periodStart;
    const effectiveEnd = employmentEnd && employmentEnd < periodEnd ? employmentEnd : periodEnd;
    if (effectiveEnd < effectiveStart) return 0;

    const workedDays = diffDaysInclusive(effectiveStart, effectiveEnd);
    return Math.min(1, Math.max(0, round4(workedDays / totalDays)));
}
```

Overtime, commission, and bonus amounts are deliberately **not** scaled by this factor when applied in `prorateEarnings` — they represent actuals already tied to real days worked or one-off events, not a rate that should be diluted by partial attendance. Only `basicSalary` and `allowances` get scaled.

3.5 The Engine Entrypoint


```
// engine.ts
export function computeInputHash(input: PayrollCalculationInput, ruleVersion: string): string {
  return createHash('sha256').update(stableStringify({ input, ruleVersion })).digest('hex');
}

export function runPayrollCalculation(
  input: PayrollCalculationInput,
  ruleSet: CountryRuleSet,
): PayrollCalculationResult {
  const validation = ruleSet.validate(input);
  const prorationFactor = calculateProrationFactor(input.period);

  const fullEarnings = ruleSet.calculateEarnings(input.earnings);
  const earnings = prorateEarnings(fullEarnings, prorationFactor);

  const statutoryDeductions = ruleSet.calculateStatutoryDeductions({
    grossPay: earnings.grossPay,
    earnings,
  });
  const tax = ruleSet.calculateTax({ grossPay: earnings.grossPay, statutoryDeductions });

  const voluntaryDeductions = input.deductions?.voluntary ?? {};
  const totalVoluntaryDeductions = round2(sum(Object.values(voluntaryDeductions)));
  const totalStatutoryEmployeeAmount = sum(statutoryDeductions.map((d) => d.employeeAmount));
  const totalDeductions = round2(totalStatutoryEmployeeAmount + tax.netTax + totalVoluntaryDeductions);
  const netPay = round2(earnings.grossPay - totalDeductions);

  return {
    employeeId: input.employeeId,
    countryCode: ruleSet.countryCode,
    currency: input.currency,
    ruleVersion: ruleSet.version,
    prorationFactor,
    earnings,
    statutoryDeductions,
    voluntaryDeductions,
    totalVoluntaryDeductions,
    tax,
    totalDeductions,
    grossPay: earnings.grossPay,
    netPay,
    validation,
    inputHash: computeInputHash(input, ruleSet.version),
  };
}
```

Notice the order of operations: **proration happens between earnings calculation and everything downstream** — statutory deductions and tax are always computed on the *prorated* gross pay, so a mid-month joiner's NSSF/PAYE/etc. are correctly based on what they actually earned that period, not their full monthly rate.

3.6 The Registry — Versioned Rules by Country

```
// registry.ts
const REGISTRY: Record<string, CountryRuleSet[]> = {
  KE: kenyaRuleSets,
  NG: nigeriaRuleSets,
  ZA: southAfricaRuleSets,
};

export class UnsupportedCountryError extends Error { /* ... */ }
export class NoApplicableRuleVersionError extends Error { /* ... */ }

/**
 * Resolves the ruleset that applies for a country on a given date, picking
 * the most recent version whose effectiveFrom is on or before that date.
 * A payroll run always records which version it used, and re-running the
 * same period later reproduces it by pinning effectiveDate to the period
 * rather than "now".
 */
export function getCountryRuleSet(countryCode: string, effectiveDate: Date = new Date()): CountryRuleSet {
  const ruleSets = REGISTRY[countryCode.toUpperCase()];
  if (!ruleSets || ruleSets.length === 0) throw new UnsupportedCountryError(countryCode);

  const applicable = ruleSets
    .filter((rs) => new Date(rs.effectiveFrom).getTime() <= effectiveDate.getTime())
    .sort((a, b) => new Date(b.effectiveFrom).getTime() - new Date(a.effectiveFrom).getTime());

  const ruleSet = applicable[0];
  if (!ruleSet) throw new NoApplicableRuleVersionError(countryCode, effectiveDate);
  return ruleSet;
}

export function getRuleSetByVersion(countryCode: string, version: string): CountryRuleSet {
  const ruleSet = REGISTRY[countryCode.toUpperCase()]?.find((rs) => rs.version === version);
  if (!ruleSet) throw new UnsupportedCountryError(`${countryCode}@${version}`);
  return ruleSet;
}
```

Adding a fourth country is exactly: write a new `countries/<name>/` module implementing `CountryRuleSet`, add it to `REGISTRY`. Nothing else in the engine, the API, or the frontend calculator needs to change.

3.7 Kenya (KE) — Two Versions, One Historical Transition

Kenya is the most complex ruleset because it's the one with a real legislative transition baked in: NHIF was replaced by SHIF (Social Health Insurance Fund) on 1 October 2024.

```
// countries/kenya/constants.ts
export const PAYE_BRACKETS = [
  { min: 0, max: 24_000, rate: 0.10 },
  { min: 24_000, max: 32_333, rate: 0.25 },
  { min: 32_333, max: 500_000, rate: 0.30 },
  { min: 500_000, max: 800_000, rate: 0.325 },
  { min: 800_000, max: Infinity, rate: 0.35 },
];
export const PERSONAL_RELIEF = 2_400;

export const NSSF_TIERS = {
  tier1: { lowerLimit: 0, upperLimit: 7_000, rate: 0.06 },
  tier2: { lowerLimit: 7_000, upperLimit: 36_000, rate: 0.06 },
};

export const NHIF_BANDS = [
  { max: 5_999, amount: 150 }, { max: 7_999, amount: 300 }, /* ... */ { max: Infinity, amount: 1_700 },
];

export const HOUSING_LEVY_RATE = 0.015; // 1.5% employee + 1.5% employer, uncapped

export const KENYA_V1_VERSION = 'KE-2024.1';
export const KENYA_V1_EFFECTIVE_FROM = '2024-01-01';
export const KENYA_V2_VERSION = 'KE-2024.2';
export const KENYA_V2_EFFECTIVE_FROM = '2024-10-01'; // SHIF Act 2023 commencement

export const SHIF_RATE = 0.0275;
export const SHIF_MINIMUM_CONTRIBUTION = 300;
```

PAYE is a standard graduated-bracket walk:

```
// countries/kenya/payee.ts
function calculateGrossPay(taxableIncome: number): number {
  const income = Math.max(0, taxableIncome);
  let tax = 0;
  for (const bracket of PAYE_BRACKETS) {
    if (income <= bracket.min) break;
    const upper = Math.min(income, bracket.max);
    tax += (upper - bracket.min) * bracket.rate;
  }
  return tax;
}

export function calculatePaye(taxableIncome: number): TaxResult {
  const grossTax = round2(calculateGrossPay(taxableIncome));
  const netTax = round2(Math.max(0, grossTax - PERSONAL_RELIEF));
  return { code: 'PAYE', taxableIncome: round2(Math.max(0, taxableIncome)), grossTax, relief: PERSONAL_RELIEF, netTax };
}
```

NSSF is a two-tier contribution:

```
// countries/kenya/nssf.ts
export function calculateNssf(grossPay: number): StatutoryDeductionLine {
  const tier1 = Math.min(grossPay, NSSF_TIERS.tier1.upperLimit) * NSSF_TIERS.tier1.rate;
  const tier2Base = Math.max(0, Math.min(grossPay, NSSF_TIERS.tier2.upperLimit) - NSSF_TIERS.tier2.lowerLimit);
  const tier2 = tier2Base * NSSF_TIERS.tier2.rate;
  const employeeAmount = round2(tier1 + tier2);
  return { code: 'NSSF', label: 'NSSF', employeeAmount, employerAmount: employeeAmount };
}
```

SHIF (the current scheme) is a flat percentage with a floor:

```
// countries/kenya/shif.ts
export function calculateShif(grossPay: number): StatutoryDeductionLine {
  const employeeAmount = Math.max(SHIF_MINIMUM_CONTRIBUTION, round2(grossPay * SHIF_RATE));
  return { code: 'SHIF', label: 'Social Health Insurance Fund', employeeAmount, employerAmount: 0 };
}
```

Housing Levy is a flat, uncapped percentage matched by the employer:

```
// countries/kenya/housingLevy.ts
export function calculateHousingLevy(grossPay: number): StatutoryDeductionLine {
  const amount = round2(grossPay * HOUSING_LEVY_RATE);
  return { code: 'HOUSING_LEVY', label: 'Affordable Housing Levy', employeeAmount: amount, employerAmount: amount };
}
```

Both versions of Kenya's `CountryRuleSet` share the same `calculateEarnings` / `validate` and only differ in which health-scheme function they call:

```
// countries/kenya/index.ts
export const kenyaV1: CountryRuleSet = {
  countryCode: 'KE', version: KENYA_V1_VERSION, effectiveFrom: KENYA_V1_EFFECTIVE_FROM, currency: 'KES',
  calculateEarnings,
  calculateStatutoryDeductions: ({ grossPay }) => [
    calculateNssf(grossPay), calculateNhif(grossPay), calculateHousingLevy(grossPay),
  ],
  calculateTax: ({ grossPay, statutoryDeductions }) => {
    // NSSF, NHIF and Housing Levy are all pre-tax deductible (Finance Act 2023)
    const deductible = sum(statutoryDeductions.map((d) => d.employeeAmount));
    return calculatePaye(grossPay - deductible);
  },
  validate,
};

export const kenyaV2: CountryRuleSet = {
  countryCode: 'KE', version: KENYA_V2_VERSION, effectiveFrom: KENYA_V2_EFFECTIVE_FROM, currency: 'KES',
  calculateEarnings,
  calculateStatutoryDeductions: ({ grossPay }) => [
    calculateNssf(grossPay), calculateShif(grossPay), calculateHousingLevy(grossPay),
  ],
  calculateTax: ({ grossPay, statutoryDeductions }) => {
    const deductible = sum(statutoryDeductions.map((d) => d.employeeAmount));
    return calculatePaye(grossPay - deductible);
  },
  validate,
};

export const kenyaRuleSets: CountryRuleSet[] = [kenyaV1, kenyaV2];
```

A payroll period dated before 1 Oct 2024 resolves to `kenyaV1` (NHIF); on or after resolves to `kenyaV2` (SHIF) — automatically, via `getCountryRuleSet('KE', periodDate)`.

3.8 Nigeria (NG)

```
// countries/nigeria/constants.ts
export const ANNUAL_PAYE_BRACKETS = [
  { min: 0, max: 300_000, rate: 0.07 },
  { min: 300_000, max: 600_000, rate: 0.11 },
  { min: 600_000, max: 1_100_000, rate: 0.15 },
  { min: 1_100_000, max: 1_600_000, rate: 0.19 },
  { min: 1_600_000, max: 3_200_000, rate: 0.21 },
  { min: 3_200_000, max: Infinity, rate: 0.24 },
];

export const PENSION_EMPLOYEE_RATE = 0.08; // Pension Reform Act
export const PENSION_EMPLOYER_RATE = 0.10;
export const NHF_EMPLOYEE_RATE = 0.025; // National Housing Fund Act, employee-only
```

```
// countries/nigeria/pension.ts
export function calculatePension(grossPay: number): StatutoryDeductionLine {
  return {
    code: 'PENSION', label: 'Pension (PRA 2014)',
    employeeAmount: round2(grossPay * PENSION_EMPLOYEE_RATE),
    employerAmount: round2(grossPay * PENSION_EMPLOYER_RATE),
  };
}

// countries/nigeria/nhf.ts – National Housing Fund, 2.5% of BASIC (not gross), no employer match
export function calculateNhf(basicSalary: number): StatutoryDeductionLine {
  return {
    code: 'NHF', label: 'National Housing Fund',
    employeeAmount: round2(basicSalary * NHF_EMPLOYEE_RATE), employerAmount: 0,
  };
}
```

```
// countries/nigeria/index.ts
export const nigeriaV1: CountryRuleSet = {
  countryCode: 'NG', version: NIGERIA_V1_VERSION, effectiveFrom: NIGERIA_V1_EFFECTIVE_FROM, currency: 'NGN',
  calculateEarnings,
  calculateStatutoryDeductions: ({ grossPay, earnings }) => [
    calculatePension(grossPay),
    calculateNhf(earnings.basicSalary), // note: basic salary, not gross pay
  ],
  calculateTax: ({ grossPay, statutoryDeductions }) => {
    const deductible = sum(statutoryDeductions.map((d) => d.employeeAmount));
    return calculatePaye(grossPay - deductible);
  },
  validate,
};
export const nigeriaRuleSets: CountryRuleSet[] = [nigeriaV1];
```

3.9 South Africa (ZA)

South Africa’s PAYE is the odd one out: SARS publishes *annual* tax bands with a base-tax-plus-marginal-rate formula, so the monthly taxable base has to be annualized, taxed, then divided back down — rather than walking brackets directly on the monthly figure like Kenya and Nigeria do.

```
// countries/south-africa/constants.ts
export const ANNUAL_PAYE_BRACKETS = [
  { min: 0, max: 237_100, rate: 0.18, base: 0 },
  { min: 237_100, max: 370_500, rate: 0.26, base: 42_678 },
  { min: 370_500, max: 512_800, rate: 0.31, base: 77_362 },
  { min: 512_800, max: 673_000, rate: 0.36, base: 121_475 },
  { min: 673_000, max: 857_900, rate: 0.39, base: 179_147 },
  { min: 857_900, max: 1_817_000, rate: 0.41, base: 251_258 },
  { min: 1_817_000, max: Infinity, rate: 0.45, base: 644_489 },
]; // SARS 2024/25 tax year
export const ANNUAL_PRIMARY_REBATE = 17_235;
export const UIF_RATE = 0.01; // 1% employee + 1% employer
export const UIF_MONTHLY_CEILING = 17_712; // contribution base is capped
export const SDL_RATE = 0.01; // employer-only
```

```
// countries/south-africa/payee.ts
function calculateAnnualGrossTax(annualTaxableIncome: number): number {
  const income = Math.max(0, annualTaxableIncome);
  const bracket = [...ANNUAL_PAYE_BRACKETS].reverse().find((b) => income > b.min) ?? ANNUAL_PAYE_BRACKETS[0]!;
  return bracket.base + (income - bracket.min) * bracket.rate;
}

export function calculatePaye(monthlyTaxableIncome: number): TaxResult {
  const annualTaxable = Math.max(0, monthlyTaxableIncome) * 12;
  const annualGrossTax = calculateAnnualGrossTax(annualTaxable);
  const annualNetTax = Math.max(0, annualGrossTax - ANNUAL_PRIMARY_REBATE);
  const netTax = round2(annualNetTax / 12);
  return {
    code: 'PAYE', taxableIncome: round2(monthlyTaxableIncome),
    grossTax: round2(annualGrossTax / 12), relief: round2(ANNUAL_PRIMARY_REBATE / 12), netTax,
  };
}
```

```
// countries/south-africa/uif.ts – capped at the monthly ceiling before applying the rate
export function calculateUif(grossPay: number): StatutoryDeductionLine {
  const contributionBase = Math.min(grossPay, UIF_MONTHLY_CEILING);
  const amount = round2(contributionBase * UIF_RATE);
  return { code: 'UIF', label: 'UIF', employeeAmount: amount, employerAmount: amount };
}

// countries/south-africa/sdl.ts – employer-only, never touches net pay or taxable income
export function calculateSdl(grossPay: number): StatutoryDeductionLine {
  return { code: 'SDL', label: 'Skills Development Levy', employeeAmount: 0, employerAmount: round2(grossPay * SDL_RATE) };
}
```

```
// countries/south-africa/index.ts
export const southAfricaV1: CountryRuleSet = {
  countryCode: 'ZA', version: SOUTH_AFRICA_V1_VERSION, effectiveFrom: SOUTH_AFRICA_V1_EFFECTIVE_FROM, currency: 'ZAR',
  calculateEarnings,
  calculateStatutoryDeductions: ({ grossPay }) => [calculateUif(grossPay), calculateSdl(grossPay)],
  calculateTax: ({ grossPay }) => {
    // UIF is NOT tax-deductible under SARS rules (unlike retirement contributions), so PAYE runs on gross pay directly
    return calculatePaye(grossPay);
  },
  validate,
};
export const southAfricaRuleSets: CountryRuleSet[] = [southAfricaV1];
```

Note the deliberate asymmetry versus Kenya/Nigeria: South Africa's `calculateTax` ignores `statutoryDeductions` entirely, because UIF genuinely isn't a pre-tax deduction under SARS rules. Getting this wrong (subtracting UIF before computing PAYE) is exactly the kind of bug that produces a *plausible-looking but wrong* number — it was caught and fixed during this project's audit history precisely because the engine's country modules are isolated enough that a bracket/deductibility bug in one country never touches another's tests.

3.10 Testing the Engine

Because every function here is pure, tests are simple input → output assertions with no mocking:

```
// __tests__/kenya.spec.ts
describe('Kenya payroll rules', () => {
  it('computes NHIF-era PAYE, NSSF, NHIF and Housing Levy for a mid-range salary', () => {
    const result = runPayrollCalculation(
      {
        employeeId: 'emp-1', countryCode: 'KE', currency: 'KES',
        earnings: { basicSalary: 50_000 },
        period: { periodStart: '2024-06-01', periodEnd: '2024-06-30' },
      },
      getCountryRuleSet('KE', new Date('2024-06-15')),
    );
    expect(result.ruleVersion).toBe('KE-2024.1'); // resolves to NHIF-era ruleset
    expect(result.statutoryDeductions.find((d) => d.code === 'NSSF')?.employeeAmount).toBe(2_160);
    // ...
  });

  it('resolves to the SHIF-era ruleset on or after 1 Oct 2024', () => {
    const ruleSet = getCountryRuleSet('KE', new Date('2024-10-01'));
    expect(ruleSet.version).toBe('KE-2024.2');
  });
});
```

Registry-level tests assert the version-resolution boundary itself (the day before vs. the day of a transition), and a `computeInputHash` test asserts that key order in the input object never changes the resulting hash — the actual guarantee idempotency depends on.

Part 4 — Backend Foundations

With the pure calculation engine done, the next layer is everything the API needs before a single payroll-domain endpoint can be written safely: configuration, authentication, tenant isolation, and RBAC. Get these four right first — every module built in Parts 5–9 leans on all of them.

4.1 Environment Validation — Fail Fast, Not Silently Wrong

The single worst failure mode for a multi-tenant SaaS handling PII and money is booting successfully with broken configuration — a missing encryption key, or a JWT secret still set to the checked-in dev default. `env.validation.ts` is wired into `ConfigModule.forRoot({ validate })`, so it runs before any module (and its DB/Redis connections) initializes:

```
// config/env.validation.ts
const INSECURE_DEFAULT_SECRETS = new Set([
  'dev-access-secret-change-me', 'dev-refresh-secret-change-me',
]);
const MIN_SECRET_LENGTH = 16;

export function validateEnv(config: Record<string, unknown>): Record<string, unknown> {
  const validated = plainToInstance(EnvironmentVariables, config, { enableImplicitConversion: true });
  const errors = validateSync(validated, { skipMissingProperties: false });
  if (errors.length > 0) {
    throw new Error(`Invalid environment configuration:\n${errors.map(...).join('\n')}`);
  }

  if ((validated.NODE_ENV ?? 'development') === 'production') {
    const problems: string[] = [];
    for (const [key, value] of [
      ['JWT_ACCESS_SECRET', validated.JWT_ACCESS_SECRET],
      ['JWT_REFRESH_SECRET', validated.JWT_REFRESH_SECRET],
    ] as const) {
      if (!value) problems.push(`${key} must be set in production`);
      else if (INSECURE_DEFAULT_SECRETS.has(value)) problems.push(`${key} is still the checked-in dev default`);
      else if (value.length < MIN_SECRET_LENGTH) problems.push(`${key} is too short`);
    }
    if (!validated.ENCRYPTION_KEY) {
      problems.push('ENCRYPTION_KEY must be set in production');
    }
    if (problems.length > 0) {
      throw new Error(`Refusing to start in production with insecure configuration:\n- ${problems.join('\n- ')}`);
    }
  }
  return config;
}
```

The key design choice: **dev-mode defaults are allowed to exist** (so a fresh clone runs with zero setup) **but production checks for exactly those known-insecure values by name** and refuses to boot rather than silently running with a secret anyone who has read the source code can reconstruct.

4.2 Bootstrap — `main.ts`

```

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    bufferLogs: true,
    // Needed to verify the Paystack webhook signature, which is computed
    // over the exact raw request bytes rather than the re-serialized body.
    rawBody: true,
  });
  app.useLogger(app.get(Logger));
  app.use(helmet());

  const configService = app.get(ConfigService<AppConfig, true>);
  const sentryDsn = configService.get('sentryDsn', { infer: true });
  if (sentryDsn) {
    Sentry.init({ dsn: sentryDsn, environment: configService.get('nodeEnv', { infer: true }), tracesSampleRate: 0.1 });
  }

  app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
  app.enableCors({ origin: configService.get('corsOrigin', { infer: true }), credentials: true });

  await app.listen(configService.get('port', { infer: true }));
}

```

`rawBody: true` is not boilerplate — it exists specifically so the Paystack webhook handler (Part 7) can verify a signature computed over the exact bytes Paystack sent, which re-serializing a parsed JSON body would silently break. `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })` means every DTO is the actual security boundary for request bodies: unknown fields are stripped, and if a client tries to smuggle a field the DTO doesn't declare, the pipe rejects the whole request rather than quietly dropping it.

4.3 AppModule — Wiring Order Matters

```

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, load: [configuration], validate: validateEnv }),
    LoggerModule.forRootAsync({ /* pino, redacts Authorization header + password fields */ }),
    ThrottlerModule.forRootAsync({ /* rate limiting */ }),
    CacheModule.registerAsync({ isGlobal: true, /* Redis-backed */ }),
    BullModule.forRootAsync({ /* Redis connection for queues */ }),
    TerminusModule, PrismaModule, AuditModule,
    AuthModule, TenantsModule, EmployeesModule, PayrollModule, PayrollCalculatorModule,
    PayslipsModule, BillingModule, BankExportModule, BrandingModule, HealthModule, CryptoModule,
    ComplianceReportsModule, NotificationsModule, ApiKeysModule, PublicApiModule, WebhooksModule,
    AccountingModule, SalaryComponentsModule, DocumentsModule, LeaveModule, HolidaysModule,
    AttendanceModule, AnalyticsModule, LoansModule,
  ],
  controllers: [AppController],
  providers: [
    { provide: APP_GUARD, useClass: JwtAuthGuard },
    { provide: APP_GUARD, useClass: RolesGuard },
    { provide: APP_GUARD, useClass: PermissionsGuard },
    { provide: APP_GUARD, useClass: ThrottlerGuard },
    { provide: APP_INTERCEPTOR, useClass: TenantContextInterceptor },
    { provide: APP_INTERCEPTOR, useClass: AuditInterceptor },
    { provide: APP_FILTER, useClass: SentryExceptionFilter },
  ],
})
export class AppModule {}

```

Both guard and interceptor **order is significant**. `APP_GUARD` providers run in registration order: `JwtAuthGuard` first (populates `request.user`), then `RolesGuard` and `PermissionsGuard` (both read `request.user`), then `ThrottlerGuard`. `APP_INTERCEPTOR`s run in order too: `TenantContextInterceptor` must run before `AuditInterceptor`, because the audit interceptor's writes need the tenant context already established to be scoped correctly by the Prisma extension (§4.6).

Pino redacts `Authorization`, `password`, `adminPassword`, and the two JWT response fields from every log line — a structured logger is only safe by default if you actively tell it what never to print.

4.4 Authentication — JWT Access + Refresh

Passwords are hashed with bcrypt at 12 salt rounds. Both tokens are signed with separate secrets, and the refresh token itself is never stored in plaintext — only its bcrypt hash, on the `User` row:

```
// auth/auth.service.ts
const SALT_ROUNDS = 12;

@Injectable()
export class AuthService {
  async signup(dto: SignupDto) {
    const existing = await this.prisma.user.findUnique({ where: { email: dto.adminEmail } });
    if (existing) throw new ConflictException('An account with this email already exists');

    const passwordHash = await bcrypt.hash(dto.adminPassword, SALT_ROUNDS);
    const { tenant, user } = await this.prisma.$transaction(async (tx) => {
      const tenant = await tx.tenant.create({
        data: {
          name: dto.tenantName,
          countryCode: dto.countryCode,
          // Derives currency from the country's own pricing catalog rather
          // than defaulting every tenant to KES regardless of country.
          defaultCurrency: getPricingForCountry(dto.countryCode).currency,
        },
      });
      const user = await tx.user.create({
        data: { tenantId: tenant.id, email: dto.adminEmail, passwordHash, role: Role.ADMIN },
      });
      return { tenant, user };
    });

    const tokens = await this.issueTokens(user);
    return { tenant, user: this.toAuthenticatedUser(user), ...tokens };
  }

  async refresh(userId: string, refreshToken: string) {
    const user = await this.prisma.user.findUnique({ where: { id: userId } });
    if (!user?.refreshTokenHash) throw new UnauthorizedException('Session expired, please log in again');

    const matches = await bcrypt.compare(refreshToken, user.refreshTokenHash);
    if (!matches) {
      // Reuse of a rotated-out refresh token – revoke the session outright.
      await this.prisma.user.update({ where: { id: user.id }, data: { refreshTokenHash: null } });
      throw new UnauthorizedException('Invalid refresh token');
    }
    return { user: this.toAuthenticatedUser(user), ...(await this.issueTokens(user)) };
  }

  private async issueTokens(user: User) {
    const jwtConfig = this.configService.get('jwt', { infer: true });
    const accessPayload: JwtAccessPayload = {
      sub: user.id, email: user.email, role: user.role, tenantId: user.tenantId, employeeId: user.employeeId,
    };
    const accessToken = await this.jwtService.signAsync(accessPayload, {
      secret: jwtConfig.accessSecret, expiresIn: jwtConfig.accessExpiresIn,
    });
    const refreshToken = await this.jwtService.signAsync({ sub: user.id }, {
      secret: jwtConfig.refreshSecret, expiresIn: jwtConfig.refreshExpiresIn,
    });
    // Only the HASH of the refresh token is persisted – never the raw token.
    await this.prisma.user.update({
      where: { id: user.id }, data: { refreshTokenHash: await bcrypt.hash(refreshToken, SALT_ROUNDS) },
    });
    return { accessToken, refreshToken };
  }
}
```

Every field the rest of the system needs to authorize a request — `role`, `tenantId`, `employeeId` — is embedded directly in the access token payload, so authorization never requires a database round-trip on every request. The tradeoff is that a role change doesn't take effect until the user's token expires or they log in again; that's an accepted tradeoff for a 15-minute access token lifetime.

Reuse detection: if `refresh()` is called with a refresh token that doesn't match the currently-stored hash — meaning an older, already-rotated-out token is being replayed — the session is revoked outright (`refreshTokenHash: null`) rather than just rejected, on the theory that a mismatched refresh token is a signal the token may have leaked.

`JwtAccessStrategy` (Passport) extracts the bearer token, verifies it against the access secret, and maps the payload onto

`AuthenticatedRequestUser` :

```
// auth/strategies/jwt-access.strategy.ts
@Injectable()
export class JwtAccessStrategy extends PassportStrategy(Strategy, 'jwt-access') {
  constructor(configService: ConfigService<AppConfig, true>) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: configService.get('jwt', { infer: true }).accessSecret,
    });
  }
  validate(payload: JwtAccessPayload): AuthenticatedRequestUser {
    return { id: payload.sub, email: payload.email, role: payload.role, tenantId: payload.tenantId, employeeId:
      payload.employeeId };
  }
}
```

4.5 Dual-Guard RBAC — Roles + Fine-Grained Permissions

Authorization runs as two independent global guards that both must pass — they are not alternatives to each other.

`RolesGuard` checks the coarse `@Roles(Role.ADMIN, Role.HR)` decorator against `request.user.role` :

```
@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [context.getHandler(),
      context.getClass()]);
    if (!requiredRoles || requiredRoles.length === 0) return true;

    const request = context.switchToHttp().getRequest();
    // API-key-authenticated requests carry a placeholder role that must
    // never be allowed to satisfy a real role check – fail closed.
    if (request.isApiKeyAuth) {
      throw new ForbiddenException('API-key authentication is not permitted on role-restricted routes');
    }
    const user: AuthenticatedRequestUser | undefined = request.user;
    if (!user || !requiredRoles.includes(user.role)) {
      throw new ForbiddenException(`Requires one of roles: ${requiredRoles.join(', ')}`);
    }
    return true;
  }
}
```

`PermissionsGuard` checks the fine-grained `@RequirePermission(Permission.PAYROLL_RUN)` decorator against a role → permission map:

```
// common/permissions/permission.enum.ts
export enum Permission {
  EMPLOYEE_WRITE = 'employee:write', EMPLOYEE_TERMINATE = 'employee:terminate',
  PAYROLL_RUN = 'payroll:run', PAYROLL_CORRECT = 'payroll:correct', PAYROLL_READ = 'payroll:read',
  LEAVE_APPROVE = 'leave:approve', LEAVE_TYPE_MANAGE = 'leave-type:manage',
  DOCUMENT_DELETE = 'document:delete', BILLING_MANAGE = 'billing:manage', REPORTS_READ = 'reports:read',
  API_KEY_MANAGE = 'apikey:manage', WEBHOOK_MANAGE = 'webhook:manage', TENANT_MANAGE = 'tenant:manage',
  BRANDING_MANAGE = 'branding:manage', ATTENDANCE_MANAGE = 'attendance:manage',
  LOAN_MANAGE = 'loan:manage', SALARY_COMPONENT_MANAGE = 'salary-component:manage',
}

// common/permissions/role-permissions.map.ts
const ADMIN_ONLY_PERMISSIONS: Permission[] = [
  Permission.BILLING_MANAGE, Permission.API_KEY_MANAGE, Permission.WEBHOOK_MANAGE,
  Permission.TENANT_MANAGE, Permission.EMPLOYEE_TERMINATE, Permission.BRANDING_MANAGE,
];

export const ROLE_PERMISSIONS: Record<Role, Permission[]> = {
  [Role.ADMIN]: Object.values(Permission),
  [Role.HR]: Object.values(Permission).filter((p) => !ADMIN_ONLY_PERMISSIONS.includes(p)),
  [Role.EMPLOYEE]: [],
};
```

Both guards are applied together on every write-sensitive controller (e.g. `@Roles(Role.ADMIN, Role.HR)` + `@RequirePermission(Permission.PAYROLL_RUN)` on the run-payroll endpoint) — currently the permission map is entirely role-derived, so the two checks are redundant in practice today. That redundancy is deliberate: consolidating onto one declared-permission model per endpoint is what lets access control evolve later (e.g. a custom per-tenant role) without having to touch every controller’s role list.

Both guards share a critical fail-closed check: an API-key-authenticated request (Part 9’s public read-only API) carries a placeholder role that must never satisfy a role or permission check meant for real human users — so both guards explicitly reject `request.isApiKeyAuth` rather than letting the placeholder accidentally pass.

4.6 Structural Multi-Tenancy

This is the piece that makes tenant isolation a property of the system rather than a discipline every developer has to remember. It has two halves: request-scoped context, and a Prisma extension that reads it.

Half 1 — `AsyncLocalStorage`-backed context, set once per request and readable anywhere in that request’s async call chain without threading `tenantId` through every function signature:

```
// common/tenant/tenant-context.ts
const storage = new AsyncLocalStorage<TenantContextStore>();

export const TenantContext = {
  run<T>(store: TenantContextStore, fn: () => T): T {
    return storage.run(store, fn);
  },
  getTenantId(): string | undefined {
    return storage.getStore()?.tenantId;
  },
};
```

`TenantContextInterceptor` establishes it, with one subtlety worth calling out explicitly because it’s easy to get wrong: `next.handle()` returns a *cold* Observable — nothing runs until something subscribes — and that subscription normally happens inside Nest’s own pipeline, by which point a naive synchronous `TenantContext.run(...)` would already have returned and torn the context down before the controller even executes. The fix is to subscribe to `next.handle()` yourself, synchronously, *inside* the `run` callback:

```

@Injectables()
export class TenantContextInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    const request = context.switchToHttp().getRequest();
    const tenantId: string | undefined = request.user?.tenantId;
    if (!tenantId) return next.handle();

    return new Observable((subscriber) => {
      TenantContext.run({ tenantId }, () => {
        next.handle().subscribe({
          next: (value) => subscriber.next(value),
          error: (error) => subscriber.error(error),
          complete: () => subscriber.complete(),
        });
      });
    });
  }
}

```

Half 2 — a Prisma client extension that intercepts every query against a defined set of tenant-scoped models and rewrites its `where` / `data` to enforce the tenant boundary, transparently to the calling service code:

```

// prisma/tenant-scoping.extension.ts
// Models with a direct tenantId column. Models reached only through a
// relation (Employee -> Company -> Tenant) aren't listed - Prisma can't
// generically inject a nested join filter for those; those rely on explicit
// company.tenantId ownership checks in each service instead.
export const TENANT_SCOPED_MODELS = new Set<Prisma.ModelName>([
  'BrandingConfig', 'Company', 'User', 'SalaryComponent', 'LeaveType',
  'AuditLog', 'Subscription', 'Invoice', 'UsageRecord', 'Notification',
  'ApiKey', 'WebhookEndpoint', 'Loan',
]);

export function scopeQueryArgs(operation: string, args: Record<string, unknown>, tenantId: string) {
  const scoped = { ...args } as { where?: Record<string, unknown>; data?: unknown };

  if (READ_OPS.has(operation) || WHERE_SCOPED_WRITE_OPS.has(operation)) {
    scoped.where = { ...(scoped.where ?? {}), tenantId };
  }
  if (operation === 'create') {
    assertNotCrossTenant(scoped.data, tenantId); // reject a spoofed tenantId in the payload outright
    scoped.data = { ...(scoped.data as object), tenantId };
  }
  // ...createMany, update, upsert handled the same way
  return scoped;
}

export const tenantScopingExtension = Prisma.defineExtension({
  name: 'tenant-scoping',
  query: {
    $allModels: {
      async $allOperations({ model, operation, args, query }) {
        if (!model || !TENANT_SCOPED_MODELS.has(model)) return query(args);
        const tenantId = TenantContext.getTenantId();
        if (!tenantId) return query(args); // no context = cron/seed/signup path, deliberately unenforced
        return query(scopeQueryArgs(operation, args as Record<string, unknown>, tenantId) as typeof args);
      },
    },
  },
});

```

`scopeQueryArgs` is deliberately a **pure function** separate from the `$extends` wiring, for the same reason the payroll engine is pure: it can be unit-tested exhaustively (every operation type, every cross-tenant-spoofing attempt) with zero database.

The result: a developer writing `this.prisma.company.findMany({ where: { name: 'Acme' } })` inside an authenticated request gets `tenantId` injected automatically — even if they forgot to add it themselves. Attempting `this.prisma.company.create({ data: { tenantId: 'someone-elses-tenant', ... } })` throws `ForbiddenException` rather than silently overwriting it. Outside a request

(cron jobs, queue processors, the signup flow creating the very first `Tenant / Company / User`), `TenantContext` is unset and the extension is a no-op — those paths legitimately need to operate before or without a tenant context.

Models reached only through a relation (`Employee → Company → Tenant`) aren't in `TENANT_SCOPED_MODELS` because Prisma can't generically inject a nested-join filter for an arbitrary relation depth — those services instead do an explicit ownership check, e.g. `assertCompanyBelongsToTenant(companyId, tenantId)`, before touching the child rows.

4.7 Field-Level Encryption

Sensitive PII — KRA PIN, NSSF/NHIF numbers, bank account numbers — is encrypted at rest with AES-256-GCM, via explicit `encrypt()` / `decrypt()` calls in the owning service (not a transparent Prisma middleware, matching the fact this codebase has no other implicit `$use` hook):

```
// common/crypto/encryption.service.ts
const ALGORITHM = 'aes-256-gcm';
const DEV_FALLBACK_KEY = '0'.repeat(64); // 32 zero bytes, hex – dev only

@Injectable()
export class EncryptionService {
  constructor(private readonly configService: ConfigService<AppConfig, true>) {
    const configuredKey = this.configService.get('encryptionKey', { infer: true });
    const nodeEnv = this.configService.get('nodeEnv', { infer: true });
    // Fail closed: never silently encrypt production PII with a well-known key.
    if (!configuredKey && nodeEnv === 'production') {
      throw new Error('ENCRYPTION_KEY is not set. Refusing to start in production without a real encryption key.');
```

Ciphertext is stored as `iv:authTag:ciphertext`, all hex-encoded, in a single string column — GCM's auth tag means a tampered ciphertext fails to decrypt rather than silently returning corrupted plaintext.

4.8 Audit Logging

`AuditInterceptor` (registered globally, running *after* `TenantContextInterceptor` — see §4.3) writes an `AuditLog` row for mutating requests, capturing `actorId`, `action`, `entityType` / `entityId`, and `before` / `after` snapshots where applicable. Because it runs inside the same tenant context established by the interceptor ahead of it, its own writes go through the same `tenantScopingExtension` as everything else — an audit log entry is automatically stamped with the correct `tenantId` with no special-casing.

With authentication, tenant isolation, RBAC, encryption, and auditing all in place, Part 5 builds the actual payroll domain API on top of this foundation.

Part 5 — Core Payroll API

With the foundation from Part 4 in place, this part builds the domain modules that actually move data: employees, payroll execution, and payslip generation. Every module below follows the same skeleton — **Controller** handles HTTP + decorators, **Service** handles business logic + Prisma — so once you’ve built Employees, the rest are variations on the same shape.

5.1 Employees Module — The CRUD Template

Every write endpoint declares both `@Roles(...)` and `@RequirePermission(...)` (Part 4 §4.5); every tenant-scoped lookup goes through `CurrentTenant()` and an ownership check before touching data.

```
// employees/employees.controller.ts
@Controller('employees')
export class EmployeesController {
  constructor(private readonly employeesService: EmployeesService) {}

  @Roles(Role.ADMIN, Role.HR)
  @RequirePermission(Permission.EMPLOYEE_WRITE)
  @Post()
  create(@CurrentTenant() tenantId: string, @Body() dto: CreateEmployeeDto) {
    return this.employeesService.create(tenantId, dto);
  }

  @Get('/:id') // no @Roles — an EMPLOYEE may read (their own record is enforced deeper, e.g. in findMine-style endpoints elsewhere)
  findOne(@CurrentTenant() tenantId: string, @Param('id') id: string) {
    return this.employeesService.findOne(tenantId, id);
  }

  @Roles(Role.ADMIN)
  @RequirePermission(Permission.EMPLOYEE_TERMINATE)
  @Post('/:id/terminate')
  terminate(
    @CurrentTenant() tenantId: string,
    @CurrentUser() user: AuthenticatedRequestUser,
    @Param('id') id: string,
    @Body() dto: TerminateEmployeeDto,
  ) {
    return this.employeesService.terminate(tenantId, user.id, id, dto);
  }

  // ...contracts, salary-structures, onboarding-tasks follow the same shape
}
```

PII is encrypted on the way in, decrypted on the way out — explicitly, every time, not through a Prisma middleware:

```

// employees/employees.service.ts
@Injectable()
export class EmployeesService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly tenantsService: TenantsService,
    private readonly encryptionService: EncryptionService,
    private readonly auditService: AuditService,
  ) {}

  private decryptEmployee<T extends Employee>(employee: T): T {
    return {
      ...employee,
      kraPin: this.encryptionService.decrypt(employee.kraPin),
      nssfNumber: this.encryptionService.decrypt(employee.nssfNumber),
      nhifNumber: this.encryptionService.decrypt(employee.nhifNumber),
      taxIdNumber: this.encryptionService.decrypt(employee.taxIdNumber),
      pensionNumber: this.encryptionService.decrypt(employee.pensionNumber),
      bankAccountNumber: this.encryptionService.decrypt(employee.bankAccountNumber),
    };
  }

  /**
   * New employees start in ONBOARDING, not ACTIVE – excluded from payroll
   * runs and active-employee billing counts until their checklist completes.
   */
  async create(tenantId: string, dto: CreateEmployeeDto) {
    await this.tenantsService.assertCompanyBelongsToTenant(dto.companyId, tenantId);
    const tenant = await this.prisma.tenant.findUniqueOrThrow({ where: { id: tenantId }, select: { countryCode: true } });

    const created = await this.prisma.$transaction(async (tx) => {
      const employee = await tx.employee.create({
        data: {
          companyId: dto.companyId,
          firstName: dto.firstName, lastName: dto.lastName, email: dto.email,
          status: 'ONBOARDING',
          kraPin: this.encryptionService.encrypt(dto.kraPin),
          nssfNumber: this.encryptionService.encrypt(dto.nssfNumber),
          nhifNumber: this.encryptionService.encrypt(dto.nhifNumber),
          taxIdNumber: this.encryptionService.encrypt(dto.taxIdNumber),
          pensionNumber: this.encryptionService.encrypt(dto.pensionNumber),
          bankAccountNumber: this.encryptionService.encrypt(dto.bankAccountNumber),
          currency: dto.currency ?? getPricingForCountry(tenant.countryCode).currency,
          department: dto.department, jobRole: dto.jobRole, employmentType: dto.employmentType,
        },
      });

      // Default onboarding checklist, seeded in the same transaction
      const tasks = getDefaultOnboardingTasks(tenant.countryCode);
      await tx.onboardingTask.createMany({
        data: tasks.map((task, index) => ({ employeeId: employee.id, title: task.title, isRequired: task.isRequired,
          order: index })),
      });

      return employee;
    });

    return this.decryptEmployee(created);
  }

  async findOne(tenantId: string, employeeId: string) {
    const employee = await this.prisma.employee.findUnique({
      where: { id: employeeId },
      include: { contracts: true, salaryStructures: true, company: true },
    });

    // Employee isn't in TENANT_SCOPED_MODELS (it's reached via Company, not
    // a direct tenantId column) – so ownership is checked explicitly here.
    if (!employee || employee.company.tenantId !== tenantId) {
      throw new NotFoundException('Employee not found');
    }

    return this.decryptEmployee(employee);
  }

  /** Salary structure in effect for a given date – used by payroll runs */
  async getActiveSalaryStructure(employeeId: string, asOf: Date) {

```



```

    return this.prisma.salaryStructure.findFirst({
      where: { employeeId, effectiveFrom: { lte: asOf }, OR: [{ effectiveTo: null }, { effectiveTo: { gte: asOf } } ] },
      orderBy: { effectiveFrom: 'desc' },
    });
  }

  /**
   * Full offboarding: TERMINATED status, closes any open contract, and
   * revokes portal access (deactivates the linked User, clears its refresh
   * token so an existing session can't be used past this point). Does NOT
   * attempt to prorate a final paycheck — that's a separate feature; the
   * calculation engine supports proration (Part 3 §3.4) but it isn't wired
   * to a termination date as an input source yet.
   */
  async terminate(tenantId: string, actorId: string, employeeId: string, dto: TerminateEmployeeDto) {
    const employee = await this.findOne(tenantId, employeeId);
    if (employee.status === 'TERMINATED') throw new BadRequestException('Employee is already terminated');

    const terminationDate = dto.terminationDate ? new Date(dto.terminationDate) : new Date();
    const updated = await this.prisma.$transaction(async (tx) => {
      const result = await tx.employee.update({
        where: { id: employeeId },
        data: { status: 'TERMINATED', terminatedAt: terminationDate, terminationReason: dto.reason },
      });
      await tx.contract.updateMany({ where: { employeeId, endDate: null }, data: { endDate: terminationDate } });
      await tx.user.updateMany({ where: { employeeId }, data: { isActive: false, refreshTokenHash: null } });
      return result;
    });

    await this.auditService.record({
      tenantId, actorId, action: 'employee.terminate', entityType: 'Employee', entityId: employeeId,
      before: { status: employee.status }, after: { status: 'TERMINATED', terminatedAt: terminationDate.toISOString(),
      reason: dto.reason ?? null },
    });
    return this.decryptEmployee(updated);
  }
}

```

Onboarding gate: `completeOnboarding()` refuses to flip an employee from `ONBOARDING` to `ACTIVE` while any `isRequired` task is still incomplete — this is the actual enforcement point that keeps an incompletely-onboarded employee out of payroll runs (which filter `status: 'ACTIVE'`) and out of billing's active-employee count.

5.2 Payroll Module — Execution, Idempotency, Concurrency

This is the module where every earlier architectural decision pays off at once: the pure engine (Part 3), tenant scoping (Part 4), and the `PayrollEntry` schema (Part 2) all come together in `executeRun`.

Bounded-concurrency per-employee computation. Each employee's calculation (fetch their active salary structure, resolve salary components, resolve loan deductions, run the pure engine) is independent of every other employee's — but a plain `Promise.all` over hundreds or thousands of employees would fire that many concurrent DB queries at once and exhaust the connection pool. A small worker-pool helper bounds it:

```

const RUN_COMPUTATION_CONCURRENCY = 10;

async function mapWithConcurrency<T, R>(items: T[], concurrency: number, fn: (item: T) => Promise<R>): Promise<R[]> {
  const results: R[] = new Array(items.length);
  let nextIndex = 0;
  async function worker() {
    while (nextIndex < items.length) {
      const current = nextIndex++;
      results[current] = await fn(items[current]!); // index-based assignment preserves order
    }
  }
  await Promise.all(Array.from({ length: Math.min(concurrency, items.length) }, worker));
  return results;
}

```

Order-preservation matters here only insofar as correctness downstream doesn't actually depend on it — `computeRunIdempotencyKey` sorts entry hashes before hashing, and `aggregateTotals` is a commutative sum — but assigning into `results[current]` by index rather than pushing keeps the result array's order predictable regardless.

The run itself, trimmed to the essential shape:

```

private async executeRun(params: ExecuteRunParams) {
  const { tenantId, actorId, companyId, period, periodStart: periodStartInput, periodEnd: periodEndInput, employeeIds,
    isOffCycle, reason, force } = params;

  const company = await this.tenantsService.assertCompanyBelongsToTenant(companyId, tenantId);
  const tenant = await this.prisma.tenant.findUniqueOrThrow({ where: { id: tenantId } });
  const periodStart = new Date(periodStartInput);
  const ruleSet = await this.rulesCache.resolve(tenant.countryCode, periodStart); // Part 3 §3.6 registry, cached

  const employees = await this.prisma.employee.findMany({
    where: { companyId: company.id, status: 'ACTIVE', ...(employeeIds ? { id: { in: employeeIds } } : {}) },
  });

  const perEmployeeResults = await mapWithConcurrency(employees, RUN_COMPUTATION_CONCURRENCY, async (employee) => {
    const salaryStructure = await this.employeesService.getActiveSalaryStructure(employee.id, periodStart);
    if (!salaryStructure) return null; // no active structure = skipped, not an error

    const { allowances, voluntaryDeductions: componentDeductions } =
      await this.salaryComponentsService.resolveStructureEarnings(salaryStructure.id, salaryStructure.basicSalary,
        salaryStructure.allowances);
    const { voluntaryDeductions: loanDeductions, repayments: loanRepayments } =
      await this.loansService.resolvePayrollDeductions(tenantId, employee.id, period);

    const input: PayrollCalculationInput = {
      employeeId: employee.id, countryCode: ruleSet.countryCode, currency: salaryStructure.currency,
      earnings: { basicSalary: salaryStructure.basicSalary, allowances },
      ...(Object.keys({ ...componentDeductions, ...loanDeductions }).length > 0
        ? { deductions: { voluntary: { ...componentDeductions, ...loanDeductions } } : {}},
        period: { periodStart: periodStartInput, periodEnd: periodEndInput },
    );
    return { employee, result: runPayrollCalculation(input, ruleSet), loanRepayments };
  });

  const computations = perEmployeeResults.filter((c) => c !== null);

  // Single-currency-per-tenant enforcement: fail the WHOLE run rather than
  // silently excluding a mismatched employee – a wrong currency on a salary
  // structure is a data error, not routine payroll behavior.
  const currencyErrors = computations.flatMap(({ employee, result }) =>
    result.validation.filter((i) => i.severity === 'error' && i.field === 'currency').map((issue) => ({ employee, issue })));
  if (currencyErrors.length > 0) {
    throw new BadRequestException(`Cannot run payroll – currency mismatch for ${currencyErrors.length} employee(s):
    ...`);
  }

  const idempotencyKey = this.computeRunIdempotencyKey(companyId, period, ruleSet.version, computations, isOffCycle,
    reason);
  const existingRun = await this.prisma.payrollRun.findUnique({ where: { idempotencyKey }, include: { entries: true } });
  if (existingRun && !force) return existingRun; // re-submitting the same request is a no-op, not a duplicate run

  const totals = this.aggregateTotals(computations);
  const run = await this.prisma.$transaction(async (tx) => {
    const payrollRun = await tx.payrollRun.create({
      data: { companyId: companyId, period, periodStart, periodEnd: new Date(periodEndInput),
        countryCode: ruleSet.countryCode, currency: ruleSet.currency, ruleVersion: ruleSet.version,
        status: 'COMPLETED', idempotencyKey, totals, initiatedById: actorId, isOffCycle, reason },
    });
    for (const { employee, result } of computations) {
      await tx.payrollEntry.create({
        data: { payrollRunId: payrollRun.id, employeeId: employee.id, currency: result.currency,
          prorationFactor: result.prorationFactor, grossPay: result.grossPay, totalTax: result.tax.netTax,
          totalStatutoryDeductions: sum(result.statutoryDeductions.map((d) => d.employeeAmount)),
          totalVoluntaryDeductions: result.totalVoluntaryDeductions, totalDeductions: result.totalDeductions,
          netPay: result.netPay, earningsBreakdown: result.earnings, statutoryDeductions: result.statutoryDeductions,
          taxBreakdown: result.tax, inputHash: result.inputHash },
      });
    }
    return tx.payrollRun.findUniqueOrThrow({ where: { id: payrollRun.id }, include: { entries: true } });
  });

  await this.auditService.record({ tenantId, actorId, action: isOffCycle ? 'payroll.run.off-cycle' : 'payroll.run',
    entityType: 'PayrollRun', entityId: run.id, after: totals });
  await this.payslipEmailService.enqueueForRun(tenantId, run.id); // async, off the request thread (Part 8)
  for (const entry of run.entries) {

```

```

    const computation = computations.find((c) => c.employee.id === entry.employeeId);
    if (computation?.loanRepayments.length) await this.loansService.markRepaymentsPaid(computation.loanRepayments,
        entry.id);
  }
  await this.notificationsService.dispatchForRoles(tenantId, [Role.ADMIN, Role.HR], 'PAYROLL_RUN_COMPLETED', `Payroll
    run for ${run.period} has completed...`, { metadata: { runId: run.id } });
  void this.webhooksService.dispatch(tenantId, 'payroll.run.completed', { runId: run.id, totals: run.totals }).catch(()
    => {});

  return run;
}

```

Idempotency key derivation — the single most important correctness invariant in this module:

```

private computeRunIdempotencyKey(companyId: string, period: string, ruleVersion: string, computations: Computation[],
  isOffCycle: boolean, reason?: string): string {
  const entryHashes = computations.map((c) => `${c.employee.id}:${c.result.inputHash}`).sort(); // order-independent
  const payload: Record<string, unknown> = { companyId, period, ruleVersion, entryHashes };
  if (isOffCycle) { payload.isOffCycle = true; payload.reason = reason; }
  return createHash('sha256').update(stableStringify(payload)).digest('hex');
}

```

`entryHashes` is sorted before hashing specifically so `computations`’ array order — which now depends on `mapWithConcurrency`’s worker-pool scheduling and is therefore non-deterministic across runs — never affects the resulting key. Two submissions of “run June payroll for these employees” always resolve to the same `idempotencyKey` regardless of goroutine-style scheduling noise, so the `existingRun` check above is a reliable duplicate-submission guard.

Corrections never mutate history. `createCorrection` re-resolves the *exact original ruleset* via `getRuleSetByVersion` (not “whatever’s current” via the registry’s date-based resolution) — so a correction to a run from months ago is computed with bit-identical rules to the original, then persisted as a brand-new off-cycle `PayrollEntry` linked back via `PayrollCorrection`. The original entry row is never touched.

5.3 Payslips — Branded PDF Generation

Payslip PDFs are React components rendered server-side with `@react-pdf/renderer`, which is what makes tenant branding a matter of passing props rather than templating strings:

```
// payslips/payslips.service.ts
async generate(tenantId: string, payrollEntryId: string, actor?: AuthenticatedRequestUser) {
  const entry = await this.prisma.payrollEntry.findUnique({
    where: { id: payrollEntryId },
    include: { employee: { include: { company: true } }, payrollRun: true },
  });
  if (!entry || entry.employee.company.tenantId !== tenantId) throw new NotFoundException('Payroll entry not found');
  // actor is omitted for system-initiated generation (the post-run bulk
  // email job). When present and EMPLOYEE, they may only fetch their own.
  if (actor?.role === Role.EMPLOYEE && actor.employeeId !== entry.employeeId) {
    throw new ForbiddenException('You may only access your own payslip');
  }

  const branding = await this.brandingService.getBranding(tenantId); // logo URL, primary/secondary color, falls back to
  APP_NAME

  const earnings = entry.earningsBreakdown as EarningsBreakdown;
  const earningLines: PayslipLineItem[] = [
    { label: 'Basic Salary', amount: earnings.basicSalary },
    ...Object.entries(earnings.allowanceBreakdown ?? {}).map(([label, amount]) => ({ label, amount })),
  ];
  if (earnings.overtimeAmount) earningLines.push({ label: 'Overtime', amount: earnings.overtimeAmount });

  const pdfBuffer = await renderToBuffer(
    PayslipDocument({
      branding, companyName: entry.employee.company.name,
      employeeName: `${entry.employee.firstName} ${entry.employee.lastName}`,
      period: entry.payrollRun.period, currency: entry.currency,
      earnings: earningLines,
      statutoryDeductions: (entry.statutoryDeductions as StatutoryLine[]).map((d) => ({ label: d.label, amount:
        d.employeeAmount })),
      tax: { label: (entry.taxBreakdown as TaxResult).code, amount: (entry.taxBreakdown as TaxResult).netTax },
      grossPay: entry.grossPay, totalDeductions: entry.totalDeductions, netPay: entry.netPay,
    }),
  );

  const pdfPath = join(STORAGE_DIR, `${payrollEntryId}.pdf`);
  await mkdir(STORAGE_DIR, { recursive: true });
  await writeFile(pdfPath, pdfBuffer);
  await this.prisma.payslip.upsert({ where: { payrollEntryId }, create: { payrollEntryId, pdfPath }, update: { pdfPath,
    generatedAt: new Date() } });

  return pdfBuffer;
}
```

`PayslipDocument` is a `@react-pdf/renderer` component tree (`<Document>` → `<Page>` → `<View>` / `<Text>`) that reads `branding.logoUrl` / `branding.primaryColor` and renders them into the header — the same pattern used for compliance report PDFs in Part 6.

5.4 Payroll Calculator & Bank Export

Two smaller modules round out this part:

- **payroll-calculator** module exposes a single unauthenticated `POST /payroll-calculator/preview` endpoint that calls `runPayrollCalculation` directly with a client-supplied `countryCode` / `basicSalary` / `allowances`, with no `Employee` or `PayrollRun` persistence at all. This is what powers the public marketing-site “estimate your payroll” calculator (Part 10) — it reuses the exact same engine and rules as real payroll runs, so the public estimate and the real payslip can never drift apart.
- **bank-export** module takes a completed `PayrollRun` and produces a bank-specific CSV/text format (account number, amount, reference) employers upload to their bank’s bulk-payment portal — one formatter per supported bank format, sharing a common `BankExportRow` interface so adding a new bank format doesn’t touch the run-fetching logic.

Part 6 builds the compliance reporting layer on top of the same `PayrollRun` / `PayrollEntry` data these modules already produce.

Part 6 — Compliance Reports

Every report in this module reads already-persisted `PayrollEntry.statutoryDeductions` and `PayrollEntry.taxBreakdown` JSON columns — **it never recalculates**. This matters: a compliance report always reflects exactly what a payslip said at the time, even if the underlying rules have since changed, because it’s reading history rather than re-deriving it.

6.1 Design: Country-Gated, JSON-Driven

Because `PayrollEntry` stores statutory deduction lines as a JSON array of `{ code, label, employeeAmount, employerAmount }` (Part 3 §3.2) rather than fixed columns, every report is really just “find the line with this `code`, sum it across employees or periods.” A tiny helper does the lookup:

```
function findLine(lines: StatutoryDeductionLine[], code: string): StatutoryDeductionLine | undefined {
  return lines.find((line) => line.code === code);
}
```

Kenya’s health-insurance line needs a small twist: NHIF was replaced by SHIF on 1 Oct 2024 (Part 3 §3.7), so a report spanning both eras — a full-year P9, for instance — must resolve whichever code is actually present on each individual entry:

```
function findHealthInsuranceLine(lines: StatutoryDeductionLine[]): StatutoryDeductionLine | undefined {
  return findLine(lines, 'SHIF') ?? findLine(lines, 'NHIF');
}
```

Every report method asserts the tenant’s actual country before doing anything else — so a KRA P9 can never accidentally be generated for a Nigerian tenant just because a caller passed the wrong `companyId`:

```
private async assertCompanyCountry(tenantId: string, companyId: string, expectedCountryCode: 'KE' | 'NG' | 'ZA') {
  const company = await this.prisma.company.findFirst({ where: { id: companyId, tenantId }, include: { tenant: true } });
  if (!company) throw new NotFoundException('Company not found for this tenant');
  if (company.tenant.countryCode !== expectedCountryCode) {
    throw new BadRequestException(`This report is only available for ${expectedCountryCode} tenants, but this tenant's country is ${company.tenant.countryCode}`);
  }
  return company;
}
```

Scope honesty: Nigeria and South Africa reports are documented as best-effort summaries built from the regulatory fields consistently required across states/SARS — they are explicitly *not* submission-ready FIRS TaxPro Max or SARS e@syFile exports, which need additional government-issued reference data this system doesn’t hold. Kenya’s KRA reports (P9/P10/NSSF/NHIF), by contrast, are the most complete because Kenya is the primary market.

6.2 Kenya — P9, P10, NSSF & NHIF/SHIF Remittance

P9 (annual tax deduction card, PDF) — one row per pay period across a tax year, for a single employee:

```

async generateP9(tenantId: string, companyId: string, employeeId: string, taxYear: string): Promise<Buffer> {
  await this.assertCompanyCountry(tenantId, companyId, 'KE');

  const entries = await this.prisma.payrollEntry.findMany({
    where: { employeeId, payrollRun: { companyId, periodStart: { gte: yearStart }, periodEnd: { lte: yearEnd } } },
    include: { payrollRun: true, employee: { include: { company: true } } },
    orderBy: { payrollRun: { periodStart: 'asc' } },
  });
  if (entries.length === 0) throw new NotFoundException('No payroll entries found for this employee in the given tax year');

  const rows: P9MonthRow[] = entries.map((entry) => {
    const statutory = entry.statutoryDeductions as StatutoryDeductionLine[];
    const tax = entry.taxBreakdown as TaxResult;
    return {
      period: entry.payrollRun.period, grossPay: entry.grossPay,
      nssf: findLine(statutory, 'NSSF')?.employeeAmount ?? 0,
      taxableIncome: tax.taxableIncome, grossTax: tax.grossTax, relief: tax.relief, payeTax: tax.netTax,
    };
  });
  const totals = rows.reduce((acc, row) => ({ /* sum every numeric field, period: 'TOTAL' */ }), /* zeroed accumulator */);

  const employee = entries[0]!.employee;
  const branding = await this.brandingService.getBranding(tenantId);
  return renderToBuffer(P9Document({
    branding, companyName: employee.company.name, employeeName: `${employee.firstName} ${employee.lastName}`,
    kraPin: this.encryptionService.decrypt(employee.kraPin), // decrypted only at the point of rendering into the PDF
    taxYear, currency: entries[0]!.currency, rows, totals,
  }));
}

```

Note `kraPin` is decrypted only at the point it's needed for the PDF — everywhere else in the request path it stays ciphertext.

P10 (monthly PAYE return, CSV) and the **NSSF / NHIF-SHIF remittance schedules** all follow the same shape: find the company's latest run for the period, walk `run.entries`, pull the relevant line via `findLine` / `findHealthInsuranceLine`, format as CSV.

```

const P10_CSV_HEADERS = ['period', 'employee_count', 'total_taxable_pay', 'total_paye', 'total_nssf', 'total_nhif'] as const;

async generateP10Csv(tenantId: string, companyId: string, period: string): Promise<string> {
  await this.assertCompanyCountry(tenantId, companyId, 'KE');
  const run = await this.findLatestRun(companyId, period);

  let totalTaxablePay = 0, totalPaye = 0, totalNssf = 0, totalNhif = 0;
  for (const entry of run.entries) {
    const statutory = entry.statutoryDeductions as StatutoryDeductionLine[];
    const tax = entry.taxBreakdown as TaxResult;
    totalTaxablePay += tax.taxableIncome;
    totalPaye += tax.netTax;
    totalNssf += findLine(statutory, 'NSSF')?.employeeAmount ?? 0;
    totalNhif += findHealthInsuranceLine(statutory)?.employeeAmount ?? 0; // resolves SHIF or NHIF per-entry
  }
  const row = [run.period, run.entries.length, totalTaxablePay.toFixed(2), totalPaye.toFixed(2), totalNssf.toFixed(2), totalNhif.toFixed(2)];
  return [P10_CSV_HEADERS.join(','), row.map(csvEscape).join(',')].join('\n');
}

```

`csvEscape` quotes any field containing a comma, quote, or newline and doubles embedded quotes — the minimum needed for a CSV that survives a name like `O'Brien, Jr.` intact:

```

function csvEscape(value: string | number): string {
  const str = String(value);
  return /[",\n]/.test(str) ? `"${str.replace(/"/g, '""')}"` : str;
}

```

NSSF and NHIF/SNIF remittance schedules are per-employee rows instead of one aggregate row — each includes the employee's decrypted statutory ID number (`nssfNumber` / `nhifNumber`) alongside the contribution amount, since that's what the remittance filing actually needs.

6.3 Nigeria — PAYE, Pension, NHF Remittance Schedules

Three CSV schedules, all gated to 'NG' tenants, each pulling a different statutory line:

- **PAYE remittance** (`generatePayeRemittanceCsv`) — per-employee `taxIdNumber` (decrypted), `taxableIncome`, `netTax`. Due the 10th of the following month to the employee's State Internal Revenue Service.
- **Pension remittance** (`generatePensionRemittanceCsv`) — per-employee `pensionNumber` (RSA PIN, decrypted), employee (8%) and employer (10%) amounts plus their combined total. Due within 7 working days of payment, remittable to the employee's PFA/PFC.
- **NHF remittance** (`generateNhfRemittanceCsv`) — per-employee employee-only contribution (2.5% of basic, no employer match — Part 3 §3.8), remittable to the Federal Mortgage Bank of Nigeria.

```
const NG_PENSION_CSV_HEADERS = ['employee_number', 'employee_name', 'rsa_pin', 'employee_amount', 'employer_amount', 'total'] as const;

async generatePensionRemittanceCsv(tenantId: string, companyId: string, period: string): Promise<string> {
  await this.assertCompanyCountry(tenantId, companyId, 'NG');
  const run = await this.findLatestRun(companyId, period);
  const rows = run.entries.map((entry) => {
    const pension = findLine(entry.statutoryDeductions as StatutoryDeductionLine[], 'PENSION');
    const employeeAmount = pension?.employeeAmount ?? 0, employerAmount = pension?.employerAmount ?? 0;
    return [
      entry.employee.employeeNumber ?? entry.employee.id,
      `${entry.employee.firstName} ${entry.employee.lastName}`,
      this.encryptedService.decrypt(entry.employee.pensionNumber) ?? '',
      employeeAmount.toFixed(2), employerAmount.toFixed(2), (employeeAmount + employerAmount).toFixed(2),
    ];
  });
  return [NG_PENSION_CSV_HEADERS.join(','), ...rows.map((r) => r.map(csvEscape).join(',')).join('\n')];
}
```

6.4 South Africa — EMP201 & IRP5

EMP201 is a single aggregated declaration row (not per-employee) — PAYE + UIF (both employee and employer sides) + SDL for the whole company's run in a period, due to SARS by the 7th of the following month:

```
const ZA_EMP201_CSV_HEADERS = ['period', 'employee_count', 'total_paye', 'total_uif_employee', 'total_uif_employer', 'total_sdl'] as const;

async generateEmp201Csv(tenantId: string, companyId: string, period: string): Promise<string> {
  await this.assertCompanyCountry(tenantId, companyId, 'ZA');
  const run = await this.findLatestRun(companyId, period);
  let totalPaye = 0, totalUifEmployee = 0, totalUifEmployer = 0, totalSdl = 0;
  for (const entry of run.entries) {
    const statutory = entry.statutoryDeductions as StatutoryDeductionLine[];
    const tax = entry.taxBreakdown as TaxResult;
    const uif = findLine(statutory, 'UIF'), sdl = findLine(statutory, 'SDL');
    totalPaye += tax.netTax;
    totalUifEmployee += uif?.employeeAmount ?? 0;
    totalUifEmployer += uif?.employerAmount ?? 0;
    totalSdl += sdl?.employerAmount ?? 0; // SDL is employer-only (Part 3 §3.9) — no employee-side field exists to sum
  }
  const row = [run.period, run.entries.length, totalPaye.toFixed(2), totalUifEmployee.toFixed(2), totalUifEmployer.toFixed(2), totalSdl.toFixed(2)];
  return [ZA_EMP201_CSV_HEADERS.join(','), row.map(csvEscape).join(',').join('\n')];
}
```

IRP5 is the South African analogue of Kenya's P9: a best-effort annual employee tax-certificate summary PDF, one row per pay period in the tax year, with gross remuneration, UIF contribution, taxable income, and PAYE — structurally identical to `generateP9` (same year-window query, same per-row-then-totals reduction, same `renderToBuffer` + branding pattern) but reading South Africa's **UIF** line instead of Kenya's **NSSF** line.

6.5 Adding a Fourth Country's Compliance Reports

The pattern scales the same way the payroll engine does: add the country's `CountryCode` literal to `assertCompanyCountry`'s union type, write CSV header constants + a service method per required filing, and pull whichever `StatutoryDeductionLine.code` that country's ruleset produces (Part 3 §3.6's registry). No existing country's report method needs to change.

Part 7 — Billing & Payments

7.1 Schema Recap & Provider Abstraction

Billing rests on four models from Part 2 §2.3 Cluster 6: `Plan` (per-country pricing catalog), `Subscription`, `Invoice`, `UsageRecord` (employee-count snapshots), and `PaymentTransaction` (raw provider-attempt log independent of `Invoice.status`).

Every payment provider — Paystack today, M-Pesa today, anything added later — implements one interface, so `BillingService` never branches on which provider a tenant picked:

```
// billing/providers/payment-provider.interface.ts
export interface ChargeParams {
  amount: number; currency: string; reference: string;
  customerId?: string;
  phoneNumber?: string; // required by phone-based providers like M-Pesa
  email?: string;       // required by hosted-checkout providers like Paystack
}

export interface ChargeResult { providerReference: string; status: string; raw: unknown; }

export interface PaymentProvider {
  readonly type: PaymentProviderType;
  createCustomer(customer: ProviderCustomer): Promise<string>;
  createSubscription(params: { customerId: string; planPricePerEmployee: number; currency: string; quantity: number }):
    Promise<ProviderSubscriptionResult>;
  charge(params: ChargeParams): Promise<ChargeResult>;
}
```

```
// billing.service.ts
private getProvider(type: PaymentProviderType): PaymentProvider {
  return type === PaymentProviderType.MPESA ? this.mpesaProvider : this.paystackProvider;
}
```

7.2 Paystack Provider

Paystack subscriptions need an existing card authorization (obtained from a prior successful transaction) that doesn't exist yet at signup time, so `createSubscription` just books a local billing period the same way M-Pesa does — actual collection happens per-invoice via `charge()`, which initializes a hosted-checkout transaction and returns a reference; confirmation arrives later, asynchronously, via webhook:

```
// billing/providers/paystack.provider.ts
@Injectable()
export class PaystackProvider implements PaymentProvider {
  readonly type = PaymentProviderType.PAYSTACK;

  async charge(params: ChargeParams): Promise<ChargeResult> {
    if (!this.secretKey) {
      this.logger.warn('PAYSTACK_SECRET_KEY not set – simulating a successful charge!');
      return { providerReference: `stub_paystack_${params.reference}`, status: 'succeeded', raw: null };
    }
    if (!params.email) throw new Error('Paystack charge requires an email');

    const response = await axios.post(`${PAYSTACK_BASE_URL}/transaction/initialize`, {
      email: params.email,
      amount: Math.round(params.amount * 100), // Paystack expects amounts in the smallest currency unit (kobo/cents)
      currency: params.currency, reference: params.reference,
      callback_url: `${this.callbackBaseUrl}/billing`,
    }, { headers: this.authHeaders() });

    return { providerReference: response.data.data.reference, status: 'pending', raw: response.data.data };
  }
}
```

Config-gated dev stub: every provider checks its own credentials before making a real API call and returns a deterministic stub result when unconfigured — this is what lets the entire billing flow (subscribe → invoice → pay → webhook confirm) be exercised end-to-end in local dev and CI with zero real payment credentials, while still exercising the exact same code path a production deployment uses.

7.3 M-Pesa (Daraja) Provider — STK Push

M-Pesa has no customer object (the phone number supplied at charge time is the identity) and no native recurring-subscription concept, so `charge()` is where the real integration work is: obtaining an OAuth token, building the Lipa Na M-Pesa password, and triggering an STK push prompt on the payer's phone.

```
// billing/providers/mpesa.provider.ts
async charge(params: ChargeParams): Promise<ChargeResult> {
  const mpesaConfig = this.configService.get('mpesa', { infer: true });
  if (!mpesaConfig.consumerKey || !mpesaConfig.consumerSecret || !mpesaConfig.shortcode || !mpesaConfig.passkey) {
    this.logger.warn('M-Pesa credentials not configured – simulating a successful STK push');
    return { providerReference: `stub_mpesa_${params.reference}`, status: 'pending', raw: null };
  }
  if (!params.phoneNumber) throw new Error('M-Pesa charge requires a phoneNumber');

  const accessToken = await this.getAccessToken(mpesaConfig.consumerKey, mpesaConfig.consumerSecret);
  const timestamp = this.formatTimestamp(new Date()); // yyyyMMddHHmmss
  const password = Buffer.from(`${mpesaConfig.shortcode}${mpesaConfig.passkey}${timestamp}`).toString('base64');

  // Safaricom does not sign STK push callbacks, so a shared-secret token is
  // embedded in the callback URL itself to reject spoofed requests.
  const callbackUrl = mpesaConfig.callbackToken
    ? `${mpesaConfig.callbackUrl}?token=${encodeURIComponent(mpesaConfig.callbackToken)}`
    : mpesaConfig.callbackUrl;

  const response = await axios.post(`${this.baseUrl}/mpesa/stkpush/v1/processrequest`, {
    BusinessShortCode: mpesaConfig.shortcode, Password: password, Timestamp: timestamp,
    TransactionType: 'CustomerPayBillOnline', Amount: Math.round(params.amount),
    PartyA: params.phoneNumber, PartyB: mpesaConfig.shortcode, PhoneNumber: params.phoneNumber,
    CallBackURL: callbackUrl, AccountReference: params.reference, TransactionDesc: params.reference,
  }, { headers: { Authorization: `Bearer ${accessToken}` } });

  return { providerReference: response.data.CheckoutRequestID, status: response.data.ResponseCode === '0' ? 'pending' : 'failed', raw: response.data };
}

private async getAccessToken(consumerKey: string, consumerSecret: string): Promise<string> {
  const credentials = Buffer.from(`${consumerKey}:${consumerSecret}`).toString('base64');
  const response = await axios.get(`${this.baseUrl}/oauth/v1/generate?grant_type=client_credentials`, { headers: {
    Authorization: `Basic ${credentials}` } });
  return response.data.access_token;
}
```

7.4 Webhook Verification — Signed vs. Unsigned

The two providers deliver payment confirmation completely differently, and the webhook controller handles each on its own terms rather than forcing a shared abstraction where none exists.

Paystack signs the webhook body with HMAC-SHA512 over the *exact raw bytes* — which is exactly why `main.ts` enables `rawBody: true` (Part 4 §4.2). Verification uses a constant-time comparison to avoid a timing side-channel:

```
// billing/payment-webhooks.controller.ts
@Public()
@Post('paystack')
@HttpCode(200)
async paystack(@Req() req: RawBodyRequest<Request>, @Headers('x-paystack-signature') signature: string | undefined) {
  const secretKey = this.configService.get('paystack', { infer: true }).secretKey;
  if (!secretKey || !req.rawBody) return { received: false };

  const expectedSignature = crypto.createHmac('sha512', secretKey).update(req.rawBody).digest('hex');
  if (!signature || !this.timingSafeEqualHex(signature, expectedSignature)) {
    this.logger.warn('Paystack webhook signature verification failed');
    return { received: false };
  }

  const event = JSON.parse(req.rawBody.toString('utf8'));
  const reference: string | undefined = event?.data?.reference;
  if (event.event === 'charge.success') {
    await this.billingService.confirmInvoicePaidByTransactionReference(PaymentProviderType.PAYSTACK, reference);
  } else if (event.event === 'charge.failed') {
    await this.billingService.recordTransactionFailure(PaymentProviderType.PAYSTACK, reference);
  }
  return { received: true };
}

private timingSafeEqualHex(a: string, b: string): boolean {
  const bufA = Buffer.from(a, 'hex'), bufB = Buffer.from(b, 'hex');
  if (bufA.length !== bufB.length) return false;
  return crypto.timingSafeEqual(bufA, bufB);
}
```

M-Pesa doesn't sign STK push callbacks at all, so the system embeds its own shared-secret token as a query parameter on the callback URL it registers with Safaricom, and checks that token on the way in instead:

```
@Public()
@Post('mpesa')
@HttpCode(200)
async mpesa(@Body() body: MpesaStkCallback, @Query('token') token: string | undefined) {
  const expectedToken = this.configService.get('mpesa', { infer: true }).callbackToken;
  if (expectedToken && token !== expectedToken) {
    // Safaricom only cares that we ack with ResultCode 0 – a mismatched
    // token means we simply drop the payload rather than process it.
    return { ResultCode: 0, ResultDesc: 'Accepted' };
  }
  const callback = body?.Body?.stkCallback;
  if (callback?.CheckoutRequestID) {
    if (callback.ResultCode === 0) {
      await this.billingService.confirmInvoicePaidByTransactionReference(PaymentProviderType.MPESA,
        callback.CheckoutRequestID);
    } else {
      await this.billingService.recordTransactionFailure(PaymentProviderType.MPESA, callback.CheckoutRequestID);
    }
  }
  return { ResultCode: 0, ResultDesc: 'Accepted' };
}
```

Both webhook endpoints are decorated `@Public()` (bypassing the global `JwtAuthGuard`) since the caller is Paystack's or Safaricom's servers, not an authenticated user — their own signature/token check is the actual authorization boundary here.

7.5 The Payment Flow, End to End

```
// billing.service.ts
async payInvoice(tenantId: string, invoiceId: string, dto: PayInvoiceDto) {
  const invoice = await this.prisma.invoice.findUnique({ where: { id: invoiceId } });
  if (!invoice || invoice.tenantId !== tenantId) throw new NotFoundException('Invoice not found');

  const provider = this.getProvider(invoice.provider);
  const result = await provider.charge({
    amount: invoice.amount, currency: invoice.currency, reference: invoice.id,
    phoneNumber: dto.phoneNumber, email: adminUser?.email,
  });

  // The provider.charge() HTTP call happens BEFORE the transaction starts –
  // an external call must never hold a DB transaction open. But the
  // resulting PaymentTransaction row and the conditional Invoice update
  // must commit together, or a crash between them could leave a
  // transaction recorded 'succeeded' against an invoice stuck OPEN forever.
  const chargeSucceeded = await this.prisma.$transaction(async (tx) => {
    await tx.paymentTransaction.create({
      invoiceId: invoice.id, provider: invoice.provider, reference: result.providerReference,
      amount: invoice.amount, currency: invoice.currency, status: result.status, rawResponse: result.raw,
    });
    // Paystack and real M-Pesa only ever return 'pending' here – actual
    // confirmation arrives later via the webhook. 'succeeded' only happens
    // synchronously for the unconfigured-provider dev stub.
    if (result.status === 'succeeded' || result.status === 'paid') {
      await this.markInvoicePaidTx(tx, invoice, result.providerReference);
      return true;
    }
    return false;
  });

  if (chargeSucceeded) await this.dispatchInvoicePaidSideEffects(tenantId, invoice);
  return result;
}

/** Called from both webhook handlers once a 'pending' charge actually completes. */
async confirmInvoicePaidByTransactionReference(provider: PaymentProviderType, transactionReference: string) {
  const transaction = await this.prisma.paymentTransaction.findFirst({ where: { provider, reference:
    transactionReference }, orderBy: { createdAt: 'desc' } });
  if (!transaction) return null;
  const invoice = await this.prisma.invoice.findUnique({ where: { id: transaction.invoiceId } });
  if (!invoice || invoice.status === InvoiceStatus.PAID) return invoice; // already handled – idempotent

  await this.prisma.$transaction(async (tx) => {
    await tx.paymentTransaction.update({ where: { id: transaction.id }, data: { status: 'succeeded' } });
    await this.markInvoicePaidTx(tx, invoice, transactionReference);
  });
  await this.dispatchInvoicePaidSideEffects(invoice.tenantId, invoice);
  return invoice;
}
```

The `chargeSucceeded` /side-effects split is deliberate: webhook dispatch and accounting sync are best-effort — wrapped in `.catch(() => {})` — and run only *after* the payment-state transaction has already committed, so a downstream integration failure can never roll back a real, already-confirmed payment.

7.6 Automated Monthly Billing Cycle

A cron job (`@nestjs/schedule`) runs on the 1st of every month and generates one invoice per `ACTIVE` subscription — `TRIALING` / `PAST_DUE` / `CANCELED` subscriptions are skipped:

```
// billing/billing-cycle.service.ts
@Injectable()
export class BillingCycleService {
  @Cron('0 6 1 * *') // 06:00 on the 1st of every month
  async runBillingCycle(): Promise<void> {
    const period = currentPeriod(); // "YYYY-MM"
    const subscriptions = await this.prisma.subscription.findMany({ where: { status: SubscriptionStatus.ACTIVE } });

    for (const subscription of subscriptions) {
      try {
        const alreadyBilled = await this.prisma.usageRecord.findUnique({
          where: { tenantId_period: { tenantId: subscription.tenantId, period } },
        });
        if (alreadyBilled) continue; // idempotency guard – safe to re-run the whole cycle for the same period
        await this.billingService.generateInvoice(subscription.tenantId, period);
      } catch (error) {
        this.logger.error(`Failed to generate invoice for tenant ${subscription.tenantId}, period ${period}`, error as Error);
      }
    }
  }
}
```

Idempotency here rides on the same `UsageRecord (tenantId, period)` unique constraint that `generateInvoice()` writes to via `recordUsage()` internally — the cron job doesn't need its own separate dedup mechanism, and a single tenant's invoicing failure (caught and logged inside the loop) never aborts the rest of the cycle for every other tenant.

Plan and currency safety: `subscribe()` refuses to attach a tenant to a plan priced for a different country (`plan.countryCode !== tenant.countryCode`) — the same single-currency-per-tenant invariant from Part 2 applied to billing, so a Kenyan tenant can never end up billed in Nigerian-priced Naira by a UI bug.

Part 8 — Notifications & Background Jobs

8.1 Two Independent Queues, One Reason: Keep the Request Thread Free

Two BullMQ queues exist in this system, and they exist for the same underlying reason — never let per-employee or per-channel fan-out work add latency to the HTTP request that triggered it:

1. **notifications queue** — fans a single logical notification out to email/SMS/push channels.
2. **payslip-emails queue** — renders and emails a payslip PDF for every entry in a just-completed payroll run.

Both follow the identical shape: a ***Service** writes synchronously what must be immediately consistent (an in-app **Notification** row; nothing, in the payslip case), then enqueues a job for everything that can happen asynchronously, and a ***Processor** (**WorkerHost**) consumes it off the request thread with BullMQ's built-in retry.

8.2 In-App Notifications + Multi-Channel Fan-Out

NotificationsService has two tiers of method: **create** / **createForRoles** write only the in-app row; **dispatch** / **dispatchForRoles** do that *and* enqueue out-of-band delivery.

```
// notifications/notifications.service.ts
@Injectable()
export class NotificationsService {
  constructor(
    private readonly prisma: PrismaService,
    @InjectQueue(NOTIFICATIONS_QUEUE) private readonly queue: Queue<NotificationDeliverJobData>,
  ) {}

  /** Best-effort, never-throw, logged — a notification failing to write must never break the request that triggered it. */
  async create(tenantId: string, userId: string, type: string, message: string, metadata?: Record<string, unknown>): Promise<void> {
    try {
      await this.prisma.notification.create({ data: { tenantId, userId, type, message, metadata } });
    } catch (error) {
      this.logger.error(`Failed to create notification (${type}) for user ${userId}`, error as Error);
    }
  }

  async dispatch(tenantId: string, userId: string, type: string, message: string, options?: DispatchOptions): Promise<void> {
    await this.create(tenantId, userId, type, message, options?.metadata);
    await this.enqueueDelivery(tenantId, userId, type, message, options);
  }

  private async enqueueDelivery(tenantId: string, userId: string, type: string, message: string, options?: DispatchOptions): Promise<void> {
    try {
      await this.queue.add(NOTIFICATION_DELIVER_JOB, {
        tenantId, userId, type, message, metadata: options?.metadata,
        channels: options?.channels ?? [NotificationChannel.EMAIL],
      });
    } catch (error) {
      this.logger.error(`Failed to enqueue notification delivery (${type}) for user ${userId}`, error as Error);
    }
  }
}
```

Every method here is wrapped in try/catch-and-log rather than letting an error propagate — this is a deliberate, repeated pattern across the whole notifications layer (mirrored in **MailService.sendMail**, **PayslipEmailService.enqueueForRun**, and **AuditService.record**): **a service call that's awaited directly in a primary request's path must never be able to fail that request**. A payroll run that completed successfully stays completed even if the email provider is down. (**PayslipEmailService.sendPayslipEmail** itself is the one exception, and deliberately so — see Part 17 §17.2: once payslip delivery moved to one BullMQ job per entry, that function is only ever called from inside the queue processor, where letting an error through is what makes BullMQ retry it.)

The processor consumes the job and fans out to whichever channels were requested:

```
// notifications/notifications.processor.ts
@Processor(NOTIFICATIONS_QUEUE)
export class NotificationsProcessor extends WorkerHost {
  async process(job: Job<NotificationDeliverJobData>): Promise<void> {
    const { userId, type, message, metadata, channels } = job.data;
    const user = await this.prisma.user.findUnique({ where: { id: userId } });
    if (!user) return;

    const rendered = renderNotificationTemplate(type, message, metadata);
    for (const channel of channels) {
      switch (channel) {
        case NotificationChannel.EMAIL:
          await this.mailService.sendMail(user.email, rendered.subject, rendered.html);
          break;
        case NotificationChannel.SMS:
          if (!user.phone) { this.logger.warn(`Skipping SMS for user ${userId}: no phone on file`); break; }
          await this.smsProvider.send(user.phone, rendered.sms);
          break;
        case NotificationChannel.PUSH:
          await this.pushProvider.send(userId, rendered.subject, rendered.sms);
          break;
      }
    }
  }
}
```

`SmsProvider` and `PushProvider` are both injected via interface tokens (`SMS_PROVIDER`, `PUSH_PROVIDER`), letting `notifications.module.ts` swap in a no-op implementation when unconfigured — the same config-gated pattern as the payment providers (Part 7 §7.2).

8.3 Mail Service

A thin, direct wrapper around `nodemailer` — deliberately not `@nestjs-modules/mailer` — mirroring the “wrap the SDK directly” pattern used for Paystack/M-Pesa:

```
// notifications/mail.service.ts
@Injectable()
export class MailService {
  private transporter: nodemailer.Transporter | null = null;

  private getTransporter(): nodemailer.Transporter {
    if (!this.transporter) {
      const smtp = this.configService.get('smtp', { infer: true });
      this.transporter = nodemailer.createTransport({
        host: smtp.host, port: smtp.port,
        auth: smtp.user && smtp.pass ? { user: smtp.user, pass: smtp.pass } : undefined,
      });
    }
    return this.transporter;
  }

  async sendMail(to: string, subject: string, html: string, attachments?: MailAttachment[]): Promise<void> {
    try {
      const smtp = this.configService.get('smtp', { infer: true });
      await this.getTransporter().sendMail({ from: smtp.from, to, subject, html, attachments });
    } catch (error) {
      this.logger.error(`Failed to send mail to ${to}: ${subject}`, error as Error);
    }
  }
}
```

8.4 SMS — Africa’s Talking

Real HTTP integration, config-gated exactly like the payment providers — inert until

`AFRICAS_TALKING_API_KEY` / `AFRICAS_TALKING_USERNAME` are set, at which point a factory in `notifications.module.ts` selects it over a `NoopSmsProvider` :

```
// notifications/providers/africas-talking-sms.provider.ts
@Injectable()
export class AfricasTalkingSmsProvider implements SmsProvider {
  async send(to: string, message: string): Promise<SmsSendResult> {
    const config = this.configService.get('africasTalking', { infer: true });
    if (!config.apiKey || !config.username) {
      return { success: false, error: 'SMS provider not configured' };
    }

    const body = new URLSearchParams({ username: config.username, to, message, ...(config.senderId ? { from:
      config.senderId } : {}) });
    const response = await axios.post<AfricasTalkingResponse>(API_URL, body, {
      headers: { apiKey: config.apiKey, 'Content-Type': 'application/x-www-form-urlencoded', Accept: 'application/json'
    },
    });

    const recipient = response.data.SMSMessageData?.Recipients?.[0];
    return recipient?.status === 'Success' ? { success: true } : { success: false, error: recipient?.status };
  }
}
```

8.5 Payslip Email — Bulk Dispatch Off the Payroll-Run Thread

This is the direct counterpart, in Part 5's payroll run, to §8.1's general principle. `PayrollService.executeRun` enqueues rather than sends inline:

```

// notifications/payslip-email.service.ts
@Injectable()
export class PayslipEmailService {
  /** Call this from request-handling code – enqueues, doesn't send. */
  async enqueueForRun(tenantId: string, payrollRunId: string): Promise<void> {
    await this.payslipEmailsQueue.add(PAYSLIP_EMAILS_DELIVER_JOB, { tenantId, payrollRunId });
  }

  /** Never throws. Called by the queue processor, not directly from request-handling code. */
  async sendPayslipEmailsForRun(tenantId: string, payrollRunId: string): Promise<void> {
    try {
      const run = await this.prisma.payrollRun.findUnique({ where: { id: payrollRunId }, include: { entries: true } });
      if (!run) return;
      for (const entry of run.entries) {
        await this.sendPayslipEmail(tenantId, entry.id); // sequential + awaited – see note below
      }
    } catch (error) {
      this.logger.error(`Failed to send payslip emails for run ${payrollRunId}`, error as Error);
    }
  }

  async sendPayslipEmail(tenantId: string, payrollEntryId: string): Promise<void> {
    try {
      const entry = await this.prisma.payrollEntry.findUnique({
        where: { id: payrollEntryId }, include: { employee: { include: { user: true, company: true } }, payrollRun: true },
      });
      if (!entry || entry.employee.company.tenantId !== tenantId) return;
      const email = entry.employee.user?.email;
      if (!email) return; // employee has no linked login – nowhere to send a payslip email

      const buffer = await this.payslipsService.generate(tenantId, payrollEntryId); // reuses Part 5's PDF renderer directly
      await this.mailService.sendMail(email, `Your Payslip – ${entry.payrollRun.period}`, `<p>Hi ${entry.employee.firstName},</p>...`, [{ filename: 'payslip.pdf', content: buffer }]);
    } catch (error) {
      this.logger.error(`Failed to send payslip email for entry ${payrollEntryId}`, error as Error);
    }
  }
}

```

`sendPayslipEmailsForRun` loops sequentially and awaits each send — a known, documented tradeoff (adds latency proportional to headcount for very large companies) rather than a hidden one; a bounded-concurrency map (the same `mapWithConcurrency` pattern from Part 5 §5.2) is the natural upgrade path if that ever becomes a real bottleneck, but wasn't needed yet.

The processor is almost embarrassingly thin, because all the error handling already lives in the service method it calls:

```

// notifications/payslip-emails.processor.ts
@Processor(PAYSLIP_EMAILS_QUEUE)
export class PayslipEmailsProcessor extends WorkerHost {
  async process(job: Job<PayslipEmailsJobData>): Promise<void> {
    const { tenantId, payrollRunId } = job.data;
    await this.payslipEmailService.sendPayslipEmailsForRun(tenantId, payrollRunId);
  }
}

```

8.6 Queue Registration

Each queue is registered with `BullModule.registerQueue({ name: ... })` inside its owning feature module (`notifications.module.ts`), on top of the root Redis connection configured once in `AppModule` (Part 4 §4.3):

```
// app.module.ts
BullModule.forRootAsync({
  useFactory: (configService: ConfigService<AppConfig, true>) => ({
    connection: new Redis(configService.get('redisUrl', { infer: true })) ?? 'redis://localhost:6379', {
      maxRetriesPerRequest: null },
  }),
  inject: [ConfigService],
}),
```

`maxRetriesPerRequest: null` is required by BullMQ's blocking-connection model — without it, ioredis's own retry limit fights with BullMQ's internal reconnect logic on a long-lived worker connection.

Part 9 — HR Features

9.1 Leave Management

Lazy, on-read accrual — no cron job. Rather than a nightly job that increments every employee’s leave balance, `getOrCreateBalance` recomputes `accruedDays` deterministically every time it’s called, from first principles: whole calendar months elapsed since the later of “Jan 1 of the year” or “the employee’s earliest contract start date,” pro-rated against the leave type’s annual entitlement.

```
// leave/leave.service.ts
function wholeMonthsBetween(start: Date, end: Date): number {
  if (end <= start) return 0;
  let months = (end.getFullYear() - start.getFullYear()) * 12 + (end.getMonth() - start.getMonth());
  if (end.getDate() < start.getDate()) months -= 1;
  return Math.max(0, months);
}

async getOrCreateBalance(tenantId: string, employeeId: string, leaveTypeId: string, year: number) {
  const leaveType = await this.prisma.leaveType.findUnique({ where: { id: leaveTypeId } });
  const earliestContract = await this.prisma.contract.findFirst({ where: { employeeId }, orderBy: { startDate: 'asc' } });

  const jan1 = new Date(Date.UTC(year, 0, 1));
  // Exclusive upper bound (Jan 1 of the FOLLOWING year), not Dec 31 — "whole
  // calendar months" from Jan 1 to Dec 31 is only 11 months (one day short
  // of a full 12th), which would make a fully-elapsed year permanently
  // under-accrue by one month's worth of leave.
  const yearEndExclusive = new Date(Date.UTC(year + 1, 0, 1));
  const today = new Date();

  const periodStart = earliestContract && earliestContract.startDate > jan1 ? earliestContract.startDate : jan1;
  const periodEnd = today < yearEndExclusive ? today : yearEndExclusive;
  const monthsElapsed = wholeMonthsBetween(periodStart, periodEnd);
  const accruedDays = Math.min(leaveType.daysPerYear, (monthsElapsed * leaveType.daysPerYear) / 12);

  return this.prisma.leaveBalance.upsert({
    where: { employeeId_leaveTypeId_year: { employeeId, leaveTypeId, year } },
    create: { employeeId, leaveTypeId, year, accruedDays, usedDays: 0 },
    update: { accruedDays }, // usedDays is untouched here — only a decision changes it
  });
}
```

Recomputing and upserting on every call keeps `accruedDays` monotonic and idempotent regardless of how many times it’s read in a given month. `usedDays` is deliberately left untouched by this method — it only changes when a leave request is actually approved (below). Deliberate scope cuts, called out explicitly rather than left implicit: no annual-upfront-grant alternative, no year-to-year carry-over, no nightly accrual job.

Requesting leave deducts public holidays (Part 9.3) from the requested date range, then checks the balance computed above:

```

async createLeaveRequest(tenantId: string, employeeId: string, dto: CreateLeaveRequestDto) {
  const leaveType = await this.prisma.leaveType.findUnique({ where: { id: dto.leaveTypeId } });
  const startDate = new Date(dto.startDate), endDate = new Date(dto.endDate);
  if (endDate < startDate) throw new BadRequestException('endDate cannot be before startDate');

  // Calendar days inclusive, minus public holidays for the tenant's country.
  // Simplification, stated explicitly: weekends are still not excluded.
  const tenant = await this.prisma.tenant.findUniqueOrThrow({ where: { id: tenantId } });
  const calendarDays = Math.round((endDate.getTime() - startDate.getTime()) / MS_PER_DAY) + 1;
  const holidaysInRange = this.holidaysService.countHolidaysInRange(tenant.countryCode, startDate, endDate);
  const daysRequested = Math.max(0, calendarDays - holidaysInRange);

  if (leaveType.isPaid) {
    const balance = await this.getOrCreateBalance(tenantId, employeeId, leaveType.id, startDate.getFullYear());
    if (balance.accumulatedDays - balance.usedDays < daysRequested) {
      throw new BadRequestException('Insufficient leave balance');
    }
  }

  const request = await this.prisma.leaveRequest.create({
    data: { employeeId, leaveTypeId: leaveType.id, startDate, endDate, daysRequested, reason: dto.reason, status:
      'PENDING' },
  });
  await this.notificationsService.createForRoles(tenantId, [Role.ADMIN, Role.HR], 'LEAVE_REQUEST_PENDING', `...`, {
    leaveRequestId: request.id });
  return request;
}

```

Approval increments `usedDays` on the balance row created above, and notifies the employee's linked `User` (if they have portal access):

```

async decide(tenantId: string, approverId: string, requestId: string, decision: 'APPROVED' | 'REJECTED') {
  const request = await this.prisma.leaveRequest.findUnique({ where: { id: requestId }, include: { employee: { include:
    { company: true, user: true } }, leaveType: true } });
  if (!request || request.employee.company.tenantId !== tenantId) throw new NotFoundException('Leave request not
    found');
  if (request.status !== 'PENDING') throw new BadRequestException('Leave request has already been decided');

  const updated = await this.prisma.leaveRequest.update({ where: { id: requestId }, data: { status: decision,
    approverId, decidedAt: new Date() } });

  if (decision === 'APPROVED' && request.leaveType.isPaid) {
    // Guaranteed to exist – createLeaveRequest already called
    // getOrCreateBalance for this exact employee/leaveType/year.
    await this.prisma.leaveBalance.update({
      where: { employeeId_leaveTypeId_year: { employeeId: request.employeeId, leaveTypeId: request.leaveTypeId, year:
        request.startDate.getFullYear() } },
      data: { usedDays: { increment: request.daysRequested } },
    });
  }
  if (request.employee.user) {
    await this.notificationsService.create(tenantId, request.employee.user.id, 'LEAVE_REQUEST_DECIDED', `Your leave
      request has been ${decision.toLowerCase()}`, { leaveRequestId: request.id, decision });
  }
  return updated;
}

```

9.2 Loans & Advances

A rejected loan never touches the payroll pipeline — the repayment schedule (`LoanRepayment` rows) is only generated the moment a loan is approved, computed as an even split of `principal` across `installments`, with rounding remainder absorbed into the final installment so the schedule always sums to exactly `principal`:

```
// loans/loans.service.ts
async decide(tenantId: string, actorId: string, loanId: string, dto: DecideLoanDto) {
  const loan = await this.findLoanOrThrow(tenantId, loanId);
  if (loan.status !== 'PENDING') throw new BadRequestException('Only pending loans can be approved or rejected');

  if (dto.decision === 'REJECTED') {
    const updated = await this.prisma.loan.update({ where: { id: loanId }, data: { status: 'REJECTED', approvedById: actorId, decidedAt: new Date(), closedAt: new Date() } });
    await this.notifyEmployee(tenantId, loan.employeeId, 'LOAN_REJECTED', `Your loan request has been rejected...`, { loanId });
    return updated;
  }

  const installmentAmount = round2(loan.principal / loan.installments);
  const repaymentsData = Array.from({ length: loan.installments }, (_, i) => {
    const isLast = i === loan.installments - 1;
    // Last installment absorbs the rounding remainder so the schedule sums
    // exactly to principal, never a cent over or under due to round2().
    const amountDue = isLast ? round2(loan.principal - installmentAmount * (loan.installments - 1)) : installmentAmount;
    return { loanId, installmentNo: i + 1, period: addMonthsToPeriod(loan.startPeriod, i), amountDue };
  });

  const updated = await this.prisma.$transaction(async (tx) => {
    await tx.loanRepayment.createMany({ data: repaymentsData });
    return tx.loan.update({ where: { id: loanId }, data: { status: 'ACTIVE', installmentAmount, approvedById: actorId, decidedAt: new Date() } });
  });
  await this.notifyEmployee(tenantId, loan.employeeId, 'LOAN_APPROVED', `Your loan of ${loan.currency} ${loan.principal} has been approved...`, { loanId });
  return updated;
}
```

The payroll integration point — this is the function `PayrollService.executeRun` calls per-employee (Part 5 §5.2) before the pure engine even runs, folding any installment due *this period* into the same `deductions.voluntary` map salary components use:

```
async resolvePayrollDeductions(tenantId: string, employeeId: string, period: string): Promise<{ voluntaryDeductions: Record<string, number>; repayments: DueLoanRepayment[] }> {
  const due = await this.prisma.loanRepayment.findMany({
    where: { period, status: 'PENDING', loan: { employeeId, tenantId, status: 'ACTIVE' } },
  });
  const voluntaryDeductions: Record<string, number> = {};
  for (const repayment of due) {
    // Keyed per loan so multiple concurrent loans stay distinguishable on the payslip.
    voluntaryDeductions[`LOAN_REPAYMENT_${repayment.loanId}`] = repayment.amountDue;
  }
  return { voluntaryDeductions, repayments: due.map((r) => ({ id: r.id, loanId: r.loanId, amountDue: r.amountDue })) };
}
```

Then, once the run has *already committed*, `markRepaymentsPaid` closes the loop — and, like the webhook side-effects in Part 7, it never throws, because a notification/bookkeeping failure here must not retroactively fail an already-committed payroll run:

```
async markRepaymentsPaid(repayments: DueLoanRepayment[], payrollEntryId: string): Promise<void> {
  const affectedLoanIds = new Set<string>();
  try {
    for (const repayment of repayments) {
      await this.prisma.loanRepayment.update({ where: { id: repayment.id }, data: { status: 'PAID', amountPaid: repayment.amountDue, paidAt: new Date(), payrollEntryId } });
      affectedLoanIds.add(repayment.loanId);
    }
  } catch (error) {
    this.logger.error('Failed to mark loan repayments paid', error as Error);
    return; // worst case, reconciled manually later
  }
  for (const loanId of affectedLoanIds) await this.closeIfFullyRepaid(loanId); // auto-transitions to PAID_OFF once every installment clears
}
```

Early payoff is the mirror image: remaining `PENDING` installments are marked `SKIPPED`, never `PAID` — because the balance was settled outside payroll (e.g. cash), and `PAID` would falsely imply it came out of a payslip.

9.3 Public Holidays

A small, pure lookup service — country-and-year holiday definitions come from a static table (`holiday-definitions.ts`), not an external API, so it works fully offline and deterministically:

```
// holidays/holidays.service.ts
@Injectable()
export class HolidaysService {
  listForCountry(countryCode: string, year: number): HolidayDefinition[] {
    return getHolidayDefinitions(countryCode, year).sort((a, b) => a.date.getTime() - b.date.getTime());
  }

  isHoliday(countryCode: string, date: Date): boolean {
    return getHolidayDefinitions(countryCode, date.getUTCFullYear()).some((h) => isSameUtcDate(h.date, date));
  }

  /** Count of public holidays falling within [start, end], inclusive, spanning any number of years. */
  countHolidaysInRange(countryCode: string, start: Date, end: Date): number {
    let count = 0;
    for (let year = start.getUTCFullYear(); year <= end.getUTCFullYear(); year++) {
      for (const holiday of getHolidayDefinitions(countryCode, year)) {
        if (holiday.date >= start && holiday.date <= end) count++;
      }
    }
    return count;
  }
}
```

`countHolidaysInRange` is the direct dependency Leave Management uses (§9.1) to avoid charging an employee's balance for a public holiday that happens to fall inside their requested date range. Looping year-by-year (rather than assuming a request never crosses a year boundary) is what makes a leave request spanning, say, late December into early January still count both years' holiday tables correctly.

9.4 Attendance

`AttendanceRecord` (Part 2 §2.3 Cluster 5) is a simple one-row-per-employee-per-day model — `@@unique([employeeId, date])` enforces that a given day can only have one status (`PRESENT` / `ABSENT` / `LEAVE`) recorded for a given employee, so re-marking the same day is an upsert, not a duplicate insert. `AttendanceService` is a thin CRUD layer with the same tenant-ownership check pattern as every other module (`findOne` → verify `employee.company.tenantId === tenantId` before allowing read/write) and a `@Roles(Role.ADMIN, Role.HR)` + `@RequirePermission(Permission.ATTENDANCE_MANAGE)` gate on writes. Attendance is currently a standalone HR record — it does not yet feed into payroll proration (Part 3 §3.4's engine-level proration is driven by employment start/end dates, not day-by-day attendance).

9.5 Employee Documents

Documents (contracts, ID scans, certificates) are stored on local disk under `DOCUMENT_STORAGE_DIR`, one subdirectory per employee, with a UUID-prefixed filename to avoid collisions — the DB row is the source of truth for metadata, the filesystem just holds bytes:

```
// documents/documents.service.ts
async upload(tenantId: string, employeeId: string, actor: AuthenticatedRequestUser, file: Express.Multer.File, type:
string) {
  await this.assertAccess(tenantId, employeeId, actor); // EMPLOYEE role may only touch their own; ADMIN/HR unrestricted
  within tenant

  const employeeDir = join(STORAGE_DIR, employeeId);
  await mkdir(employeeDir, { recursive: true });
  const fileName = `${randomUUID()}-${file.originalname}`;
  await writeFile(join(employeeDir, fileName), file.buffer);

  return this.prisma.document.create({
    data: { employeeId, type, fileName: file.originalname, mimeType: file.mimetype, sizeBytes: file.size, url:
      join(employeeId, fileName), uploadedById: actor.id },
  });
}

private async assertAccess(tenantId: string, employeeId: string, actor: AuthenticatedRequestUser) {
  await this.employeesService.findOne(tenantId, employeeId); // throws NotFoundException on cross-tenant/unknown
  employee
  if (actor.role === Role.EMPLOYEE && actor.employeeId !== employeeId) {
    throw new ForbiddenException('You may only access your own documents');
  }
}
```

Deletion mirrors the DB-then-disk order deliberately: the `Document` row is deleted first, then the file is unlinked in a best-effort try/catch — a missing file on disk (already cleaned up some other way) must never block removing the metadata row that a user explicitly asked to delete:

```
async remove(tenantId: string, documentId: string, actor: AuthenticatedRequestUser) {
  const document = await this.findOne(tenantId, documentId, actor);
  await this.prisma.document.delete({ where: { id: documentId } });
  try {
    await unlink(join(STORAGE_DIR, document.url));
  } catch (error) {
    this.logger.warn(`Failed to unlink document file for ${documentId}: ${error as Error}.message`);
  }
}
```

With HR operations complete, Part 10 moves to the frontend — starting with the architectural patterns (API client, auth/branding context) that every page in Part 11 builds on.

Part 10 — Frontend Foundations & Marketing Site

10.1 Route Groups: Three Audiences, One App

Next.js App Router route groups split the app by audience without affecting the URL structure:

```
app/
├─ (marketing)/    # public: /, /pricing, /reviews, /features, /contact, /privacy, ...
├─ (auth)/         # /login, /signup — no sidebar, no auth required
├─ (app)/          # authenticated: /dashboard, /employees, /payroll, /leave, /loans, ...
├─ offline/       # Serwist fallback route, shown when navigation fails with no network
└─ sw.ts          # service worker source
```

Each group has its own `layout.tsx`: `(marketing)` wraps pages in the public `Header / Footer`; `(app)` wraps pages in the authenticated shell (sidebar, `AuthProvider`, `BrandingProvider`, React Query's `QueryClientProvider`) and redirects to `/login` when `useAuth()` resolves with no user.

10.2 The API Client — Token Refresh Is Transparent to Every Caller

Every backend module gets a matching `lib/*-api.ts` file on the frontend (`employees-api.ts`, `payroll-api.ts`, `leave-api.ts`, `loans-api.ts`, ...) — thin, typed wrapper functions around one shared `apiFetch`. The interesting logic lives entirely in that one shared function, so no individual page or hook ever has to think about auth:

```
// lib/api-client.ts
let refreshPromise: Promise<AuthTokensDto> | null = null;

async function refreshTokens(): Promise<AuthTokensDto> {
  const refreshToken = tokenStorage.getRefreshToken();
  if (!refreshToken) throw new ApiError(401, 'No refresh token');
  const response = await fetch(`${API_URL}/auth/refresh`, { method: 'POST', headers: { Authorization: `Bearer ${refreshToken}` } });
  if (!response.ok) { tokenStorage.clear(); throw new ApiError(response.status, 'Session expired'); }
  const data = await response.json();
  tokenStorage.setTokens(data.accessToken, data.refreshToken);
  return data;
}

export async function apiFetch<T>(path: string, options: ApiFetchOptions = {}, isRetry = false): Promise<T> {
  const { skipAuth, headers, ...rest } = options;
  const accessToken = tokenStorage.getAccessToken();
  const response = await fetch(`${API_URL}${path}`, {
    ...rest,
    headers: {
      ...(rest.body && !(rest.body instanceof FormData) ? { 'Content-Type': 'application/json' } : {}),
      ...(accessToken && !skipAuth ? { Authorization: `Bearer ${accessToken}` } : {}),
      ...headers,
    },
  });
  if (response.status === 401 && !skipAuth && !isRetry) {
    try {
      // Concurrent 401s from several in-flight requests share ONE refresh
      // call rather than each independently racing to refresh – the second
      // caller awaits the first's in-flight promise instead of triggering
      // its own token rotation.
      refreshPromise ??= refreshTokens().finally(() => { refreshPromise = null; });
      await refreshPromise;
      return apiFetch<T>(path, options, true); // retry once, with the new token
    } catch {
      tokenStorage.clear();
      throw new ApiError(401, 'Session expired, please log in again');
    }
  }
  if (!response.ok) {
    const body = await response.json().catch(() => ({ message: response.statusText }));
    throw new ApiError(response.status, body.message ?? 'Request failed');
  }
  if (response.status === 204) return undefined as T;
  return response.json() as Promise<T>;
}
```

The `refreshPromise` module-level singleton is the detail worth internalizing: without it, five simultaneous 401s (e.g. a dashboard firing off five parallel React Query fetches right as the access token expires) would each independently call `/auth/refresh`, racing to rotate the same refresh token — and because `AuthService.refresh` treats a stale/rotated-out refresh token as reuse and revokes the session (Part 4 §4.4), that race would log the user out. Coalescing every concurrent 401 onto one in-flight refresh call avoids that entirely.

A second helper handles binary responses — payslip PDFs, bank-export CSVs — which don't fit the JSON-in/JSON-out shape:

```
export async function apiDownload(path: string, filename: string): Promise<void> {
  const response = await fetch(`${API_URL}${path}`, { headers: accessToken ? { Authorization: `Bearer ${accessToken}` } : {} });
  const blob = await response.blob();
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url; link.download = filename;
  document.body.appendChild(link); link.click(); link.remove();
  URL.revokeObjectURL(url);
}
```

10.3 AuthContext & BrandingContext

`AuthContext` owns the current user and the three auth mutations; it resolves `/auth/me` once on mount if a token is already present (surviving a page refresh) and otherwise starts unauthenticated:

```
// contexts/AuthContext.tsx
export function AuthProvider({ children }: PropsWithChildren) {
  const [user, setUser] = useState<AuthenticatedUserDto | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    if (!tokenStorage.getAccessToken()) { setIsLoading(false); return; }
    apiFetch<AuthenticatedUserDto>('/auth/me').then(setUser).catch(() => tokenStorage.clear()).finally(() =>
      setIsLoading(false));
  }, []);

  const login = useCallback(async (email: string, password: string) => {
    const data = await apiFetch<{ user: AuthenticatedUserDto } & AuthTokensDto>('/auth/login', {
      method: 'POST', body: JSON.stringify({ email, password }), skipAuth: true,
    });
    tokenStorage.setTokens(data.accessToken, data.refreshToken);
    setUser(data.user);
  }, []);
  // signup, logout follow the same shape
  return <AuthContext.Provider value={{ user, isLoading, login, signup, logout }}>{children}</AuthContext.Provider>;
}
```

`BrandingContext` layers on top of it: authenticated users get their tenant's actual branding (`/branding`); everyone else — including the entire public marketing site — gets a sensible fallback (`/branding/default` , which resolves to `APP_NAME`) so a payslip preview or the app shell never renders with missing logo/color data mid-fetch:

```
// contexts/BrandingContext.tsx
const DEFAULT_BRANDING: BrandingConfigDto = { appName: APP_NAME };

export function BrandingProvider({ children }: PropsWithChildren) {
  const { user } = useAuth();
  const [branding, setBranding] = useState<BrandingConfigDto>(DEFAULT_BRANDING);

  const fetchBranding = useCallback(async () => {
    const path = user ? '/branding' : '/branding/default';
    try {
      setBranding(await apiFetch<BrandingConfigDto>(path, user ? {} : { skipAuth: true }));
    } catch {
      setBranding(DEFAULT_BRANDING);
    }
  }, [user]);

  useEffect(() => { void fetchBranding(); }, [fetchBranding]);
  return <BrandingContext.Provider value={{ ...branding, refreshBranding: fetchBranding }}>{children}>
    </BrandingContext.Provider>;
}
```

Refetching automatically when `user` changes (via the `useCallback` dependency) is what makes logging in as a different tenant immediately swap the visible logo/color — no manual cache invalidation needed anywhere else in the app.

10.4 Data Fetching — React Query, No Hand-Rolled `useEffect`

Every data-driven page follows the same shape: a `useQuery` for reads, a `useMutation` (with `queryClient.invalidateQueries` on success) for writes, and explicit loading/error rendering — never a bare `useEffect` fetch with local `useState` for loading flags. A representative page (Part 11 covers the full payroll page):

```

const calculationQuery = useQuery({
  queryKey: ['payroll-calculate', country, salaryValue, allowances, deductions],
  queryFn: () => calculatePayroll({ country, salary: salaryValue, allowances, deductions }),
  enabled: salaryValue > 0,
});

// ...
{calculationQuery.isPending ? (
  <SkeletonRows count={6} />
) : calculationQuery.isError ? (
  <p className="text-red-600 dark:text-red-400 text-sm">
    {calculationQuery.error instanceof ApiError ? calculationQuery.error.message : 'Failed to calculate. Please try again.'}
  </p>
) : calculationQuery.data ? (
  <BreakdownView result={calculationQuery.data} />
) : null}

```

Inputs are debounced (`useDebounceValue` , 400–500ms) before entering the query key, so typing in the pricing calculator’s salary field doesn’t fire a network request per keystroke — React Query’s key-based caching then means moving back to a previously-typed value returns the cached result instantly with no request at all.

10.5 The Public Payroll Calculator — Same Engine, Zero Auth

The marketing-site `/pricing` page embeds a fully live tax calculator, unauthenticated, calling the `payroll-calculator` module from Part 5 §5.4 directly. Because it’s the exact same `runPayrollCalculation` + country registry as real payroll, the country dropdown and every number shown is provably accurate, not a marketing approximation:

```

// app/(marketing)/pricing/page.tsx
const countriesQuery = useQuery({ queryKey: ['calculator-countries'], queryFn: listSupportedCountries, staleTime: Infinity });

const calculationQuery = useQuery({
  queryKey: ['payroll-calculate', country, salaryValue, allowances, deductions],
  queryFn: () => calculatePayroll({ country, salary: salaryValue, allowances, deductions }),
  enabled: salaryValue > 0,
});

<Select value={country} onChange={setCountry}>
  <SelectContent>
    {(countriesQuery.data ?? []).map((c) => (
      <SelectItem key={c.countryCode} value={c.countryCode}>{getCountryName(c.countryCode)} {c.currency}</SelectItem>
    ))}
  </SelectContent>
</Select>

```

Rendering `getCountryName(c.countryCode)` (“South Africa (ZAR)”) instead of the raw ISO pair (“ZA — ZAR”) is a small but real UX fix: users reading a bare `ZA` code in a dropdown reasonably mistake it for an abbreviation of an unfamiliar country rather than recognizing it as South Africa, which is precisely the kind of confusion that makes people think a country “isn’t supported” when it actually is.

`getPricingForCountry / formatPrice` (from `@repo/pricing` , the same shared package `AuthService.signup` and `BillingService` use — Part 4 §4.4, Part 7 §7.6) drive both the plan-tier cards above the calculator and the calculator’s own currency formatting, so pricing display and actual billing currency can never drift apart.

Statutory deduction line items get an inline tooltip explaining what they are (`DEDUCTION_TOOLTIPS` , keyed by the same `StatutoryDeductionLine.code` the engine produces — `NSSF` , `NHIF` , `HOUSING_LEVY` , `PENSION` , `NHF` , `UIF` , `SDL`), and any `ValidationIssue` s the engine’s `validate()` returns (Part 3 §3.2) are surfaced directly in a warning box — the same validation real payroll runs would hit.

10.6 Reviews & Testimonials

Marketing testimonials live in one shared data file consumed by two surfaces — the homepage teaser (first 3) and a dedicated `/reviews` page (all of them) — so there’s one place to add a new review rather than two copies drifting apart:

```
// lib/testimonials-data.ts
export interface Testimonial { name: string; role: string; company: string; quote: string; }
export const testimonials: Testimonial[] = [ /* ... */ ];
```

```
// app/(marketing)/reviews/page.tsx
export default function ReviewsPage() {
  return (
    <div className="py-20">
      {testimonials.map((t) => <TestimonialCard key={t.name} {...t} />)}
      <Link href="/contact">Share Your Story</Link>
    </div>
  );
}
```

`TestimonialsSection` on the homepage renders `testimonials.slice(0, 3)` plus a “Read more reviews” link to `/reviews` — the header’s “Reviews” nav item points at `/reviews` directly rather than an in-page `#testimonials` anchor, so it’s a real page from every entry point, not just a homepage scroll target.

10.7 Offline Support — Serwist

The employee self-service portal needs to show a cached payslip with no connection, so the app registers a real service worker via `@serwist/next` rather than skipping PWA support entirely:

```
// app/sw.ts — runs in ServiceWorkerGlobalScope, transpiled/bundled by Next.js/Serwist at build time
import { defaultCache } from '@serwist/next/worker';
import { Serwist } from 'serwist';

const serwist = new Serwist({
  precacheEntries: self.__SW_MANIFEST,
  skipWaiting: true,
  clientsClaim: true,
  navigationPreload: true,
  runtimeCaching: defaultCache,
  fallbacks: {
    entries: [{ url: '/offline', matcher: ({ request }) => request.destination === 'document' }],
  },
});
serwist.addEventListeners();
```

Any navigation that fails with no network falls back to `/offline` — a static page rather than the browser’s own offline error — and `runtimeCaching: defaultCache` gives every other asset (JS/CSS bundles, images) Serwist’s default stale-while-revalidate strategy, so a returning visitor’s shell loads instantly from cache even before the network request resolves.

Part 11 moves into the authenticated app itself — the sidebar shell and the full set of feature pages built on everything from Parts 10.2–10.4.

Part 11 — Frontend Application Pages

11.1 The Authenticated Shell

`(app)/layout.tsx` wraps every authenticated route in `AuthGuard`, the sidebar, and the header:

```
// app/(app)/layout.tsx
export default function AppLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <div className={`$${geistSans.variable} $${geistMono.variable} antialiased min-h-screen bg-background`}>
      <AuthGuard>
        <MobileSidebarProvider>
          <Sidebar />
          <div className="lg:pl-64">
            <AppHeader />
            <main className="py-6"><div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">{children}</div></main>
          </div>
        </MobileSidebarProvider>
      </AuthGuard>
    </div>
  );
}
```

`AuthGuard` is the client-side route protection layer — it redirects to `/login` the moment `useAuth()` resolves with no user, and renders a spinner rather than a flash of protected content while that resolution is still in flight:

```
// components/AuthGuard.tsx
export function AuthGuard({ children }: PropsWithChildren) {
  const { user, isLoading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!isLoading && !user) router.replace('/login');
  }, [isLoading, user, router]);

  if (isLoading || !user) {
    return <div className="flex min-h-screen items-center justify-center"><Spinner /></div>;
  }
  return <>{children}</>;
}
```

`RoleGuard` layers finer-grained UI-level restriction *inside* a page that's already past `AuthGuard` — for pages or sections only ADMIN/HR should see, mirroring the backend's `@Roles()` decorator (Part 4 §4.5) but as a rendering guard rather than a request guard (the actual authorization is always still enforced server-side; this only controls what's shown):

```
// components/RoleGuard.tsx
export function RoleGuard({ allow, children }: PropsWithChildren<{ allow: Role[] }>) {
  const { user } = useAuth();
  if (!user || !allow.includes(user.role)) {
    return <Card><CardContent className="p-6"><p className="text-muted-foreground">You don't have permission to view this page.</p></CardContent></Card>;
  }
  return <>{children}</>;
}
```

11.2 Page Inventory

Route	Purpose
<code>/dashboard</code>	Headline metrics (headcount, latest run totals, pending approvals)
<code>/employees</code>	List + create/edit employee, onboarding checklist, contracts, salary structures
<code>/payroll</code>	Run payroll, list past runs
<code>/payroll/[id]</code>	Run detail: totals, per-employee entries, payslip + bank export downloads
<code>/loans</code>	Request/approve/reject/pay-off loans, repayment schedules
<code>/leave</code>	Leave types, balances, request/approve leave
<code>/analytics</code>	Department cost breakdown, leave usage trends
<code>/compliance</code>	Generate P9/P10/NSSF/NHIF (KE), PAYE/pension/NHF (NG), EMP201/IRP5 (ZA)
<code>/billing</code>	Subscription, plan, invoices, pay-invoice
<code>/settings</code>	Company details, branding (logo/color), salary components
<code>/employee-portal</code>	Self-service: my payslips, my leave, my loans, my documents

Every list/detail page follows the same three-layer pattern: a `lib/*-api.ts` typed client (Part 10 §10.2), a React Query `useQuery` / `useMutation` pair, and a shadcn/ui component tree with explicit loading/error/empty states — never a bare fetch-in- `useEffect` .

11.3 Template: Payroll Run List → Detail

The `/payroll` list page fetches runs for the current company and lets ADMIN/HR trigger a new run:

```
// app/(app)/payroll/page.tsx
const runsQuery = useQuery({ queryKey: ['payrollRuns', companyId], queryFn: () => listPayrollRuns(companyId!), enabled: !!companyId });

const runPayrollMutation = useMutation({
  mutationFn: runPayroll,
  onSuccess: (run) => {
    toast.success(`Payroll run created for ${run.period}`);
    queryClient.invalidateQueries({ queryKey: ['payrollRuns'] });
    router.push(`/payroll/${run.id}`); // navigates straight to the detail route rather than an inline expand
  },
  onError: (error) => toast.error('Failed to run payroll', { description: error instanceof ApiError ? error.message : undefined }),
});
```

`/payroll/[id]` is a real route (not a client-side modal/expand) — this matters because a payroll run detail is something people bookmark, link in Slack, or revisit days later, and a real URL survives all three:

```
// app/(app)/payroll/[id]/page.tsx
export default function PayrollRunDetailPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = use(params); // React 19's `use()` unwraps the Next.js 15 async params promise directly in the
    component body

  const runQuery = useQuery({ queryKey: ['payrollRun', id], queryFn: () => getPayrollRun(id) });

  const handleDownloadPayslip = async (entryId: string) => {
    try { await downloadPayslip(entryId); } catch (err) { toast.error(errorMessage(err, 'Failed to download payslip')); }
  };

  if (runQuery.isLoading) return <PageSkeleton cards={3} rows={5} />;
  if (runQuery.error) return <ErrorState message={errorMessage(runQuery.error, 'Failed to load payroll run')} />;

  const run = runQuery.data!;
  return (
    <>
      <SummaryCards totals={run.totals} status={run.status} />
      <Button onClick={() => downloadBankExport(run.id, run.period)}><Download />Bank Export</Button>
      <Table>
        {run.entries.map((entry) => (
          <TableRow key={entry.id}>
            <TableCell>{entry.employee.firstName} {entry.employee.lastName}</TableCell>
            <TableCell>{formatCurrency(entry.netPay, entry.currency)}</TableCell>
            <TableCell><Button size="sm" onClick={() => handleDownloadPayslip(entry.id)}><FileDown />Payslip</Button>
          </TableCell>
        </TableRow>
        ))}
      </Table>
    </>
  );
}
```

PageSkeleton is a shared shimmer-block component used across every list/detail page for the loading state — one component, consistent loading UX everywhere, instead of each page inventing its own spinner.

11.4 Template: Create → Approve Mutation Flow (Loans)

The Loans page demonstrates the full read-modify-write cycle a lot of the HR pages share: a dialog-based create form, a role-gated approval action, and cache invalidation tying them together.

```
// app/(app)/loans/page.tsx
function NewLoanDialog({ onCreate }: { onCreate: () => void }) {
  const [open, setOpen] = useState(false);
  const [employeeId, setEmployeeId] = useState('');
  const [principal, setPrincipal] = useState('');
  // ...installments, startPeriod, reason

  const createMutation = useMutation({
    mutationFn: () => createLoan({ employeeId, principal: Number(principal), installments: Number(installments),
      startPeriod, reason: reason || undefined }),
    onSuccess: () => {
      toast.success('Loan request created – pending approval');
      setOpen(false);
      setEmployeeId(''); setPrincipal(''); // reset form state
      onCreate(); // triggers the parent's queryClient.invalidateQueries
    },
    onError: (error) => toast.error('Could not create the loan request', { description: errorMessage(error, 'Please
      check the details and try again') }),
  });

  return (
    <Dialog open={open} onOpenChange={setOpen}>
      <DialogTrigger asChild><Button><Plus />New Loan Request</Button></DialogTrigger>
      <DialogContent>{/* form fields, submit calls createMutation.mutate() */</DialogContent>
    </Dialog>
  );
}
```


Approval/rejection is gated to ADMIN/HR at the UI layer with `RoleGuard` (§11.1), and the actual mutation (`decideLoan`) is a thin wrapper calling `POST /loans/:id/decide` — the same endpoint Part 9 §9.2’s `LoansService.decide` implements, with the identical `LOAN_MANAGE` permission enforced server-side regardless of what the UI shows or hides.

11.5 Analytics — Real Data, Not Mock Fixtures

`PayrollAnalytics` derives every chart from live API data rather than a static fixture file: department cost breakdown groups active employees by `Employee.department` and sums each one’s current `basicSalary` (from their active `SalaryStructure`, resolved the same way `EmployeesService.getActiveSalaryStructure` does — Part 5 §5.1); leave usage aggregates approved `LeaveRequest.daysRequested` by month. Both are plain client-side reductions over data already fetched via the existing `employees-api` / `leave-api` clients — no separate analytics-specific backend endpoint was needed for either chart.

11.6 Employee Self-Service Portal

`/employee-portal` reuses the same `lib/*-api.ts` clients as the admin pages, but every underlying endpoint enforces “own records only” server-side for an `EMPLOYEE`-role caller — the pattern established in Part 5 (`PayslipsService.generate`), Part 9 (`DocumentsService.assertAccess`), and mirrored in `PayrollService.findMine` / `LeaveService.findMine` / `LoansService.findMine`, all of which key off `AuthenticatedRequestUser.employeeId` embedded in the JWT (Part 4 §4.4) rather than trusting a client-supplied employee ID. The frontend page itself is a tabbed view (Overview / Payslips / Leave / Loans / Documents) built from those same `findMine`-style read endpoints, so there is no separate “employee” backend module — just role-scoped views over the same data admin/HR see for everyone.

11.7 Payslip Branding on the Frontend

`/settings` exposes logo upload and primary/secondary color pickers backed by `BrandingConfig` (Part 2 §2.3 Cluster 1). Saving triggers `BrandingContext.refreshBranding()` (Part 10 §10.3), which immediately updates the app shell’s own rendered color/logo — the same `BrandingConfigDto` shape is what `PayslipsService` / `ComplianceReportsService` (Parts 5–6) read server-side to brand generated PDFs, so what an admin sees previewed in Settings is what actually appears on a real payslip, not a decorative-only setting.

With every application page built on the same foundation, Part 12 closes out the guide with testing strategy, Docker/CI deployment, and the full environment-variable reference needed to actually run all of this.

Part 12 — Testing, DevOps & Environment Reference

12.1 Testing Strategy — Three Layers

Layer 1: pure-function unit tests (`packages/payroll-rules`). No mocking of any kind — every test is a direct input → output assertion against `runPayrollCalculation` or an individual country's helper (Part 3 §3.10). This is the cheapest, fastest, most exhaustive layer, and deliberately where the bulk of statutory-correctness testing lives, because it's the layer with zero infrastructure dependency.

Layer 2: NestJS service unit tests (`*.service.spec.ts`). `PrismaService` and every collaborator are hand-mocked — no test database, no real Postgres. A small typed helper keeps mocks ergonomic under strict TypeScript:

```
// billing/billing.service.spec.ts
// jest.fn() with no type args resolves to Mock<UnknownFunction>, whose return
// type is `unknown` rather than `Promise<unknown>` — that makes the
// conditional type behind `.mockResolvedValue()` collapse to `never`. Pin the
// fn's shape to a promise-returning signature up front so mocks stay
// reassignable across test cases.
const asyncMock = (value?: unknown) => jest.fn<(...args: any[]) => Promise<any>>().mockResolvedValue(value);

describe('BillingService', () => {
  let prisma: any;
  beforeEach(async () => {
    prisma = {
      plan: { findUnique: asyncMock(plan), findMany: asyncMock([plan]) },
      tenant: { findUniqueOrThrow: asyncMock(tenant) },
      subscription: { upsert: asyncMock(subscription), findUnique: asyncMock(subscription) },
      // ...
    };
    // BillingService constructed directly with these mocks via Nest's TestingModule
  });

  it('rejects subscribing to a plan priced for a different country', async () => {
    prisma.plan.findUnique.mockResolvedValueOnce({ ...plan, countryCode: 'NG' });
    prisma.tenant.findUniqueOrThrow.mockResolvedValueOnce({ ...tenant, countryCode: 'KE' });
    await expect(service.subscribe('tenant-1', { planCode: 'STARTER' })).rejects.toThrow(BadRequestException);
  });
});
```

This layer is where business-rule branches get tested — currency mismatches, idempotency-key reuse, permission-derivation edge cases, the tenant-scoping extension's `scopeQueryArgs` pure function (Part 4 §4.6) — without paying for a database round-trip per test.

Layer 3: end-to-end tests (`apps/api/test/*.e2e-spec.ts`). A real Postgres instance (via `docker compose` or a CI service container), the full NestJS app bootstrapped with `Test.createTestingModule().compile()`, and real HTTP requests via `supertest` — this is the only layer that exercises the actual `TenantContextInterceptor` + Prisma extension chain end to end, confirming tenant isolation holds under a real request, not just in the unit-tested `scopeQueryArgs` function.

```
// test/employees.e2e-spec.ts (shape)
describe('Employees (e2e)', () => {
  let app: INestApplication;
  beforeEach(async () => {
    const moduleRef = await Test.createTestingModule({ imports: [AppModule] }).compile();
    app = moduleRef.createNestApplication();
    await app.init();
  });

  it('cannot read an employee belonging to another tenant', async () => {
    const { accessToken } = await signupTenantA();
    const otherEmployeeId = await createEmployeeAsTenantB();
    await request(app.getHttpServer())
      .get(`/employees/${otherEmployeeId}`)
      .set('Authorization', `Bearer ${accessToken}`)
      .expect(404); // NotFoundException, not 403 — existence isn't leaked across tenants either
  });
});
```

Returning `404` rather than `403` for a cross-tenant lookup is itself a deliberate choice worth calling out: a `403` would confirm the record exists but is forbidden, leaking existence information across the tenant boundary; `404` reveals nothing.

12.2 CI Pipeline

The GitHub Actions workflow has four jobs with a deliberate dependency chain:

```
# .github/workflows/ci.yml
jobs:
  lint-build-test:      # pnpm install, prisma generate, turbo lint/build/test – no real DB
    runs-on: ubuntu-latest
    steps:
      - run: pnpm install --frozen-lockfile
      - run: pnpm --filter api exec prisma generate
      - run: pnpm turbo run lint
      - run: pnpm turbo run build
      - run: pnpm turbo run test

  e2e:                  # real Postgres service container, runs migrations, hits real HTTP endpoints
    needs: lint-build-test
    services:
      postgres:
        image: postgres:16-alpine
        options: --health-cmd "pg_isready -U payrollfiti" --health-interval 5s --health-retries 10
    steps:
      - run: pnpm --filter api exec prisma migrate deploy
      - run: pnpm --filter api test:e2e

  docker-build:         # PR-only: proves both images still build, doesn't push anywhere
    needs: lint-build-test
    if: github.event_name == 'pull_request'
    steps:
      - uses: docker/build-push-action@v6
        with: { file: apps/api/Dockerfile, push: false }
      - uses: docker/build-push-action@v6
        with: { file: apps/web/Dockerfile, push: false }

  docker-push:          # main-only: builds AND publishes to GHCR, tagged :latest and :<sha>
    needs: [lint-build-test, e2e]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    permissions: { contents: read, packages: write }
    steps:
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${github.actor}, password: ${secrets.GITHUB_TOKEN} }
      - uses: docker/build-push-action@v6
        with: { file: apps/api/Dockerfile, push: true, tags: "ghcr.io/${github.repository}-api:latest,...-api:${github.sha}" }
```

`docker-build` and `docker-push` are deliberately split rather than one job with a conditional `push` flag — a PR branch only needs proof the image *builds*; only a merge to `main`, after both `lint-build-test` and `e2e` have already passed, earns a real publish. The built-in `GITHUB_TOKEN` is sufficient for GHCR — no separate registry credentials need to be provisioned as repo secrets. This publishes images; it does not deploy them anywhere, since no hosting target is wired into this repository.

12.3 Dockerfiles — Turborepo Prune + Multi-Stage Build

Both `apps/api/Dockerfile` and `apps/web/Dockerfile` follow the same three-stage shape, using `turbo prune` to strip the monorepo down to just what one app actually needs before installing — so the API's image doesn't carry Next.js's entire dependency tree, and vice versa:

```

FROM node:20-alpine AS base
RUN corepack enable

FROM base AS pruner
WORKDIR /app
RUN apk add --no-cache libc6-compat
COPY . .
RUN pnpm dlx turbo prune api --docker # strips the monorepo to just `api` + its actual dependencies

FROM base AS installer
WORKDIR /app
RUN apk add --no-cache libc6-compat openssl
COPY --from=pruner /app/out/json/ .
RUN pnpm install --frozen-lockfile # dependency-only layer, cacheable independent of source changes
COPY --from=pruner /app/out/full/ .
ENV DATABASE_URL="postgresql://placeholder:placeholder@localhost:5432/placeholder"
RUN pnpm --filter api exec prisma generate # placeholder URL is enough – generate never connects to a real DB
RUN pnpm turbo run build --filter=api...

FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN apk add --no-cache openssl
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nestjs
COPY --from=installer /app .
USER nestjs # never run the production process as root
EXPOSE 3000
CMD ["node", "apps/api/dist/main.js"]

```

`turbo prune`’s two-phase `out/json` (manifests only) → `out/full` (actual source) copy is what lets Docker’s layer cache short-circuit the `pnpm install` layer whenever only source code changed and no dependency changed — a meaningfully faster rebuild loop than copying everything up front.

12.4 Environment Variable Reference

Every variable below is read through `configuration.ts`’s typed `AppConfig` — nothing in the codebase reads `process.env` directly outside that one file (Part 4 §4.1’s `env.validation.ts` validates the subset that’s security-critical).

Variable	Default (dev)	Required in prod?	Purpose
APP_NAME	PayrollFiti	No	Fallback branding when a tenant has no <code>BrandingConfig</code>
NODE_ENV	development	—	Gates the production-only checks in <code>env.validation.ts</code> and <code>EncryptionService</code>
PORT	3000	No	API listen port
COUNTRY_DEFAULT	KE	No	Fallback country code where one isn't otherwise resolvable
CORS_ORIGIN	http://localhost:3001	Yes	Allowed frontend origin; also used to build Paystack's checkout <code>callback_url</code>
DATABASE_URL	—	Yes	Postgres connection string (Prisma)
REDIS_URL	redis://localhost:6379	Yes	BullMQ queues + cache-manager
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	insecure dev defaults	Yes	Signing secrets — boot refuses to start in production with the checked-in default or under 16 chars (Part 4 §4.1)
JWT_ACCESS_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN	15m / 7d	No	Token lifetimes
THROTTLE_TTL / THROTTLE_LIMIT	60000 / 100	No	Rate limit window (ms) and request cap per window
PAYSLIP_STORAGE_DIR	./storage/payslips	No	Local disk path for generated payslip PDFs
DOCUMENT_STORAGE_DIR	./storage/documents	No	Local disk path for uploaded employee documents
ENCRYPTION_KEY	insecure dev fallback	Yes	64-hex-char AES-256-GCM key for PII field encryption (Part 4 §4.7) — generate via <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>
PAYSTACK_SECRET_KEY	unset (stubbed)	For real payments	Paystack API secret key
MPESA_CONSUMER_KEY / MPESA_CONSUMER_SECRET / MPESA_SHORTCODE / MPESA_PASSKEY	unset (stubbed)	For real M-Pesa	Daraja API credentials
MPESA_ENV	sandbox	No	<code>sandbox</code> or <code>production</code> — selects the Daraja base URL
MPESA_CALLBACK_URL / MPESA_CALLBACK_TOKEN	unset	For real M-Pesa	STK push callback URL + shared-secret token (Part 7 §7.4)
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS / SMTP_FROM	unset / 587 / ...	For real email	Nodemailer transport config
AFRICAS_TALKING_API_KEY / _USERNAME / _SENDER_ID	unset (no-op provider)	For real SMS	SMS gateway credentials
SENTRY_DSN	unset (disabled)	No	Error tracking — inert until set
NEXT_PUBLIC_API_URL	http://localhost:3000	Yes	Frontend's API base URL (build-time + runtime)
NEXT_PUBLIC_APP_NAME	PayrollFiti	No	Frontend fallback branding, mirrors <code>APP_NAME</code>

Every third-party integration — Paystack, M-Pesa, SMTP, Africa's Talking, Sentry — follows the same config-gated-stub pattern: unset in development, the app runs and the feature degrades to a logged no-op or deterministic stub rather than crashing; set in production, the exact same code path makes real calls. This is what makes the entire system runnable end-to-end, feature-complete in terms of code paths exercised, with zero real credentials.

12.5 Local Development Quickstart

```
git clone <repo> && cd payrollfiti
pnpm install
cp .env.example .env                                # fill in JWT secrets + ENCRYPTION_KEY for anything beyond pure local
dev
docker compose up -d db redis                       # infra only – run the apps themselves outside Docker for fast HMR
pnpm --filter api exec prisma migrate deploy
pnpm --filter api exec prisma db seed               # optional: sample tenant/employees/payroll history
pnpm dev                                             # turbo run dev – API on :3000, web on :3001
```

Parts 1–3 established the deterministic, versioned payroll core; Parts 4–9 built the multi-tenant, RBAC-guarded API around it, module by module, each new feature reusing the primitives the ones before it established; Parts 10–11 built a frontend where every page is a thin, typed, React-Query-backed view over that same API; Part 12 ties it all together into a pipeline that lints, tests at three layers, and ships. Parts 13–14 close two gaps that only became visible once the system had real, multi-role users: how an employee actually gets and recovers access, and how the app behaves as an installed, notification-capable PWA rather than just a responsive website. Following all fourteen parts in order reconstructs PayrollFiti from an empty repository to the system described in Part 1 — the same order it was actually built in.

Part 13 — Employee Onboarding & Authentication

Part 4 §4.4 covered JWT issuance and refresh, but only for the `signup()` path — one `ADMIN` creating a brand-new tenant. It never answered three questions that come up the moment a second, non-admin person needs to use the system: how does an employee get an account at all, where do they and the employer actually log in, and what happens when either of them forgets a password. This part closes all three, in the order a real rollout hits them.

13.1 Who Logs In Where

There is no subdomain-per-tenant scheme and no separate employee login page. `User.email` is globally unique — not scoped per tenant — so one email address can only ever back a single login, whatever its role. Every user, `ADMIN`, `HR`, or `EMPLOYEE`, authenticates at the same `/login` page; the API doesn't even know which "kind" of login is being attempted until the credentials resolve to a `User` row with a `role` on it.

What differs is where login sends you afterward. `/dashboard` calls `GET /employees` and `GET /payroll-runs`, both gated `@Roles(ADMIN, HR)` — an `EMPLOYEE` landing there would get two 403s instead of their portal, so the frontend branches on the role returned by the login response, not by asking twice:

```
// contexts/AuthContext.tsx
const login = useCallback(async (email: string, password: string) => {
  const data = await apiFetch<{ user: AuthenticatedUserDto } & AuthTokensDto>(
    '/auth/login',
    { method: 'POST', body: JSON.stringify({ email, password }), skipAuth: true },
  );
  // Not scoped by tenant/user identity – without this, switching accounts
  // in the same tab could keep showing the PREVIOUS session's cached data.
  queryClient.clear();
  tokenStorage.setTokens(data.accessToken, data.refreshToken);
  setUser(data.user);
  return data.user; // returned, not just set into state – see login/page.tsx below
}, [queryClient]);
```

```
// app/(auth)/login/page.tsx
const user = await login(email, password);
router.push(user.role === Role.EMPLOYEE ? '/employee-portal' : '/dashboard');
```

`login()` returns the freshly-fetched user rather than leaving the caller to read it back off the `useAuth()` hook — reading `user` from the hook immediately after `await login()` would still see the *previous* render's value, since `setUser()` doesn't re-render the calling closure synchronously.

The same role split governs what's even visible before a click. `Sidebar.tsx` tags every nav entry with an `allow: Role[]` and filters at render time; `RoleRedirectGuard` backstops it for direct URL access — six pages (Dashboard, Employees, Payroll, Leave Management, Analytics, Compliance) are wrapped so an `EMPLOYEE` navigating there directly, not just clicking a hidden link, still lands somewhere real instead of an error state built around data their role can't fetch:

```
// components/RoleRedirectGuard.tsx
export function RoleRedirectGuard({ allow, children }: PropsWithChildren<{ allow: Role[] }>) {
  const { user, isLoading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!isLoading && user && !allow.includes(user.role)) {
      router.replace('/employee-portal');
    }
  }, [isLoading, user, allow, router]);

  if (isLoading || !user || !allow.includes(user.role)) return null;
  return <>{children}</>;
}
```

This is deliberately a *redirect*, not the inline “you don’t have permission” card `RoleGuard` renders elsewhere (Settings, for instance) — those are pages a user might legitimately attempt and should be told why they can’t; Dashboard/Employees/Payroll/etc. are pages an `EMPLOYEE` should never see the shape of at all.

13.2 Employee Invites — From HR Action to Portal Access

`ADMIN / HR` create `Employee` records directly (Part 5 §5.1) for payroll purposes, but an `Employee` row has no login by itself — `Employee.user` is an optional 1:1. Portal access is a deliberate, separate action: `POST /employees/:id/invite`.

```
// prisma/schema.prisma
// A pending employee-portal signup. Deleted the moment it's redeemed — there
// is deliberately no "used" flag to check, since a redeemed invite has
// nothing left to represent once the real User row exists. tokenHash mirrors
// ApiKey's pattern: a one-way SHA-256 digest, never the raw token itself.
model EmployeeInvite {
  id          String    @id @default(uuid())
  employeeId  String    @unique
  employee    Employee  @relation(fields: [employeeId], references: [id], onDelete: Cascade)
  email       String
  tokenHash   String    @unique
  expiresAt   DateTime
  createdById String?
  createdAt   DateTime  @default(now())
}
```

```
// employees/employees.service.ts
async invite(tenantId: string, actorId: string, employeeId: string) {
  const employee = await this.prisma.employee.findUnique({ where: { id: employeeId }, include: { company: true, user: true } });
  if (!employee || employee.company.tenantId !== tenantId) throw new NotFoundException('Employee not found');
  if (employee.user) throw new BadRequestException('This employee already has portal access');

  // User.email is unique system-wide, not per-tenant. Without this check the
  // invite still sends, and only fails confusingly — a raw 500 — when the
  // person actually tries to redeem it, by which point it's too late to
  // give them a clear next step.
  const existingAccount = await this.prisma.user.findUnique({ where: { email: employee.email } });
  if (existingAccount) {
    throw new BadRequestException(
      `An account already exists for ${employee.email} — one email can only be linked to a single login. Use a different
      email for this employee, or update the existing account's role if this is the same person.`
    );
  }

  const rawToken = randomBytes(32).toString('hex');
  const tokenHash = createHash('sha256').update(rawToken).digest('hex');
  const expiresAt = new Date(Date.now() + EmployeesService.INVITE_EXPIRY_DAYS * 24 * 60 * 60 * 1000);

  await this.prisma.employeeInvite.upsert({
    where: { employeeId },
    create: { employeeId, email: employee.email, tokenHash, expiresAt, createdById: actorId },
    update: { email: employee.email, tokenHash, expiresAt, createdById: actorId },
  });

  const inviteUrl = `${this.configService.get('corsOrigin', { infer: true })}/accept-invite?token=${rawToken}`;
  await this.mailService.sendMail(employee.email, "You're invited to the employee portal",
    `<p>Hi ${employee.firstName},</p><p><a href="${inviteUrl}">Set up your account</a></p>` +
    `<p>This link expires in ${EmployeesService.INVITE_EXPIRY_DAYS} days.</p>`);
}
```

The raw token is only ever in memory and in the emailed link — the database keeps just its SHA-256 hash, same pattern as `ApiKey`. Redemption is symmetric: hash whatever token comes back, look it up, and — critically — never trust the frontend for the role or tenant being granted:


```
// auth/auth.service.ts
async acceptInvite(dto: AcceptInviteDto) {
  const tokenHash = createHash('sha256').update(dto.token).digest('hex');
  const invite = await this.prisma.employeeInvite.findUnique({
    where: { tokenHash }, include: { employee: { include: { company: true } } },
  });
  if (!invite || invite.expiresAt.getTime() < Date.now()) {
    throw new UnauthorizedException('This invite link is invalid or has expired');
  }

  const passwordHash = await bcrypt.hash(dto.password, SALT_ROUNDS);
  let user: User;
  try {
    user = await this.prisma.$transaction(async (tx) => {
      const created = await tx.user.create({
        data: { tenantId: invite.employee.company.tenantId, email: invite.email, passwordHash, role: Role.EMPLOYEE,
          employeeId: invite.employeeId },
      });
      await tx.employeeInvite.delete({ where: { id: invite.id } });
      return created;
    });
  } catch (error) {
    // The upfront check in invite() already covers this, but the email
    // could still be claimed by a brand-new signup in the window between
    // an invite being sent and redeemed – surface that clearly rather than
    // a raw constraint error.
    if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002') {
      throw new ConflictException(`An account already exists for ${invite.email}. Sign in with that email instead.`);
    }
    throw error;
  }

  return { user: this.toAuthenticatedUser(user), ...(await this.issueTokens(user)) };
}
```

`role: Role.EMPLOYEE` and `tenantId: invite.employee.company.tenantId` are both server-derived from the invite row, never from the request body — `AcceptInviteDto` only carries `token` and `password`. The frontend page is a thin form over this one call:

```
// app/(auth)/accept-invite/page.tsx
const token = useSearchParams().get('token');
// ...
await acceptInvite(token, password); // POST /auth/accept-invite, auto-logs in
router.push('/employee-portal');
```

End to end: HR clicks “Invite” on an employee record → API emails a link → employee opens it, sets a password → they’re logged into `/employee-portal` immediately, using the same `/login` page as everyone else from then on.

13.3 Forgot & Reset Password — One Flow for Every Role

Login is unified, so password recovery is too — there’s no role branch anywhere in this flow. It reuses the exact token pattern from §13.2: a one-way-hashed, short-lived, single-use token, emailed as a link.

```
// prisma/schema.prisma
// One row per user with an active reset request. userId is unique so a new
// request supersedes any previous unused one rather than accumulating rows.
model PasswordResetToken {
  id          String  @id @default(uuid())
  userId      String  @unique
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  tokenHash   String  @unique
  expiresAt   DateTime
  createdAt   DateTime @default(now())
}
```

```
// auth/auth.service.ts
private static readonly RESET_TOKEN_EXPIRY_HOURS = 1;

// Always resolves the same way whether or not the email is registered –
// a caller can't use this to probe which addresses have accounts.
async forgotPassword(dto: ForgotPasswordDto): Promise<void> {
  const user = await this.prisma.user.findUnique({ where: { email: dto.email } });
  if (!user) return;

  const rawToken = randomBytes(32).toString('hex');
  const tokenHash = createHash('sha256').update(rawToken).digest('hex');
  const expiresAt = new Date(Date.now() + AuthService.RESET_TOKEN_EXPIRY_HOURS * 60 * 60 * 1000);

  await this.prisma.passwordResetToken.upsert({
    where: { userId: user.id },
    create: { userId: user.id, tokenHash, expiresAt },
    update: { tokenHash, expiresAt },
  });

  const resetUrl = `${this.configService.get('corsOrigin', { infer: true })}/reset-password?token=${rawToken}`;
  await this.mailService.sendMail(user.email, 'Reset your password',
    `

<a href="${resetUrl}">Choose a new password</a></p><p>This link expires in 1 hour.</p>`);
}

// Reissuing tokens here overwrites refreshTokenHash, which as a side effect
// invalidates any refresh token from a session issued before the reset –
// exactly the behavior you want after a password compromise or recovery.
async resetPassword(dto: ResetPasswordDto) {
  const tokenHash = createHash('sha256').update(dto.token).digest('hex');
  const resetToken = await this.prisma.passwordResetToken.findUnique({ where: { tokenHash } });
  if (!resetToken || resetToken.expiresAt.getTime() < Date.now()) {
    throw new UnauthorizedException('This password reset link is invalid or has expired');
  }

  const passwordHash = await bcrypt.hash(dto.password, SALT_ROUNDS);
  const user = await this.prisma.$transaction(async (tx) => {
    const updated = await tx.user.update({ where: { id: resetToken.userId }, data: { passwordHash } });
    await tx.passwordResetToken.delete({ where: { id: resetToken.id } });
    return updated;
  });

  return { user: this.toAuthenticatedUser(user), ...(await this.issueTokens(user)) };
}


```

Both endpoints are `@Public()` — the token itself, not a session, is what’s being verified — and both return `200` on success rather than a body-shaped resource, except `resetPassword`, which returns the same `{ user, accessToken, refreshToken }` shape as `login` / `acceptInvite` so the frontend can log the user straight in:

```
// app/(auth)/forgot-password/page.tsx
await forgotPassword(email);
setSubmitted(true); // always shown, success or not – see the service method above
```

"If an account exists for {email}, we've sent a link to reset your password. It expires in 1 hour."

```
// app/(auth)/reset-password/page.tsx
const token = useSearchParams().get('token');
const user = await resetPassword(token, password);
router.push(user.role === Role.EMPLOYEE ? '/employee-portal' : '/dashboard');
```

A “Forgot password?” link sits next to the password field on `/login`, pointing at `/forgot-password`. That closes the loop: `/login` is the one door in for both roles, `/accept-invite` is how an `EMPLOYEE` gets a key to it in the first place, and `/forgot-password` → `/reset-password` is how either role gets back in when the key is lost — three pages, one `User` table, no role-specific code path anywhere in the chain.

Part 14 turns to a different gap: none of this — login, invites, password reset — was reachable as an installed app with real push notifications until now. It picks up where Part 10 §10.7’s baseline Serwist setup left off.

Part 14 — Progressive Web App: Install, Manifest & Push

Part 10 §10.7 wired up the baseline: a real service worker via `@serwist/next`, precaching, and an `/offline` fallback page. That made the app *installable* in the technical sense, but three things were still missing before it was actually usable as an app: nothing ever prompted a user to install it, nobody had checked whether the manifest/icons were complete rather than placeholder, and — the biggest gap — `PUSH_PROVIDER` was bound to a no-op stub, so every “send push notification” call in the system trivially succeeded without any device ever receiving anything. This part builds all three, in that order.

14.1 Manifest & Icons — Auditing What’s There

Before adding anything, the existing manifest and icon set turned out to already be complete — worth showing, since “audit first, build second” applies as much to documentation as to code:

```
// app/manifest.ts
export default function manifest(): MetadataRoute.Manifest {
  return {
    name: APP_NAME,
    short_name: APP_NAME,
    start_url: '/',
    display: 'standalone',
    background_color: '#ffffff',
    theme_color: '#e11d48',
    icons: [
      { src: '/icons/icon-192.png', sizes: '192x192', type: 'image/png', purpose: 'any' },
      { src: '/icons/icon-512.png', sizes: '512x512', type: 'image/png', purpose: 'any' },
      { src: '/icons/icon-maskable-512.png', sizes: '512x512', type: 'image/png', purpose: 'maskable' },
    ],
  };
}
```

That covers both required icon purposes — `any` (rendered as-is) and `maskable` (safe-zone padding so Android can crop it into a circle/squircle without clipping content) — plus a 180×180 `apple-touch-icon.png` wired through `layout.tsx`’s `metadata.icons.apple` for iOS home-screen icons, which the web manifest spec doesn’t cover on its own. Nothing to add here; the gap was entirely in the two sections below.

14.2 The Install Prompt

Chrome/Edge/Android fire a `beforeinstallprompt` event that a page can capture and defer, then trigger later from its own UI instead of relying on the browser’s native (and easy-to-miss) install icon. iOS Safari never fires that event at all — “Add to Home Screen” there is Share-sheet-only — so the same component branches into static instructions instead of pretending the two platforms work the same way:

```
// components/pwa/InstallPrompt.tsx
export function InstallPrompt() {
  const [deferredPrompt, setDeferredPrompt] = useState<BeforeInstallPromptEvent | null>(null);
  const [showIosInstructions, setShowIosInstructions] = useState(false);
  const [dismissed, setDismissed] = useState(true);

  useEffect(() => {
    if (isStandalone() || recentlyDismissed()) return;
    setDismissed(false);

    if (isIos()) { setShowIosInstructions(true); return; }

    const onBeforeInstallPrompt = (event: Event) => {
      event.preventDefault(); // suppress the browser's own prompt – trigger ours instead
      setDeferredPrompt(event as BeforeInstallPromptEvent);
    };
    const onInstalled = () => {
      setDeferredPrompt(null);
      setDismissed(true);
      window.localStorage.removeItem(DISMISS_KEY);
    };
    window.addEventListener('beforeinstallprompt', onBeforeInstallPrompt);
    window.addEventListener('appinstalled', onInstalled);
    return () => {
      window.removeEventListener('beforeinstallprompt', onBeforeInstallPrompt);
      window.removeEventListener('appinstalled', onInstalled);
    };
  }, []);

  const install = async () => {
    if (!deferredPrompt) return;
    await deferredPrompt.prompt();
    const { outcome } = await deferredPrompt.userChoice;
    setDeferredPrompt(null);
    if (outcome === 'accepted') window.localStorage.removeItem(DISMISS_KEY);
    else dismiss();
  };
  // ...renders a dismissible bottom banner; dismiss() stamps DISMISS_KEY with Date.now()
}
```

Two guards keep the banner from being annoying: `isStandalone()` (checks `matchMedia('(display-mode: standalone)')` and iOS's non-standard `navigator.standalone`) hides it entirely once already installed, and a dismissal is remembered in `localStorage` for 14 days rather than reappearing on every visit. Mounted once, globally, in the root layout:

```
// app/layout.tsx
<Toaster />
<InstallPrompt />
```

14.3 Web Push — Backend

This is the part that was previously entirely fake. `PushProvider` is the same extension-point pattern as `SmsProvider` / `AccountingProvider` — an interface plus a `NoopPushProvider` default — but until now nothing had ever implemented it for real:

```
// notifications/push-provider.interface.ts
export interface PushProvider {
  readonly name: string;
  send(userId: string, title: string, body: string): Promise<PushSendResult>;
}
```

Real delivery needs two things the interface alone doesn't provide: a place to store *which* browsers/devices a user has subscribed (a user with two tabs open on two phones has two subscriptions), and VAPID keys to sign the push messages per RFC 8292.

```
// prisma/schema.prisma
// One row per subscribed browser/device. endpoint is the push service's
// per-subscription URL, unique by construction; p256dh/auth are the
// subscription's public key and auth secret, both required to encrypt a
// push message per the Web Push protocol (RFC 8291).
model PushSubscription {
  id          String   @id @default(uuid())
  tenantId    String
  tenant      Tenant   @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  endpoint    String   @unique
  p256dh      String
  auth        String
  userAgent   String?
  createdAt   DateTime @default(now())
}
```

```
// notifications/providers/web-push.provider.ts
@Injectable()
export class WebPushProvider implements PushProvider {
  readonly name = 'web-push';

  async send(userId: string, title: string, body: string): Promise<PushSendResult> {
    const config = this.configService.get('vapid', { infer: true });
    if (!config.publicKey || !config.privateKey) {
      return { success: false, error: 'Push provider not configured' };
    }

    // Unlike SmsProvider.send(to, ...), there's no destination address to
    // hand in — a user can have any number of subscribed devices, so this
    // provider owns its own recipient lookup rather than taking one.
    const subscriptions = await this.prisma.pushSubscription.findMany({ where: { userId } });
    if (subscriptions.length === 0) return { success: false, error: 'No push subscriptions for user' };

    webpush.setVapidDetails(config.subject, config.publicKey, config.privateKey);
    const payload = JSON.stringify({ title, body });

    const results = await Promise.all(subscriptions.map(async (subscription) => {
      try {
        await webpush.sendNotification(
          { endpoint: subscription.endpoint, keys: { p256dh: subscription.p256dh, auth: subscription.auth } },
          payload,
        );
        return true;
      } catch (error) {
        const statusCode = (error as { statusCode?: number }).statusCode;
        // The push service reporting a subscription as gone (browser
        // uninstalled, permission revoked) — prune it so it isn't retried
        // forever, rather than treating it as a transient failure.
        if (statusCode === 404 || statusCode === 410) {
          await this.prisma.pushSubscription.delete({ where: { id: subscription.id } }).catch(() => undefined);
        }
        return false;
      }
    }));

    return results.some(Boolean) ? { success: true } : { success: false, error: 'Delivery failed for all subscriptions' };
  }
}
```

Wired in with the exact same config-gated factory pattern as `SMS_PROVIDER` (Part 8 §8.4) — inert (`NoopPushProvider`) until `VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` are set, real (`WebPushProvider`) once they are, with nothing else in the codebase needing to change either way:

```
// notifications/notifications.module.ts
{
  provide: PUSH_PROVIDER,
  useFactory: (configService: ConfigService<AppConfig, true>, webPush: WebPushProvider, noop: NoopPushProvider):
    PushProvider => {
    const config = configService.get('vapid', { infer: true });
    return config.publicKey && config.privateKey ? webPush : noop;
  },
  inject: [ConfigService, WebPushProvider, NoopPushProvider],
},
```

The frontend needs somewhere to register a subscription and a way to fetch the (non-secret, by design) VAPID public key it needs to call `PushManager.subscribe()` :

```
// notifications/push-subscriptions.controller.ts
@Controller('push-subscriptions')
export class PushSubscriptionsController {
  @Get('vapid-public-key')
  getVapidPublicKey(): { publicKey: string | null } {
    const { publicKey } = this.configService.get('vapid', { infer: true });
    return { publicKey: publicKey ?? null };
  }

  @Post()
  @HttpCode(HttpStatus.NO_CONTENT)
  subscribe(@CurrentUser() user: AuthenticatedRequestUser, @Body() dto: SubscribePushDto) {
    // Upsert by endpoint: a browser re-subscribing (e.g. after the push
    // service rotates it) should update the existing row, not duplicate it.
    return this.notificationsService.subscribeToPush(user.tenantId, user.id, dto.endpoint, dto.keys, dto.userAgent);
  }

  @Delete()
  @HttpCode(HttpStatus.NO_CONTENT)
  unsubscribe(@CurrentUser() user: AuthenticatedRequestUser, @Body() dto: UnsubscribePushDto) {
    return this.notificationsService.unsubscribeFromPush(user.id, dto.endpoint);
  }
}
```

14.4 Web Push — Frontend & the Service Worker

`usePushNotifications()` owns the browser-side half: checking support, reflecting whether the current device is already subscribed, and the subscribe/unsubscribe round-trip through the service worker's `PushManager` :

```
// lib/push/use-push-notifications.ts
export function usePushNotifications() {
  const [support, setSupport] = useState<PushSupportState>('checking');
  const [subscribed, setSubscribed] = useState(false);

  useEffect(() => {
    if (!('serviceWorker' in navigator) || !('PushManager' in window)) {
      setSupport('unsupported');
      return;
    }
    setSupport('ready');
    navigator.serviceWorker.ready
      .then((registration) => registration.pushManager.getSubscription())
      .then((subscription) => setSubscribed(subscription !== null));
  }, []);

  const subscribe = useCallback(async () => {
    const permission = await Notification.requestPermission();
    if (permission !== 'granted') throw new Error('Notification permission was not granted');

    const publicKey = await getVapidPublicKey();
    if (!publicKey) throw new Error('Push notifications are not configured on the server');

    const registration = await navigator.serviceWorker.ready;
    const subscription = await registration.pushManager.subscribe({
      userVisibleOnly: true,
      applicationServerKey: urlBase64ToUint8Array(publicKey), // PushManager needs a Uint8Array, not the base64url
                                                                    string the API returns
    });
    await subscribeToPush(subscription.toJSON() as PushSubscriptionJSON);
    setSubscribed(true);
  }, []);

  const unsubscribe = useCallback(async () => {
    const registration = await navigator.serviceWorker.ready;
    const subscription = await registration.pushManager.getSubscription();
    if (subscription) {
      await subscription.unsubscribe();
      await unsubscribeFromPush(subscription.endpoint);
    }
    setSubscribed(false);
  }, []);

  return { support, subscribed, subscribe, unsubscribe };
}
```

Exposed as a bell icon in `AppHeader` — visible to every authenticated user, admin/HR and employee alike, since they share the same layout — that flips between “enable” and “turn off” based on `subscribed`.

None of the above makes anything actually *appear* when a push arrives, though — `Serwist` only handles caching/offline; the push channel itself needed its own event listeners added straight to the same service worker file from Part 10 §10.7:

```
// app/sw.ts
serviceworker.addEventListeners();

self.addEventListener('push', (event) => {
  const payload = event.data?.json() ?? { title: 'PayrollFiti', body: 'You have a new notification' };
  event.waitUntil(
    self.registration.showNotification(payload.title, {
      body: payload.body,
      icon: '/icons/icon-192.png',
      badge: '/icons/icon-192.png',
    }),
  );
});

// Focuses an already-open app tab rather than always opening a new one.
self.addEventListener('notificationclick', (event) => {
  event.notification.close();
  event.waitUntil(
    self.clients.matchAll({ type: 'window', includeUncontrolled: true }).then((clients) => {
      const existing = clients.find((client) => 'focus' in client);
      return existing ? (existing as WindowClient).focus() : self.clients.openWindow('/');
    }),
  );
});
```

End to end, the same shape as every other integration in this system: `NOTIFICATION_DELIVER_JOB` (Part 8 §8.2) calls `pushProvider.send(userId, title, body)` exactly as it already did when the provider was a no-op — `WebPushProvider` looks up that user's subscriptions, `webpush.sendNotification()` hands the encrypted payload to the browser's push service, and `app/sw.ts`'s `push` listener is what turns that into something the user actually sees. Generate a keypair once with `npm run web-push generate-vapid-keys`, set `VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` / `VAPID_SUBJECT`, and every existing `dispatch(..., { channels: [NotificationChannel.PUSH] })` call site starts delivering for real — no call site anywhere else in the codebase needed to change.

That closes the frontend's account as an installable, notification-capable app. Combined with Part 13's authentication chain and Part 12's environment reference, the guide so far describes the system as it actually runs, end to end: an `ADMIN` signing up, inviting `HR` and `EMPLOYEE` users into a single shared login, every one of them able to install the app to their home screen and receive a real push notification when a payroll run completes or a leave request needs a decision. Part 15 picks up a different thread: the systems *around* the app — API keys, webhooks, and the audit trail that watches both.

Part 15 — Developer Self-Service: API Keys, Webhooks & the Audit Log

Everything so far has been for the people using the app directly. This part is for the systems around it: a script that needs read-only API access, an external tool that wants to know the moment a payroll run completes, and — cutting across both — the question an **ADMIN** eventually asks: *who did that, and when?* All three ship together because the third one caught a real bug in how the other two were being logged.

15.1 API Keys — Machine Credentials for the Public API

`GET /public-api/v1/*` (Part 9) needed a way to authenticate that isn't "log in as a human." An **ApiKey** is a long-lived, revocable credential scoped to a tenant, generated and shown exactly once:

```
// prisma/schema.prisma
model ApiKey {
  id          String      @id @default(uuid())
  tenantId    String
  tenant      Tenant      @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  name        String
  hashedKey   String      @unique
  keyPrefix   String
  lastUsedAt  DateTime?
  revokedAt   DateTime?
  createdById String?
  createdBy   User?       @relation(fields: [createdById], references: [id])
  createdAt   DateTime    @default(now())

  @@index([tenantId])
}
```

```
// api-keys/api-keys.service.ts
/**
 * Hashing choice: SHA-256, NOT bcrypt. API keys are high-entropy
 * machine-generated tokens (32 random bytes) rather than low-entropy
 * user-chosen passwords, so there is no dictionary/brute-force risk that
 * bcrypt's deliberate slowness defends against here – a fast one-way hash
 * is sufficient and avoids needless per-request CPU overhead on every
 * public-api call.
 */
async create(tenantId: string, actorId: string, dto: CreateApiKeyDto) {
  const rawKey = `pfk_${randomBytes(24).toString('hex')}`;
  const hashedKey = createHash('sha256').update(rawKey).digest('hex');
  const keyPrefix = rawKey.slice(0, 12);

  const apiKey = await this.prisma.apiKey.create({
    data: { tenantId, name: dto.name, hashedKey, keyPrefix, createdById: actorId },
  });

  // rawKey is returned ONLY in this response – it is never retrievable
  // again after this call (only its SHA-256 hash is persisted).
  return { apiKey, rawKey };
}
```

`list()` explicitly `select` s every column except `hashedKey` rather than relying on the frontend to not render a field it was handed — the row simply isn't fetched, so there's no raw-or-hashed key material to leak even if the response were logged somewhere unexpected. (Foreshadowing §15.3.) Validation is the same SHA-256 lookup in reverse, plus a `lastUsedAt` bump on every hit:

```
// common/guards/api-key.guard.ts
async canActivate(context: ExecutionContext): Promise<boolean> {
  const request = context.switchToHttp().getRequest();
  const key = request.headers['x-api-key'];
  if (!key || typeof key !== 'string') {
    throw new UnauthorizedException('Missing X-API-Key header');
  }

  const tenantContext = await this.apiKeysService.validate(key);
  if (!tenantContext) {
    throw new UnauthorizedException('Invalid or revoked API key');
  }

  // Shaped exactly like AuthenticatedRequestUser so the existing
  // @CurrentTenant()/@CurrentUser() decorators work unmodified on
  // public-api routes. `role` here is a placeholder only – it must never
  // be relied on to grant access. `isApiKeyAuth` lets RolesGuard/
  // PermissionsGuard fail closed if a role/permission check is ever added
  // to a public-api route, instead of this placeholder silently
  // satisfying it.
  request.user = { id: 'api-key', email: '', role: Role.ADMIN, tenantId: tenantContext.tenantId, employeeId: null };
  request.isApiKeyAuth = true;
  return true;
}
```

The frontend's `ApiKeysSettings` panel holds the freshly-created `rawKey` only in local component state, never in the React Query cache — a page refresh loses it permanently, same as the server does:

```
// components/settings/ApiKeysSettings.tsx
const [revealedKey, setRevealedKey] = useState<string | null>(null);

const createMutation = useMutation({
  mutationFn: createApiKey,
  onSuccess: ({ rawKey }) => {
    setRevealedKey(rawKey); // shown once, in a dismissible amber banner
    queryClient.invalidateQueries({ queryKey: ['api-keys'] });
  },
});
```

15.2 Webhooks — Outbound Event Delivery

Where an API key is a credential presented to the server, a webhook secret is one the server holds and uses to *sign* outgoing requests — closer to how Stripe or GitHub sign their webhook payloads than to a password. That distinction drives the storage choice directly: it's stored retrievable (not hashed), because every dispatch needs to compute an HMAC with it.

```
// prisma/schema.prisma
model WebhookEndpoint {
  id          String          @id @default(uuid())
  tenantId    String
  tenant      Tenant          @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  url         String
  secret      String
  events      String[]
  isActive    Boolean         @default(true)
  deliveries  WebhookDeliveryLog[]
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  @@index([tenantId])
}
```

```
// webhooks/webhooks.service.ts
async create(tenantId: string, dto: CreateWebhookDto) {
  await assertPublicWebhookUrl(dto.url);
  const secret = randomBytes(32).toString('hex');
  // This create() response is the ONLY place the full, unmasked secret is
  // ever returned again — list() below masks it.
  return this.prisma.webhookEndpoint.create({
    data: { tenantId, url: dto.url, events: dto.events, secret },
  });
}

async list(tenantId: string) {
  const endpoints = await this.prisma.webhookEndpoint.findMany({ where: { tenantId } });
  return endpoints.map((endpoint) => ({ ...endpoint, secret: `whsec_...${endpoint.secret.slice(-4)} ` }));
}
```

`assertPublicWebhookUrl` (Part 9’s SSRF guard) runs twice — once at create/update time, and again at dispatch time, right before the actual request:

```
// webhooks/webhooks.service.ts
private async deliverOne(endpoint: WebhookEndpoint, event: string, payload: Record<string, unknown>) {
  const body = JSON.stringify(payload);
  const signature = createHmac('sha256', endpoint.secret).update(body).digest('hex');

  try {
    // Re-check at dispatch time, not just at create/update — narrows the
    // window for DNS-rebinding (endpoint.url resolving to a private
    // address only after it passed the create-time check).
    await assertPublicWebhookUrl(endpoint.url);
    const response = await axios.post(endpoint.url, payload, {
      headers: { 'X-Webhook-Signature': signature, 'X-Webhook-Event': event },
      timeout: DELIVERY_TIMEOUT_MS,
    });
    // ...records a WebhookDeliveryLog row either way, success or failure
  } catch (err) { /* ... */ }
}
```

Delivery is deliberately best-effort with no retry — a `WebhookDeliveryLog` row records every attempt’s outcome (`statusCode` , `success` , `error`), visible via `GET /webhooks/:id/deliveries` , but a failed delivery isn’t requeued. That’s an explicit, documented gap rather than an oversight: a BullMQ-backed retry queue is the natural next step given Part 17’s fan-out pattern is right there to copy, but it’s scoped out of this pass.

15.3 The Audit Log — and a Real Leak It Caught

`AuditInterceptor` (introduced earlier, Part 4) generically logs every mutating request’s response body into `AuditLog.after` . That’s convenient — no per-endpoint wiring needed — but building the viewer for it in this pass surfaced exactly why “convenient” and “safe” aren’t the same thing: `POST /api-keys` returns `{ apiKey, rawKey }` , and `POST /webhooks` returns the endpoint including its full `secret` . The interceptor was logging both, verbatim, into a table that `ADMIN` s could now browse — silently defeating the “shown once” guarantee both features are built around.

```
// prisma/schema.prisma
model AuditLog {
  id          String    @id @default(uuid())
  tenantId    String
  tenant      Tenant     @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  actorId     String?
  actor       User?      @relation(fields: [actorId], references: [id])
  action      String
  entityType   String
  entityId     String
  before      Json?
  after       Json?
  ipAddress   String?
  createdAt   DateTime @default(now())

  @@index([tenantId, entityType, entityId])
  @@index([tenantId, createdAt])
}
```

The fix goes in one place — `AuditService.record()`, the single choke point every audit write passes through, whether from the generic interceptor or a service’s own explicit call — rather than patching each call site and hoping nobody adds a new one that forgets:

```
// audit/redact-sensitive-fields.ts
const SENSITIVE_KEY_PATTERN =
  /password|secret|rawkey|hashedkey|hashedpassword|tokenhash|access token|refresh token|^token$/i;

/**
 * Recursively strips values of sensitive-looking keys before they're
 * persisted to AuditLog.before/after. Matches by key name, not by value
 * shape, so it degrades safely (over-redacts) rather than missing a field
 * with an unexpected type.
 */
export function redactSensitiveFields<T>(value: T): T {
  if (Array.isArray(value)) {
    return value.map((item) => redactSensitiveFields(item)) as unknown as T;
  }
  if (value && typeof value === 'object' && !(value instanceof Date)) {
    const result: Record<string, unknown> = {};
    for (const [key, val] of Object.entries(value as Record<string, unknown>)) {
      result[key] = SENSITIVE_KEY_PATTERN.test(key) ? '[REDACTED]' : redactSensitiveFields(val);
    }
    return result as T;
  }
  return value;
}
```

```
// audit/audit.service.ts
async record(entry: RecordAuditEntryInput): Promise<void> {
  try {
    await this.prisma.auditLog.create({
      data: {
        tenantId: entry.tenantId,
        actorId: entry.actorId ?? null,
        action: entry.action,
        entityType: entry.entityType,
        entityId: entry.entityId,
        before: redactSensitiveFields(entry.before) ?? Prisma.JsonNull,
        after: redactSensitiveFields(entry.after) ?? Prisma.JsonNull,
        ipAddress: entry.ipAddress ?? null,
      },
    });
  } catch (error) {
    this.logger.error(`Failed to write audit log for ${entry.entityType}:${entry.entityId}`, error as Error);
  }
}
```

Confirmed live before shipping: created an API key and a webhook through the running app, checked the resulting `AuditLog` rows directly via `psql` — both `rawKey` and `secret` showed up in plaintext under the old code, and `[REDACTED]` after the fix. Two pre-fix rows that had already leaked into the persistent demo tenant's database were purged by hand.

The viewer itself is a filtered, paginated table — `GET /audit-logs`, `@Roles(ADMIN)` and gated behind a dedicated `Permission.AUDIT_LOG_READ` (not just the role check, so a future non-`ADMIN` role with elevated permissions still can't read it by accident):

```
// audit/audit.service.ts
async list(tenantId: string, query: ListAuditLogsQueryDto): Promise<PaginatedAuditLogs> {
  const page = query.page ?? 1;
  const limit = query.limit ?? 25;
  const where: Prisma.AuditLogWhereInput = {
    tenantId,
    ...(query.entityType ? { entityType: query.entityType } : {}),
    ...(query.actorId ? { actorId: query.actorId } : {}),
    ...(query.action ? { action: { contains: query.action, mode: 'insensitive' } } : {}),
  };

  const [items, total] = await Promise.all([
    this.prisma.auditLog.findMany({
      where, orderBy: { createdAt: 'desc' }, skip: (page - 1) * limit, take: limit,
      include: { actor: { select: { email: true } } },
    }),
    this.prisma.auditLog.count({ where }),
  ]);

  return { items, total, page, limit };
}
```

`/audit-log` on the frontend is a filterable table (entity type, action substring) with a details dialog that pretty-prints the (now-redacted) `before / after` JSON per row — reachable only from an `ADMIN`-only sidebar entry.

Three features, one thread: giving a tenant real self-service — its own API keys, its own outbound webhooks — only pays off if the tenant can also see what happened with them, and that visibility has to be built with the same care as the credentials it's reporting on. Part 16 turns to hardening the login itself.

Part 16 — Two-Factor Authentication & Session Management

Part 13 covered how someone gets a login and how they recover it if lost. This part hardens what happens once they have one: a second factor beyond the password, and — the bigger of the two — proper multi-device sessions in place of a single shared token slot. The second half of that story turned into a real security bug hunt, not just a feature build.

16.1 TOTP Setup — Three Calls, Not One

Turning on 2FA is three separate calls rather than one, on purpose: generate a secret, prove the authenticator app actually produces matching codes before trusting it, only then flip the flag.

```
// prisma/schema.prisma — added to User
twoFactorSecretEncrypted String?
twoFactorEnabled Boolean @default(false)
twoFactorBackupCodes String[] @default([])
```

`twoFactorSecretEncrypted` uses the same AES-256-GCM `EncryptionService` already built for KRA PINs and bank account numbers (Part 6) — unlike a token hash, this secret has to be recovered in plaintext on every login to compute the expected code, so it can't be one-way hashed the way everything else in this system's credential story is.

```
// auth/auth.service.ts
async setupTwoFactor(userId: string) {
  const user = await this.prisma.user.findUniqueOrThrow({ where: { id: userId } });
  const secret = await otplib.generateSecret();
  await this.prisma.user.update({
    where: { id: userId },
    data: { twoFactorSecretEncrypted: this.encryptionService.encrypt(secret)! },
  });

  const otpauthUrl = otplib.generateURI({
    strategy: 'totp',
    issuer: this.configService.get('appName', { infer: true }),
    label: user.email,
    secret,
  });
  const qrCodeDataURL = await QRCode.toDataURL(otpauthUrl);
  return { secret, otpauthUrl, qrCodeDataURL };
}

async enableTwoFactor(userId: string, dto: TwoFactorEnabledDto) {
  const user = await this.prisma.user.findUniqueOrThrow({ where: { id: userId } });
  if (!user.twoFactorSecretEncrypted) throw new BadRequestException('Call /auth/2fa/setup first');

  const secret = this.encryptionService.decrypt(user.twoFactorSecretEncrypted!);
  if (!(await verifyTotpSafely(secret, dto.code))) throw new UnauthorizedException('Invalid code');

  const backupCodes = Array.from({ length: BACKUP_CODE_COUNT }, generateBackupCode);
  await this.prisma.user.update({
    where: { id: userId },
    data: { twoFactorEnabled: true, twoFactorBackupCodes: backupCodes.map(hashBackupCode) },
  });
  return { backupCodes }; // shown once, same treatment as an ApiKey's raw value
}
```

Backup codes get the exact same one-way SHA-256 treatment as everything else machine-generated in this schema — `hashBackupCode` normalizes (`WDPA-WR2C` → `WDPAWR2C` , uppercased) before hashing, so formatting quirks in how a user retypes a code don't cause false rejections. `otplib.verify()` turned out to have a sharp edge worth calling out: it *throws* rather than returning `{ valid: false }` for a malformed token, which matters the moment a backup code (`WDPA-WR2C` , nine characters) gets tried where a 6-digit TOTP code was expected — that crashed the real login flow with a raw 500 before a wrapper caught it:

```
// auth/auth.service.ts
async function verifyTotpSafely(secret: string, token: string): Promise<boolean> {
  try {
    const result = await otplib.verify({ secret, token });
    return result.valid;
  } catch {
    return false;
  }
}
```

Every call site that checks a live TOTP code goes through this wrapper, never `otplib.verify` directly — confirmed with a regression test that feeds it a backup-code-shaped string and asserts it falls through to the backup-code check below instead of throwing.

16.2 The Login Flow’s Second Step

`login()` no longer always returns tokens. If the user has 2FA enabled, it returns a short-lived challenge instead — the same purpose-tagged-JWT pattern already used for the accounting OAuth `state` param (Part 7), reused here as a general “prove this request is legitimate” mechanism rather than something session-specific:

```
// auth/auth.service.ts
async login(dto: LoginDto, meta: SessionMeta = {}) {
  const user = await this.validateUser(dto.email, dto.password);
  if (user.twoFactorEnabled) {
    const challengeToken = await this.jwtService.signAsync(
      { purpose: '2fa-challenge', sub: user.id },
      { secret: this.configService.get('jwt', { infer: true }).accessSecret, expiresIn: '5m' },
    );
    return { twoFactorRequired: true as const, challengeToken };
  }
  const { session, ...tokens } = await this.createSession(user, meta);
  return { user: this.toAuthenticatedUser(user, session.id), ...tokens };
}
```

`verifyTwoFactor()` redeems that challenge into real tokens, checking the `purpose` claim explicitly — a `challengeToken` is just a JWT signed with the same access secret as everything else, so without that check a token minted for an entirely different purpose (say, a future feature reusing the same pattern) could be replayed here:

```
// auth/auth.service.ts
async verifyTwoFactor(dto: TwoFactorVerifyDto, meta: SessionMeta = {}) {
  const payload = await this.jwtService.verifyAsync(dto.challengeToken, {
    secret: this.configService.get('jwt', { infer: true }).accessSecret,
  }); // throws -> caught, turned into a clear "please sign in again"
  if (payload.purpose !== '2fa-challenge') {
    throw new UnauthorizedException('This login attempt has expired – please sign in again');
  }
  // ...verifyTwoFactorCode(user, dto.code), then createSession as normal
}
```

`AuthContext.login()` on the frontend types this as a union — `AuthenticatedUserDto | TwoFactorChallenge` — and the login page branches on which one came back, swapping the password form for a code-entry form without a page navigation:

```
// app/(auth)/login/page.tsx
const result = await login(email, password);
if ('twoFactorRequired' in result) {
  setChallengeToken(result.challengeToken); // renders the code-entry form
} else {
  redirectByRole(result.role);
}
```

16.3 Sessions — Replacing a Single Shared Token Slot

Before this pass, `User` had one `refreshTokenHash String?` field — meaning exactly one active login per user, system-wide. Signing in on a second device didn't add a session; it silently invalidated the first one's refresh token, since there was only ever one slot to store a hash in. That's the kind of bug that's invisible in single-device testing and immediately obvious the moment someone opens the app on their phone while still logged in on a laptop.

The fix is a proper `Session` row per device:

```
// prisma/schema.prisma
// One row per active login (a user with two tabs open on two phones has two
// rows) – replaces the old single User.refreshTokenHash field, which meant
// logging in on a second device silently logged the first one out.
// refreshTokenHash is SHA-256, same as every other one-way token in this
// schema – NOT bcrypt, which silently truncates its input to 72 bytes and
// would make every refresh token issued for a session hash identically
// (the part that varies falls past the truncation point), defeating
// rotation-reuse detection entirely. (See §16.4.)
model Session {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  refreshTokenHash String
  userAgent   String?
  ipAddress   String?
  createdAt   DateTime @default(now())
  lastUsedAt  DateTime @default(now())

  @@index([userId])
}
```

`sessionId` gets embedded in both the access and refresh JWT payloads, so any request — or a refresh call — can identify exactly which session it belongs to:

```
// auth/types.ts
export interface JwtAccessPayload {
  sub: string; email: string; role: Role; tenantId: string; employeeId: string | null;
  sessionId: string;
}
export interface JwtRefreshPayload {
  sub: string;
  sessionId: string;
}
```

Every login-shaped entry point — `login`, `signup`, `acceptInvite`, `resetPassword`, `verifyTwoFactor` — funnels through the same `createSession()`, and `refresh()` through `rotateSession()`, which reuses the existing session id and only updates its hash:

```
// auth/auth.service.ts
private async createSession(user: User, meta: SessionMeta) {
  const session = await this.prisma.session.create({
    data: { userId: user.id, refreshTokenHash: '', userAgent: meta.userAgent, ipAddress: meta.ipAddress },
  });
  return { session, ...(await this.signAndStoreTokens(user, session.id)) };
}
```

`GET /auth/sessions` lists every active device for the caller (flagging which one is “this device”); `DELETE /auth/sessions/:id` revokes one — both scoped to `userId` so one user's request can never touch another's session by guessing an id:

```
// auth/auth.service.ts
async listSessions(userId: string, currentSessionId: string) {
  const sessions = await this.prisma.session.findMany({ where: { userId }, orderBy: { lastUsedAt: 'desc' } });
  return sessions.map((session) => ({
    id: session.id, userAgent: session.userAgent, ipAddress: session.ipAddress,
    createdAt: session.createdAt, lastUsedAt: session.lastUsedAt,
    isCurrent: session.id === currentSessionId,
  }));
}
```


The frontend's `SessionsSettings` panel renders that as a device list with a lightweight user-agent parser (`Chrome on macOS` , `Safari on iOS`) and a “Sign out” confirmation per row; revoking the *current* device's own session calls `useAuth().logout()` locally too, rather than leaving a page up that's signed out on the server but not in the UI:

```
// components/settings/SessionsSettings.tsx
const revokeMutation = useMutation({
  mutationFn: () => revokeSession(session.id),
  onSuccess: async () => {
    if (session.isCurrent) {
      await logout(); // finish the job locally, not just server-side
      return;
    }
    toast.success('Signed out');
    queryClient.invalidateQueries({ queryKey: ['sessions'] });
  },
});
```

Two places outside the auth flow itself needed to know sessions now exist. Terminating an employee (Part 5) used to only flip `User.isActive` — already-issued tokens kept working until they naturally expired, since nothing about `isActive` invalidates a JWT that's already been signed:

```
// employees/employees.service.ts
await tx.user.updateMany({ where: { employeeId }, data: { isActive: false } });
// Sessions aren't cascade-deleted by isActive going false — a terminated
// employee's already-issued tokens must stop working immediately, not
// just future logins being blocked.
await tx.session.deleteMany({ where: { user: { employeeId } } });
```

And `resetPassword` (Part 13 §13.3) now deletes every session for that user as part of the same transaction — exactly the behavior you want after a password compromise or recovery, where a reset should log out every device, not just the one completing it.

16.4 The Bug This Refactor Found

Building `Session` meant, for the first time, actually exercising refresh-token rotation across multiple real devices in live testing rather than a single mocked path. That testing surfaced a genuine, previously-undetected security bug: **refresh-token reuse detection had never worked**.

The original code (inherited unchanged from the single- `refreshTokenHash` design) compared tokens with `bcrypt.compare(refreshToken, session.refreshTokenHash)`. `bcrypt` silently truncates its input to 72 bytes — a signed JWT refresh token is well past that, and the part that actually changes between issuances (`iat` , `exp` , the signature) falls entirely past the truncation point. Every refresh token ever issued for a given session hashed to the exact same `bcrypt` digest, because `bcrypt` was only ever looking at the unchanging `sub + sessionId` prefix. A stale, already-rotated-out refresh token was therefore accepted forever — the entire point of rotation-reuse detection (treat a replayed old token as a signal of a possible leak, and revoke the session) silently did nothing.

Finding it took escalating through the debugging approach established earlier in this build: a careful curl test with direct `psql` checks around it first ruled out a testing artifact; a standalone `ts-node` script calling `AuthService` directly, with explicit delays between calls, ruled out an `iat`-collision theory; temporary `console.log` instrumentation directly in `refresh()` / `signAndStoreTokens()` finally showed the smoking gun — the hash visibly changing in the database between calls, while `bcrypt.compare` against the *old* token kept returning `true` regardless.

The fix matches the pattern already established everywhere else in this schema for exactly this class of token — `ApiKey` , `EmployeeInvite` , `PasswordResetToken` , 2FA backup codes are all SHA-256, and there was never a reason `Session.refreshTokenHash` should have been the one exception:

```
// auth/auth.service.ts
function hashRefreshToken(token: string): string {
  return createHash('sha256').update(token).digest('hex');
}

async refresh(userId: string, sessionId: string, refreshToken: string) {
  const session = await this.prisma.session.findUnique({ where: { id: sessionId } });
  if (!session || session.userId !== userId) {
    throw new UnauthorizedException('Session expired, please log in again');
  }
  const matches = hashRefreshToken(refreshToken) === session.refreshTokenHash;
  if (!matches) {
    // Reuse of a rotated-out refresh token – revoke the session outright
    // rather than just rejecting, on the theory that a mismatched refresh
    // token is a signal the token may have leaked.
    await this.prisma.session.delete({ where: { id: session.id } });
    throw new UnauthorizedException('Invalid refresh token');
  }
  // ...
}
```

Live-verified against the running dev server, the exact scenario that used to silently pass: log in, refresh once (which rotates the session's stored hash), then replay the *original*, now-stale refresh token. Before the fix, that replay succeeded. After it, it correctly comes back **401 Invalid refresh token**, and the session row is gone:

```
$ curl -X POST /auth/refresh -H "Authorization: Bearer $STALE_TOKEN"
{"message":"Invalid refresh token","error":"Unauthorized","statusCode":401}

$ psql -c 'SELECT id FROM "Session" WHERE id = ...'
(0 rows)
```

The lesson generalizes past this one field: bcrypt's 72-byte truncation isn't a theoretical footgun, it's silently wrong for anything token-shaped and long, and it will not raise an error to tell you — it just returns **true** for inputs that shouldn't match. Every hashing choice in this codebase now follows one rule: bcrypt only for a real user-chosen password (where its deliberate slowness is the point), SHA-256 for everything machine-generated and high-entropy.

Part 17 turns to a different reliability question — not who can get in, but what happens after a payroll run, when a hundred payslip emails all need to go out at once.

Part 17 — Payslip Delivery at Scale

Part 8 built the notification queue’s general shape: write the in-app row, enqueue a BullMQ job for the out-of-band channels, let a processor pick it up off the request thread. Payslip emails followed that shape too — but with one structural difference that this part corrects: everything for a payroll run was one job, not one job per payslip.

17.1 One Job Per Run Was the Wrong Grain

`PayrollService.run()` (Part 5) enqueues payslip delivery right after a run completes, so rendering and emailing a payslip per employee doesn’t add request latency proportional to headcount:

```
// payroll/payroll.service.ts
// Enqueued (one BullMQ job per entry) rather than run inline – rendering +
// emailing a payslip per employee would otherwise add request latency
// proportional to headcount. PayslipEmailsProcessor picks these up off the
// request thread, concurrently; enqueueForRun itself never throws.
await this.payslipEmailService.enqueueForRun(tenantId, run.id);
```

The queue side of that used to add a single `deliver-for-run` job carrying just `{ tenantId, payrollRunId }`. The processor’s `process()` then looped over every entry in the run *inside that one job’s execution*:

```
// notifications/payslip-emails.processor.ts – the old shape
async process(job: Job<PayslipEmailsJobData>): Promise<void> {
  const { tenantId, payrollRunId } = job.data;
  await this.payslipEmailService.sendPayslipEmailsForRun(tenantId, payrollRunId); // looped internally
}
```

Two problems fall out of that grain. First, BullMQ’s concurrency and retry both operate at the *job* level — with the whole run as one job, a company with 200 employees processes all 200 payslips strictly sequentially inside a single worker slot, no matter how much concurrency the worker is configured for. Second, and worse: `sendPayslipEmail()` caught its own errors internally and just logged them, on the reasoning that a failure “must never fail the payroll-run HTTP response that triggered it.” That reasoning was correct for the *old* call site — `enqueueForRun`, awaited synchronously in the request path — but the loop inside the queued job inherited the same never-throw behavior for no reason, which meant BullMQ never saw a failure to retry. One employee’s mail provider hiccup was silently swallowed, with no automatic second attempt, for that employee only — invisible unless someone went looking at logs.

17.2 Splitting the Job, Splitting the Guarantee

The fix changes the unit of work from “a run” to “a payslip,” and — just as importantly — separates the two different never-throw guarantees that had been conflated into one function:

```
// notifications/payslip-emails.queue.ts
export const PAYSLIP_EMAILS_QUEUE = 'payslip-emails';
export const PAYSLIP_EMAILS_DELIVER_JOB = 'deliver-for-entry';

export interface PayslipEmailsJobData {
  tenantId: string;
  payrollEntryId: string;
}
```

```

// notifications/payslip-email.service.ts
/**
 * Enqueues one job per entry in the run – one job per entry (rather than
 * one job looping over the whole run) lets BullMQ process a run's emails
 * concurrently and retry a single failed entry without resending everyone
 * else's payslip.
 *
 * This method itself must never throw – a failure here must never fail
 * the payroll-run HTTP response that triggered it.
 */
async enqueueForRun(tenantId: string, payrollRunId: string): Promise<void> {
  try {
    const run = await this.prisma.payrollRun.findUnique({ where: { id: payrollRunId }, include: { entries: true } });
    if (!run) return;

    await this.payslipEmailsQueue.addBulk(
      run.entries.map((entry) => ({
        name: PAYSLIP_EMAILS_DELIVER_JOB,
        data: { tenantId, payrollEntryId: entry.id },
      })),
    );
  } catch (error) {
    this.logger.error(`Failed to enqueue payslip emails for run ${payrollRunId}`, error as Error);
  }
}

/**
 * Called only by PayslipEmailsProcessor, one entry per job – lets errors
 * propagate so BullMQ retries just this entry, instead of swallowing them
 * the way enqueueForRun above must.
 */
async sendPayslipEmail(tenantId: string, payrollEntryId: string): Promise<void> {
  const entry = await this.prisma.payrollEntry.findUnique({
    where: { id: payrollEntryId },
    include: { employee: { include: { user: true, company: true } }, payrollRun: true },
  });
  if (!entry || entry.employee.company.tenantId !== tenantId) return;

  const email = entry.employee.user?.email;
  if (!email) return; // no linked login – nowhere to send a payslip email

  const buffer = await this.payslipsService.generate(tenantId, payrollEntryId);
  await this.mailService.sendMail(
    email,
    `Your Payslip – ${entry.payrollRun.period}`,
    `<p>Hi ${entry.employee.firstName},</p><p>Your payslip for ${entry.payrollRun.period} is attached.</p>`,
    [{ filename: 'payslip.pdf', content: buffer }],
  );
}

```

`enqueueForRun` keeps its try/catch because it's still awaited directly in the request path. `sendPayslipEmail` drops its try/catch entirely, because it's no longer called from anywhere except the processor — letting the error surface is what makes it BullMQ's problem to retry, instead of a silently-swallowed one-off failure. The processor shrinks to a straight pass-through, now running with real concurrency:

```

// notifications/payslip-emails.processor.ts
@Processor(PAYSLIP_EMAILS_QUEUE, { concurrency: 5 })
export class PayslipEmailsProcessor extends WorkerHost {
  async process(job: Job<PayslipEmailsJobData>): Promise<void> {
    const { tenantId, payrollEntryId } = job.data;
    await this.payslipEmailService.sendPayslipEmail(tenantId, payrollEntryId);
  }
}

```

Letting an error through only matters if something is configured to catch it and retry — no queue in this codebase had ever set `attempts` above BullMQ's default of 1, which is a silent no-op retry policy easy to miss since nothing errors when it's absent. The payslip-emails queue gets one explicitly, since this is now the one queue whose processor is designed around per-job retry:

```
// notifications/notifications.module.ts
BullModule.registerQueue(
  { name: NOTIFICATIONS_QUEUE },
  {
    name: PAYSLIP_EMAILS_QUEUE,
    // sendPayslipEmail() now throws on failure instead of swallowing it
    // specifically so a transient failure – mail provider hiccup, PDF
    // render error – gets retried instead of silently dropping that one
    // employee's payslip.
    defaultJobOptions: {
      attempts: 3,
      backoff: { type: 'exponential', delay: 5000 },
    },
  },
),
```

Verified live against the running dev server rather than only through mocks: added two jobs directly onto the real **payslip-emails** queue, each for a different payroll entry in a different tenant, and confirmed via BullMQ's own job-state API that both completed independently through the real processor — **sendPayslipEmail** → the real Prisma lookup → a real mail send — rather than one shared job either succeeding or failing as a unit.

```
$ npx ts-node check-fanout.ts
7 completed {"tenantId":"demo-tenant","payrollEntryId":"ba30f50b-..."}
8 completed {"tenantId":"9c7847bd-...","payrollEntryId":"3514c870-..."}
```

That closes the six-feature arc this part and the two before it cover: developer self-service that's actually auditable (Part 15), a login that's actually hardened against both a stolen password and a stale token (Part 16), and now a payroll run whose email delivery scales with headcount instead of against it, with failures isolated and retried per employee instead of silently dropped for the whole batch. Part 18 closes out two remaining gaps from earlier in the build: the notification bell's frontend half, and the accounting-sync integration Part 7 only ever mentioned in passing.

Part 18 — Accounting Sync & the In-App Notification Center

Two features that shipped earlier in this build never made it into the guide: the frontend half of the notification system Part 8 built the backend for, and a full accounting-sync integration — QuickBooks, Xero, and Zoho Books — that Part 7 only ever mentioned in passing. Both are real, wired-up features already running in the app; this part is where they were missing from, not where they were built.

18.1 The Notification Bell — Reading What Part 8 Writes

Part 8 covered `NotificationsService.create / dispatch` writing rows and fanning out to email/SMS/push, but never the controller a logged-in user actually calls to read their own notifications:

```
// notifications/notifications.controller.ts
/** No @Roles() gating — any authenticated user manages only their own notifications. */
@Controller('notifications')
export class NotificationsController {
  @Get()
  findMine(
    @CurrentTenant() tenantId: string,
    @CurrentUser() user: AuthenticatedRequestUser,
    @Query('unreadOnly') unreadOnly?: string,
  ) {
    return this.notificationsService.listForUser(tenantId, user.id, unreadOnly === 'true');
  }

  @Patch('/:id/read')
  markRead(@CurrentTenant() tenantId: string, @CurrentUser() user: AuthenticatedRequestUser, @Param('id') id: string) {
    return this.notificationsService.markRead(tenantId, user.id, id);
  }

  @Post('read-all')
  markAllRead(@CurrentTenant() tenantId: string, @CurrentUser() user: AuthenticatedRequestUser) {
    return this.notificationsService.markAllRead(tenantId, user.id);
  }
}
```

Every method here is implicitly scoped to the caller — `listForUser / markRead / markAllRead` all filter by `userId` server-side, so there's no `@Roles()` guard needed: a regular `EMPLOYEE` and an `ADMIN` hit the exact same three endpoints, each only ever seeing their own rows.

The frontend used to just show an unread count; `NotificationBell` replaces that with a real popover, and deliberately fetches two different things at two different times:

```
// components/NotificationBell.tsx
const unreadQuery = useQuery({
  queryKey: ['notifications', 'unread'],
  queryFn: () => listNotifications(true),
});
const unreadCount = unreadQuery.data?.length ?? 0;

// Only fetched once the popover is actually opened — the header renders on
// every page, so the full (read + unread) list shouldn't be a standing
// request alongside the lightweight unread-count poll above.
const listQuery = useQuery({
  queryKey: ['notifications', 'all'],
  queryFn: () => listNotifications(false),
  enabled: open,
});
```

The unread count query is cheap and always live (it's what the little red badge on the bell icon reflects, everywhere in the app); the full list — potentially many rows, with relative timestamps — only fires when the popover actually opens. Clicking an unread row marks it read and invalidates both queries in one shot, rather than optimistically mutating local state and risking it drifting from the server:

```
// components/NotificationBell.tsx
const markReadMutation = useMutation({
  mutationFn: markNotificationRead,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['notifications'] });
  },
});
```

Mounted once in `AppHeader`, next to the theme toggle and the push-notification bell from Part 14 — every authenticated user gets the same bell regardless of role, same as the rest of the header.

18.2 Accounting Integrations — the OAuth Dance

Part 7 mentioned in one sentence that “webhook dispatch and accounting sync are best-effort” after a payment settles, without ever showing what actually powers that sync. `AccountingProvider` (Part 4’s extension-point pattern again — an interface plus a swappable implementation) originally shipped as a stub with no real credentials to test against; it’s since grown into a full three-platform OAuth2 integration.

```
// prisma/schema.prisma
model AccountingIntegration {
  id                String                @id @default(uuid())
  tenantId          String                @unique
  tenant            Tenant                @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  provider          AccountingProviderType
  externalId        String
  accessTokenEncrypted String
  refreshTokenEncrypted String
  expiresAt         DateTime
  connectedById     String?
  createdAt          DateTime              @default(now())
  updatedAt         DateTime              @updatedAt
}
```

One row per tenant (`tenantId @unique`) — connecting a second platform replaces the first rather than running two in parallel, which matches the UI (§18.4) offering one “Connect” action per platform, mutually exclusive in practice. Both OAuth tokens go through `EncryptionService` (Part 4 §4.7), the same field-level AES-256-GCM encryption used for KRA PINs and bank account numbers — these are live credentials to a tenant’s real accounting system, not something to leave in plaintext.

Starting a connection needs somewhere to send the tenant back to once the OAuth consent screen redirects. Rather than a database row to track “this authorization attempt is legitimate,” it reuses the same purpose-tagged-JWT pattern as the 2FA login challenge (Part 16 §16.2) — a `state` param that’s just a short-lived, signed token:

```
// accounting/accounting-integrations.service.ts
const STATE_PURPOSE = 'accounting-oauth-state';

async getAuthorizeUrl(tenantId: string, userId: string, provider: AccountingProviderType): Promise<string> {
  const client = this.registry.get(provider);
  if (!client.isConfigured()) {
    throw new BadRequestException(`${provider} is not configured on this server — set its CLIENT_ID/CLIENT_SECRET env vars first`);
  }

  const state = await this.jwtService.signAsync(
    { purpose: STATE_PURPOSE, tenantId, userId, provider } satisfies OAuthStatePayload,
    { secret: this.configService.get('jwt', { infer: true }).accessSecret, expiresIn: '10m' },
  );
  return client.getAuthorizeUrl(state);
}
```

The callback is necessarily `@Public()` — the platform redirects the user’s browser straight back with no `Authorization` header, since it’s a plain navigation rather than an XHR call from the app’s own frontend. The signed `state` is what proves the callback belongs to a real, recent, still-valid authorization attempt instead:

```
// accounting/accounting-integrations.controller.ts
// Public: this is where QuickBooks/Xero/Zoho Books redirect the user's
// BROWSER back to after they approve access — no Authorization header is
// present, since it's a plain navigation, not an XHR call from our own
// frontend. The signed `state` param (see getAuthorizeUrl) is what proves
// the request is legitimate instead.
@Public()
@Get('callback/:provider')
@Redirect()
async callback(
  @Param('provider', new ParseEnumPipe(AccountingProviderType)) provider: AccountingProviderType,
  @Query('code') code: string,
  @Query('state') state: string,
  @Query() query: Record<string, string | undefined>,
) {
  try {
    await this.integrationsService.handleCallback(provider, code, state, query);
    return { url: `${corsOrigin}/settings?accounting=connected&provider=${provider}` };
  } catch {
    return { url: `${corsOrigin}/settings?accounting=error&provider=${provider}` };
  }
}
```

```
// accounting/accounting-integrations.service.ts
async handleCallback(provider: AccountingProviderType, code: string, state: string, callbackParams: Record<string,
  string | undefined>): Promise<void> {
  const payload = await this.jwtService.verifyAsync<OAuthStatePayload>(state, {
    secret: this.configService.get('jwt', { infer: true }).accessSecret,
  }); // throws -> caught by the controller's try/catch, redirects to ?accounting=error

  // `provider` is the URL path param the platform redirected to; cross-check
  // against the provider embedded in `state` so a tampered path segment
  // can't attach the wrong provider's tokens to this connection.
  if (payload.purpose !== STATE_PURPOSE || payload.provider !== provider) {
    throw new UnauthorizedException('This authorization link is invalid or has expired');
  }

  const client = this.registry.get(provider);
  const tokens = await client.exchangeCodeForTokens(code, callbackParams);

  await this.prisma.accountingIntegration.upsert({
    where: { tenantId: payload.tenantId },
    create: { tenantId: payload.tenantId, provider, externalId: tokens.externalId,
      accessTokenEncrypted: this.encryptionService.encrypt(tokens.accessToken)!,
      refreshTokenEncrypted: this.encryptionService.encrypt(tokens.refreshToken)!,
      expiresAt: tokens.expiresAt, connectedById: payload.userId },
    update: { /* same shape */ },
  });
}
```

18.3 Three Platforms, One Interface

`AccountingPlatformClientRegistry` is a single lookup table — `AccountingProviderType -> client` — shared by both the OAuth-flow service above and the sync-time router below, so neither duplicates the provider list:


```
// accounting/accounting-platform-client-registry.service.ts
@Injectable()
export class AccountingPlatformClientRegistry {
  private readonly clients: Record<AccountingProviderType, AccountingPlatformClient>;

  constructor(quickbooks: QuickBooksAccountingProvider, xero: XeroAccountingProvider, zohoBooks:
    ZohoBooksAccountingProvider) {
    this.clients = {
      [AccountingProviderType.QUICKBOOKS]: quickbooks,
      [AccountingProviderType.XERO]: xero,
      [AccountingProviderType.ZOHO_BOOKS]: zohoBooks,
    };
  }

  get(provider: AccountingProviderType) { return this.clients[provider]; }
  all() { return Object.values(this.clients); }
}
```

Each platform client implements the same `AccountingPlatformClient` shape — `isConfigured`, `getAuthorizeUrl`, `exchangeCodeForTokens`, `refreshAccessToken`, `syncInvoice`, `syncPayrollExpense` — against genuinely different real APIs. QuickBooks Online is representative of the shape all three take:

```
// accounting/providers/quickbooks-accounting.provider.ts
/**
 * QuickBooks Online, via Intuit's OAuth2 + REST API (v3). Both syncInvoice
 * and syncPayrollExpense post a balanced JournalEntry rather than a Sales
 * Invoice/Bill — a journal entry is the one QBO transaction type that
 * doesn't require a pre-existing Customer/Vendor/Item master record on the
 * tenant's side, which this integration has no way to know or create
 * correctly. Accounts referenced by name are looked up, or created on
 * first use, via the Account query/create endpoints.
 */
@Injectable()
export class QuickBooksAccountingProvider implements AccountingPlatformClient {
  syncPayrollExpense(credentials: ResolvedAccountingCredentials, run: { totals: unknown }) {
    const totals = parsePayrollTotals(run.totals);
    return this.postJournalEntry(credentials, [
      { accountName: 'Payroll Expense', accountType: 'Expense', debit: totals.grossPay, credit: 0 },
      { accountName: 'Payroll Liabilities', accountType: 'Other Current Liability', debit: 0, credit:
        totals.totalDeductions },
      { accountName: 'Bank', accountType: 'Bank', debit: 0, credit: totals.netPay },
    ]);
  }
}
```

That's the actual double-entry shape of a payroll run landing in a real ledger: gross pay debited to an expense account, statutory/other deductions credited to a liabilities account (money the company now owes elsewhere — tax authorities, pension funds), and net pay credited straight out of the bank account. `resolveAccount` looks each named account up by name via QuickBooks' query API and creates it on first use if it doesn't exist yet, so a brand-new QuickBooks company file doesn't need any manual chart-of-accounts setup before the first sync succeeds.

Access tokens expire; `AccountingProviderRouter` is what everything else in the codebase actually calls, and it owns refreshing a token proactively — ahead of its recorded expiry, not reactively after a sync call fails against an already-expired one:

```
// accounting/accounting-provider-router.ts
const REFRESH_SKEW_MS = 2 * 60 * 1000;

/**
 * The single AccountingProvider bound to ACCOUNTING_PROVIDER — everything
 * BillingService/PayrollService already call is unchanged; this router is
 * what makes "one global provider" become "whichever provider this tenant
 * connected, using their own OAuth tokens", entirely behind the same
 * interface a no-op provider used to satisfy.
 */
@Injectable()
export class AccountingProviderRouter implements AccountingProvider {
  private async resolveCredentials(tenantId: string) {
    const integration = await this.prisma.accountingIntegration.findUnique({ where: { tenantId } });
    if (!integration) return null;

    if (integration.expiresAt.getTime() - REFRESH_SKEW_MS > Date.now()) {
      return { client: this.registry.get(integration.provider), credentials: { /* decrypted access token */ } };
    }

    const refreshed = await client.refreshAccessToken(refreshToken);
    await this.prisma.accountingIntegration.update({ where: { tenantId }, data: { /* re-encrypted, new expiresAt */ } });
    return { client, credentials: { accessToken: refreshed.accessToken, externalId: integration.externalId } };
  }
}
```

If a tenant has no integration connected at all, `resolveCredentials` returns `null` and the router responds with `{ success: false, error: 'No accounting integration connected for this tenant' }` rather than throwing — matching `AccountingSyncResult`’s shape everywhere else, and letting the two call sites below stay simple, best-effort `.catch(() => {})` calls with no special-casing for “not connected yet” as an error condition.

18.4 Wired Into Billing and Payroll

Two call sites, both already-existing code from Part 5 and Part 7 that needed zero changes when the router replaced the original stub — the whole point of the extension-point pattern:

```
// billing/billing.service.ts — after a subscription invoice is marked paid
await this.accountingProvider.syncInvoice({
  id: invoice.id, tenantId, amount: invoice.amount, currency: invoice.currency, status: InvoiceStatus.PAID,
}).catch(() => {});
```

```
// payroll/payroll.service.ts — after a payroll run completes
await this.accountingProvider.syncPayrollExpense({
  id: run.id, tenantId, companyId: run.companyId, period: run.period, totals,
}).catch(() => {});
```

`GET /accounting/integrations ( @Roles(ADMIN) )` reports, per platform, whether it’s `configured` (this server has `CLIENT_ID` / `CLIENT_SECRET` set for it at all) and `connected` (this specific tenant has authorized it) — two independent booleans, since a platform can be configured server-wide but not yet connected by this tenant, or in principle configured differently across environments. `AccountingIntegrationsSettings` renders that as one row per platform on the Settings page:

```
// components/settings/AccountingIntegrationsSettings.tsx
const connectMutation = useMutation({
  mutationFn: getAccountingConnectUrl,
  onSuccess: (authorizeUrl) => {
    // Full-page navigation, not a popup — this is a standard OAuth consent
    // redirect, and the platform's own callback lands the browser back on
    // /settings when it's done.
    window.location.href = authorizeUrl;
  },
});
```

`disabled={!status.configured}` on the “Connect” button means an unconfigured platform’s row still renders — so an `ADMIN` can see QuickBooks/Xero/Zoho Books all exist as options — but can’t be clicked into a dead OAuth flow that would fail server-side anyway.

End to end: an `ADMIN` clicks “Connect” next to QuickBooks on `/settings`, gets redirected to Intuit’s real consent screen, approves access, lands back on `/settings?accounting=connected&provider=QUICKBOOKS`, and from that point on, every subscription payment and every completed payroll run posts a real balanced journal entry into that tenant’s actual QuickBooks company file — no polling, no manual export, and no code in `BillingService` or `PayrollService` that had to know which of three platforms, or none, a given tenant uses.

The footer’s `/blog` link has pointed at a “coming soon” placeholder since Part 1. Part 19 finally builds it.

Part 19 — The Blog: Sanity CMS, a Custom Dashboard, and SEO

The footer has linked to `/blog` since Part 1, and it's been a “coming soon” placeholder the entire time. This part builds the real thing — but not the obvious way. The usual Sanity integration means editors log into Sanity Studio, a separate app with its own URL, to write content. Here the write path is `/blog-admin`, a page inside this app's own authenticated dashboard, right next to Audit Log and Settings — Sanity is the storage engine underneath, never a UI anyone has to context-switch into.

19.1 Why a Headless Store, Not a Studio

Sanity's Content Lake is schemaless at the API level — Studio's schema definitions exist purely to give Studio's own editing UI something to render fields against. Skip Studio entirely, and there's no schema to define anywhere: whatever shape a client writes with `_type: 'post'` is the schema, enforced only by whatever code reads it back. That code is this codebase's own TypeScript types, in exactly one place:

```
// api/src/blog/blog.service.ts
interface PostFields {
  title: string;
  slug: string;
  excerpt: string;
  body: string;
  coverImageUrl?: string;
  seoTitle?: string;
  seoDescription?: string;
  countryFocus: string;
  status: 'draft' | 'published';
  publishedAt?: string;
}
```

`body` is plain Markdown text, not Sanity's Portable Text rich-text format — Portable Text needs its own renderer on the frontend and its own editing widget on the backend; a `<textarea>` and `marked.parse()` (Part 19.4) do the same job with zero extra moving parts, at the cost of a slightly less polished editing experience. `coverImageUrl` is a plain external URL string rather than Sanity's binary asset pipeline, for the same reason — there's no Studio asset uploader to feed it from anyway.

19.2 The Trust Boundary — Two Clients, One Project

Every Sanity document is either mutated by someone holding a scoped API token, or read by anyone who knows the project ID — there's no in-between. That maps directly onto the two audiences this feature has, and onto this codebase's standing rule that a write-capable third-party credential lives on the server, never the browser:

```
// api/src/blog/blog.service.ts
/**
 * Owns all write access to the blog's Sanity project – the admin
 * dashboard's only path to Sanity, using a write-scoped API token that
 * never reaches the browser. The public /blog pages read Sanity directly
 * with their own token-less, CDN-backed client instead – this service has
 * nothing to do with public reads.
 */
@Injectable()
export class BlogService {
  private readonly client: SanityClient | null;

  constructor(private readonly configService: ConfigService<AppConfig, true>) {
    const config = this.configService.get('sanity', { infer: true });
    this.client = config.projectId && config.apiToken
      ? createClient({
          projectId: config.projectId,
          dataset: config.dataset,
          apiVersion: '2024-01-01',
          token: config.apiToken,
          useCdn: false, // writes need to see their own just-written state immediately
        })
      : null;
  }

  isConfigured(): boolean {
    return this.client !== null;
  }
}
```

```
// web/lib/sanity-client.ts
/**
 * Public, token-less client – reads only `status == "published"` posts via
 * Sanity's CDN-backed API. This client can't write anything even if it
 * wanted to, since it was never given a token.
 */
function getClient() {
  if (!SANITY_PROJECT_ID) return null;
  return createClient({
    projectId: SANITY_PROJECT_ID,
    dataset: SANITY_DATASET,
    apiVersion: '2024-01-01',
    useCdn: true,
  });
}
```

Both clients degrade the same way every other optional integration in this codebase does — config-gated, not required. `isConfigured()` on the backend feeds `GET /blog-posts/status`, which `/blog-admin` checks before rendering the CRUD table at all; the frontend's `getClient()` returning `null` makes every public read function return an empty result instead of throwing, so `/blog` renders its pre-existing “coming soon” empty state rather than erroring when Sanity isn’t set up yet (\$19.5 covers actually setting it up).

Slugs need to be unique, and Sanity has no unique-field constraint to lean on — enforced with a GROQ query, parameterized (never string-interpolated) since the slug is user-supplied:

```
// api/src/blog/blog.service.ts
private async assertSlugAvailable(client: SanityClient, slug: string, excludeId?: string): Promise<void> {
  const existing = await client.fetch<{ _id: string } | null>(
    `*[ _type == $type && slug == $slug && _id != $excludeId ][0]{ _id }`,
    { type: DOCUMENT_TYPE, slug, excludeId: excludeId ?? '' },
  );
  if (existing) {
    throw new ConflictException(`A post with slug "${slug}" already exists`);
  }
}
```

19.3 The Admin Dashboard

`GET /blog-posts` and friends are gated the same way Audit Log's endpoints are — `@Roles(ADMIN)` plus a dedicated `Permission.BLOG_MANAGE`, added to `ADMIN_ONLY_PERMISSIONS` so `HR` doesn't inherit it automatically the way it inherits everything not explicitly excluded:

```
// api/src/blog/blog.controller.ts
@Controller('blog-posts')
@Roles(Role.ADMIN)
@RequirePermission(Permission.BLOG_MANAGE)
export class BlogController {
  @Get('status')
  status() {
    return { configured: this.blogService.isConfigured() };
  }
  // create, list, get, update, publish, unpublish, remove — one route each
}
```

`/blog-admin` is a single page: a status check, a table of every post (draft and published both — this is the dashboard, not the public listing), and a shared create/edit dialog. The slug field auto-derives from the title but stays editable, and stops auto-deriving the moment someone touches it directly:

```
// web/app/(app)/blog-admin/page.tsx
<Input
  id="post-title"
  value={form.title}
  onChange={(e) => {
    const title = e.target.value;
    setForm((f) => ({
      ...f,
      title,
      slug: slugTouched ? f.slug : slugify(title), // stops once the slug field itself is edited
    }));
  }}
/>
```

Publishing is a separate action from saving — `POST /blog-posts/:id/publish` sets `status` and stamps `publishedAt`; `unpublish` clears both — so a half-finished draft never accidentally goes live just because someone clicked “save”:

```
// api/src/blog/blog.service.ts
async publish(id: string): Promise<BlogPost> {
  const client = this.requireClient();
  await this.get(id);
  const updated = await client
    .patch(id)
    .set({ status: 'published', publishedAt: new Date().toISOString() })
    .commit<PostFields>();
  return toBlogPost(updated);
}
```

19.4 The Public Blog — SSG, ISR, and Structured Data

`/blog` and `/blog/[slug]` are server components that call the public client directly and render at request time, cached and revalidated every 5 minutes (`export const revalidate = 300`) — fast enough that a newly-published post shows up without a redeploy, cached enough that Sanity isn't hit on every single page view:

```
// web/app/(marketing)/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const slugs = await getPublishedSlugs();
  return slugs.map(({ slug }) => ({ slug }));
}

export async function generateMetadata({ params }: { params: Promise<{ slug: string }> }): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPublishedPostBySlug(slug);
  if (!post) return {};
  return {
    title: post.seoTitle || post.title,
    description: post.seoDescription || post.excerpt,
    alternates: { canonical: `/blog/${slug}` },
    openGraph: { type: 'article', publishedTime: post.publishedAt, /* ... */ },
  };
}
```

`seoTitle` / `seoDescription` fall back to `title` / `excerpt` when an editor leaves them blank — every post gets real metadata even if nobody filled in the SEO-specific fields. Markdown renders via `marked`, the one new dependency this feature needed on the frontend:

```
// web/app/(marketing)/blog/[slug]/page.tsx
const bodyHtml = await marked.parse(post.body);
// ...
<div className="prose dark:prose-invert max-w-none" dangerouslySetInnerHTML={{ __html: bodyHtml }} />
```

`prose` / `dark:prose-invert` come from `@tailwindcss/typography` (added via `@plugin '@tailwindcss/typography';` in `globals.css` — Tailwind v4’s CSS-based plugin registration, no `tailwind.config.js` needed) — real heading/list/code-block typography for free, rather than hand-writing it.

Each post gets `Article` JSON-LD (Part 15’s pattern, same `<JsonLd>` component) and its own dynamic Open Graph image — a per-route `opengraph-image.tsx` overriding the site-wide default from Part 15, rendered with the post’s actual title:

```
// web/app/(marketing)/blog/[slug]/opengraph-image.tsx
export default async function Image({ params }: { params: Promise<{ slug: string }> }) {
  const { slug } = await params;
  const post = await getPublishedPostBySlug(slug);
  const title = post?.title ?? APP_NAME;
  return new ImageResponse(/* branded card with the real post title */, { ...size });
}
```

`app/sitemap.ts` (Part 15) pulls every published slug alongside the static marketing routes, so a new post is discoverable the same build/revalidation cycle it’s published in:

```
// web/app/sitemap.ts
const posts = await getPublishedSlugs();
const postEntries = posts.map(({ slug, updatedAt }) => ({
  url: `${SITE_URL}/blog/${slug}`,
  lastModified: new Date(updatedAt),
  changeFrequency: 'monthly' as const,
  priority: 0.7,
}));
```

19.5 One-Time Setup — Creating the Sanity Project

Everything above already works against a real Sanity project — this is the one-time step to make one exist. No Sanity Studio deployment is part of this; the CLI command below only provisions the project itself.

1. **Create a free Sanity account** at sanity.io if you don’t have one.
2. **Provision a project.** From the repo root: `npx sanity@latest init` — pick “Create new project”, give it a name, accept the default dataset name (`production`). This prints a **Project ID**; note it down. (If you’d rather not run the CLI, `sanity.io/manage` → “Create project” gives you the same ID.)

3. **Generate an API token.** In `sanity.io/manage`, open the project → API → Tokens → “Add API token”. Name it (e.g. “PayrollFiti backend”), give it **Editor** permissions (read + write), and copy the token — Sanity only shows it once.
4. **Set the environment variables** — four total, split by which app reads them:

```
# apps/api's .env - write access, server-side only
SANITY_PROJECT_ID=<the project ID from step 2>
SANITY_DATASET=production
SANITY_API_TOKEN=<the token from step 3>

# apps/web's .env - public, read-only, safe to expose to the browser
NEXT_PUBLIC_SANITY_PROJECT_ID=<the same project ID>
NEXT_PUBLIC_SANITY_DATASET=production
```

5. **Restart both apps.** `GET /blog-posts/status` should now report `{ "configured": true }`, `/blog-admin` shows the real dashboard instead of the “Sanity isn’t connected yet” card, and `/blog` will start rendering real posts the moment the first one is published.

19.6 One More Gate: Restricting Blog Management to One Tenant

`@Roles(ADMIN)` on `BlogController` was never enough by itself, and it’s worth being explicit about why: `ADMIN` is a **per-tenant** role. Every customer who signs up for this product gets their own tenant and becomes *that tenant’s* `ADMIN` — the exact same role, in the exact same enum, as the platform owner’s own account. `@Roles(ADMIN)` alone would have let any customer’s admin open `/blog-admin` and publish to the company’s own public marketing blog, since nothing about that decorator distinguishes “an admin of some tenant” from “an admin of *the one tenant that owns this software*.”

The fix reuses the entire existing auth system rather than inventing a new “platform staff” role — the platform owner signs up through the exact same `/signup` flow every customer uses, and one specific tenant ID is designated, by configuration, as the only one allowed past this gate:

```
// common/decorators/platform-only.decorator.ts
export const PLATFORM_ONLY_KEY = 'platformOnly';

/** Restricts a route to the platform-owner's own tenant (PLATFORM_TENANT_ID),
 * regardless of role - distinct from @Roles(), which only checks role
 * *within whichever tenant the request already belongs to*. */
export const PlatformOnly = () => SetMetadata(PLATFORM_ONLY_KEY, true);
```

```
// common/guards/platform-tenant.guard.ts
@Injectable()
export class PlatformTenantGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const isPlatformOnly = this.reflector.getAllAndOverride<boolean>(PLATFORM_ONLY_KEY, [
      context.getHandler(), context.getClass(),
    ]);
    if (!isPlatformOnly) return true;

    const platformTenantId = this.configService.get('platformTenantId', { infer: true });
    const user: AuthenticatedRequestUser | undefined = request.user;

    // Fail closed: an unset PLATFORM_TENANT_ID means nobody passes, not
    // "everyone passes" - the opposite mistake would silently reopen this
    // route to every customer tenant's admin.
    if (!platformTenantId || !user || user.tenantId !== platformTenantId) {
      throw new ForbiddenException('This action is restricted to the platform team');
    }
    return true;
  }
}
```

Registered globally (`APP_GUARD`, alongside `RolesGuard` / `PermissionsGuard` in `app.module.ts`) and applied to `BlogController` alongside its existing `@Roles(ADMIN)` :


```
// blog/blog.controller.ts
@Controller('blog-posts')
@Roles(Role.ADMIN)
@RequirePermission(Permission.BLOG_MANAGE)
@PlatformOnly()
export class BlogController { /* ... */ }
```

Setup: sign up your own account through `/signup` like any customer would, find its tenant id (`SELECT "tenantId" FROM "User" WHERE email = '...'`), and set it:

```
# apps/api's .env
PLATFORM_TENANT_ID=<your own tenant's id>
```

Live-verified against the running dev server: a second, genuinely different tenant's freshly-signed-up `ADMIN` hits `GET /blog-posts/status` and correctly gets `403 { "error": "Forbidden", "message": "This action is restricted to the platform team" }`, while the platform tenant itself passes through untouched. The frontend's `/blog-admin` distinguishes this 403 from an unconfigured-Sanity response, so a customer who stumbles onto the URL sees “you don’t have permission” rather than a misleading “Sanity isn’t connected yet.”

From here on, day-to-day blogging never touches Sanity’s own UI — write, save, and publish posts entirely from `/blog-admin`, the same place every other piece of this app’s content lives, and only the platform’s own account can do it.